

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống thương mại điện tử theo kiến trúc
Microservices**

NGUYỄN ĐỨC BẢO HOÀNG

hoang.ndb227116@sis.hust.edu.vn

Chương trình đào tạo: Toán - Tin

Giảng viên hướng dẫn: TS. Vũ Khánh Quý

Chuyên ngành: Toán - Tin

Khoa: Toán - Tin

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống thương mại điện tử theo kiến trúc
Microservices**

NGUYỄN ĐỨC BẢO HOÀNG

hoang.ndb227116@sis.hust.edu.vn

Chương trình đào tạo: Toán - Tin

Giảng viên hướng dẫn: TS. Vũ Khánh Quý

Chữ kí GVHD

Chuyên ngành: Toán - Tin

Khoa: Toán - Tin

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến các thầy cô trong Khoa Khoa học và Kỹ thuật Máy tính, Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội đã tận tình giảng dạy, truyền đạt kiến thức và định hướng cho em trong suốt quá trình học tập tại trường.

Đặc biệt, em xin gửi lời cảm ơn chân thành nhất đến **TS. [Tên giảng viên hướng dẫn]**, người đã trực tiếp hướng dẫn, định hướng nghiên cứu và dành nhiều thời gian quý báu để góp ý, chỉnh sửa cho đề án này. Những lời khuyên và phản biện của thầy/cô đã giúp em nhìn nhận vấn đề một cách hệ thống và khoa học hơn, từ đó hoàn thiện đề án một cách tốt nhất.

Em cũng xin cảm ơn gia đình và bạn bè đã luôn động viên, ủng hộ và tạo điều kiện thuận lợi để em tập trung vào việc nghiên cứu và hoàn thành đề án đúng thời hạn.

Trong quá trình thực hiện, mặc dù đã cố gắng hết sức nhưng do giới hạn về thời gian và kiến thức, đề án không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ các thầy cô để đề án được hoàn thiện hơn.

Hà Nội, tháng 6 năm 2026

Sinh viên thực hiện

Nguyễn Đức Bảo Hoàng

TÓM TẮT NỘI DUNG ĐỒ ÁN

Đồ án trình bày quá trình phân tích, thiết kế và triển khai một hệ thống thương mại điện tử theo kiến trúc microservices, sử dụng Java 21, Spring Boot 3.5 và Spring Cloud 2025. Hệ thống được phân rã thành 13 microservice độc lập theo nguyên tắc Domain-Driven Design và Bounded Context, mỗi service sở hữu cơ sở dữ liệu và vòng đời triển khai riêng biệt. Giao tiếp kết hợp REST/OpenFeign đồng bộ với Apache Kafka KRaft bất đồng bộ.

Bốn distributed pattern cốt lõi được triển khai đầy đủ: Saga Orchestration với order-service làm orchestrator quản lý giao dịch phân tán; Transactional Outbox và Idempotent Consumer đảm bảo at-least-once delivery kết hợp exactly-once processing effect; CQRS-lite với Elasticsearch làm read model phục vụ tìm kiếm toàn văn. Hai tính năng kỹ thuật nổi bật là cơ chế chống over-sell trong flash sale bằng Redis Lua atomic script, buyer set và reconciliation; và tích hợp cổng thanh toán VNPAY với HMAC-SHA512, IPN callback và timeout scheduler. Bảo mật tập trung qua Keycloak 26 (OAuth 2.0/OIDC, JWT RS256) theo Trusted Subsystem Pattern tại API Gateway.

Ngoài backend, đồ án xây dựng Next.js storefront và Admin Panel, đóng gói bằng Docker Compose và triển khai production-like trên AWS EC2 với Caddy reverse proxy và GitHub Actions/GHCR. Observability stack gồm Prometheus, Grafana và Zipkin. Kết quả thực nghiệm cho thấy toàn bộ 9 nhóm smoke test đạt; kiểm thử k6 catalog, checkout và flash-sale đều đạt mục tiêu về độ trễ và tính đúng đắn; các kịch bản resilience xác nhận khả năng phục hồi khi Kafka hoặc Redis gặp sự cố và bù trừ Saga khi tồn kho không đủ.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)
Nguyễn Đức Bảo Hoàng

ABSTRACT

This thesis presents the analysis, design, and implementation of an e-commerce system based on a microservices architecture, built with Java 21, Spring Boot 3.5, and Spring Cloud 2025. The system is decomposed into 13 independent microservices following Domain-Driven Design and Bounded Context principles, each owning its own database and deployment lifecycle. Service communication combines synchronous REST/OpenFeign with asynchronous Apache Kafka in KRaft mode.

Four core distributed patterns are fully applied: Saga Orchestration with order-service as orchestrator for distributed transaction management; Transactional Outbox and Idempotent Consumer together achieving exactly-once processing effect over at-least-once delivery; and CQRS-lite with Elasticsearch as the read model for full-text search. Two notable technical highlights are a flash sale over-sell prevention mechanism using Redis Lua atomic scripts, a buyer set, and a reconciliation scheduler; and VNPAY payment gateway integration with HMAC-SHA512 verification, IPN callback handling, and timeout recovery. Security is centralized via Keycloak 26 (OAuth 2.0/OIDC, JWT RS256) with the API Gateway applying the Trusted Subsystem Pattern.

The thesis also implements a Next.js storefront and Admin Panel, packaged with Docker Compose and deployed in a production-like environment on AWS EC2 with Caddy reverse proxy and GitHub Actions/GHCR for CI/CD. The observability stack comprises Prometheus, Grafana, and Zipkin. Experimental results show all 9 backend readiness suites passing; k6 load tests for catalog browsing, checkout, and flash-sale spike all meeting their latency and correctness targets; and resilience scenarios confirming Outbox replay after Kafka failure, graceful degradation under Redis outage, and Saga compensation on inventory shortage.

Nguyen Duc Bao Hoang

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	viii
DANH MỤC THUẬT NGỮ	ix
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu đề tài	2
1.3 Phạm vi đề tài	3
1.4 Khảo sát hệ thống tương tự	4
1.5 Định hướng giải pháp.....	6
1.6 Bố cục báo cáo	8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	10
2.1 Kiến trúc Microservices	10
2.1.1 Khái niệm và đặc điểm cốt lõi	10
2.1.2 So sánh với Monolith và SOA	11
2.2 Phân rã hệ thống và Database per Service	12
2.3 Saga Orchestration — giao dịch phân tán nhiều bước	13
2.3.1 Saga Choreography	13
2.3.2 Saga Orchestration	14
2.4 Transactional Outbox và Idempotent Consumer.....	16
2.5 Xử lý đồng thời cao — Redis Lua atomic và mô hình phòng thủ đa lớp	17
2.6 Bảo mật tập trung	18
2.7 Các mẫu thiết kế hỗ trợ: CQRS-lite, Circuit Breaker và Observability	19
2.7.1 CQRS-lite — tách biệt mô hình đọc và ghi cho tìm kiếm	19
2.7.2 Circuit Breaker — ngăn sự cố dây chuyền giữa các service	20
2.7.3 Khả năng quan sát — ba trụ cột giám sát hệ thống phân tán	22

CHƯƠNG 3. GIẢI PHÁP CÔNG NGHỆ.....	24
3.1 Nền tảng phát triển	24
3.2 Hệ sinh thái Spring Cloud	25
3.3 Apache Kafka — Message Broker.....	26
3.4 Hạ tầng dữ liệu	27
3.4.1 PostgreSQL 17.....	27
3.4.2 Redis 8.....	28
3.4.3 Elasticsearch 8	29
3.5 Keycloak — Bảo mật và quản lý định danh	29
3.6 Next.js — Tầng giao diện	30
3.7 Docker — Đóng gói và triển khai.....	30
3.8 Các công nghệ và công cụ hỗ trợ.....	31
CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	32
4.1 Phân tích yêu cầu hệ thống	32
4.2 Mô hình hóa yêu cầu — Biểu đồ Use Case.....	34
4.3 Kiến trúc tổng thể.....	34
4.4 Phân rã miền nghiệp vụ và sở hữu dịch vụ.....	35
4.5 Thiết kế cơ sở dữ liệu	36
4.5.1 PostgreSQL	36
4.5.2 Elasticsearch.....	37
4.5.3 Redis	38
4.6 Thiết kế API và giao tiếp đồng bộ	39
4.7 Thiết kế luồng sự kiện — Kafka Topics	40
4.8 Thiết kế bảo mật.....	42
4.9 Thiết kế máy trạng thái.....	43

4.10 Thiết kế luồng nghiệp vụ chính	44
4.10.1 Luồng thao tác giỏ hàng.....	44
4.10.2 Luồng đăng ký người dùng.....	45
4.10.3 Luồng đặt hàng COD — Saga Orchestration.....	46
4.10.4 Luồng thông báo email	47
4.10.5 Luồng đặt hàng VNPAY	48
4.10.6 Luồng Flash Sale	48
4.10.7 Luồng tìm kiếm sản phẩm (CQRS-lite)	49
4.10.8 Luồng đánh giá sản phẩm	50
4.11 Thiết kế tác vụ nền — Scheduled Jobs	51
CHƯƠNG 5. TRIỂN KHAI VÀ KIỂM THỬ	53
5.1 Môi trường phát triển và cấu trúc dự án	53
5.2 Cài đặt các thành phần backend cốt lõi	54
5.2.1 Lớp điều phối Spring Cloud.....	54
5.2.2 Database per Service và Flyway	54
5.2.3 Saga Orchestration, Outbox và Idempotent Consumer	55
5.2.4 Thanh toán VNPAY	56
5.2.5 Flash sale high-concurrency.....	57
5.3 Cài đặt bảo mật và phân quyền.....	58
5.4 Cài đặt frontend Storefront và Admin Panel	60
5.5 Đóng gói Docker và vận hành local	61
5.6 Triển khai production-like trên AWS EC2	63
5.7 Giám sát và quan sát hệ thống.....	65
5.8 Chiến lược kiểm thử.....	66
5.9 Kết quả kiểm thử hiệu năng	67
5.10 Kết quả kiểm thử khả năng chịu lỗi.....	68

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	70
Kết luận	70
Hướng phát triển	71
TÀI LIỆU THAM KHẢO.....	73
PHỤ LỤC.....	75
A. HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH.....	75
B. ĐẶC TẢ USE CASE.....	78

DANH MỤC HÌNH VẼ

Hình 1.1	Sơ đồ tổng quan sáu trụ cột kỹ thuật của hệ thống	6
Hình 2.1	Saga Choreography Pattern — mỗi service tự phản ứng theo sự kiện	14
Hình 2.2	Saga Orchestration Pattern — orchestrator điều phối tập trung	15
Hình 2.3	Luồng hoạt động của Transactional Outbox Pattern	17
Hình 2.4	Luồng xác thực Trusted Subsystem Pattern: Gateway xác minh JWT và inject header định danh	19
Hình 2.5	Kiến trúc CQRS-lite: PostgreSQL làm mô hình ghi, Elastic-search làm mô hình đọc	20
Hình 2.6	Sơ đồ ba trạng thái Circuit Breaker: CLOSED, OPEN và HALF_OPEN	21
Hình 2.7	Ba trụ cột Observability: Metrics, Distributed Tracing và Log tương quan theo trace context	23
Hình 4.1	Biểu đồ Use Case tổng thể hệ thống	34
Hình 4.2	Sơ đồ thành phần bốn tầng của hệ thống	35
Hình 4.3	Bản đồ ngữ cảnh (Context Map) 13 Bounded Context	36
Hình 4.4	Luồng hoạt động của Transactional Outbox và Idempotent Consumer	38
Hình 4.5	Mô hình ERD các database chính: order_db, payment_db, product_db, inventory_db	38
Hình 4.6	Mô hình khóa Redis: Cart, Cache, Flash Sale	39
Hình 4.7	Sơ đồ phụ thuộc REST/Feign giữa các dịch vụ	40
Hình 4.8	Sơ đồ luồng sự kiện Kafka giữa các dịch vụ	42
Hình 4.9	Luồng xác thực và phân quyền qua API Gateway	43
Hình 4.10	Sơ đồ máy trạng thái đơn hàng	43
Hình 4.11	Sơ đồ máy trạng thái thanh toán và chiến dịch Flash Sale	44
Hình 4.12	Biểu đồ tuần tự luồng thao tác giỏ hàng	45
Hình 4.13	Biểu đồ tuần tự luồng đăng kí và đăng nhập người dùng	45
Hình 4.14	Biểu đồ tuần tự Bước 1–2 Saga: Tạo đơn hàng và đặt chỗ tồn kho qua Transactional Outbox	46
Hình 4.15	Biểu đồ tuần tự luồng đặt hàng COD — Saga Orchestration	47
Hình 4.16	Biểu đồ tuần tự luồng gửi thông báo email qua notification-service	47

Hình 4.17	Biểu đồ tuần tự luồng xử lý thanh toán thất bại và hoàn trả tài nguyên	48
Hình 4.18	Biểu đồ tuần tự luồng đặt hàng VNPAY với xử lý IPN callback	49
Hình 4.19	Biểu đồ tuần tự luồng mua hàng Flash Sale với cơ chế ba lớp bảo vệ	50
Hình 4.20	Biểu đồ tuần tự luồng tìm kiếm sản phẩm theo mô hình CQRS-lite	50
Hình 4.21	Biểu đồ tuần tự luồng đánh giá sản phẩm	51
Hình 5.1	Cấu trúc thư mục monorepo và các module chính của hệ thống	54
Hình 5.2	Eureka dashboard: 16 service đăng ký thành công và ở trạng thái UP	55
Hình 5.3	Luồng bảo mật: xác thực JWT tại Gateway và chèn trusted header cho backend	60
Hình 5.4	Giao diện Storefront: trang chủ, danh sách sản phẩm, giỏ hàng và checkout	60
Hình 5.5	Admin Panel: dashboard, quản lý sản phẩm, tồn kho, đơn hàng và flash sale	61
Hình 5.6	Sơ đồ triển khai Docker Compose ở môi trường local	63
Hình 5.7	Sơ đồ triển khai production-like trên AWS EC2 với Caddy và GHCR	65
Hình 5.8	GitHub Actions build/push image lên GHCR thành công	65
Hình 5.9	Grafana dashboard: Spring Boot Overview, JVM Overview và Saga Overview	66
Hình 5.10	Zipkin: trace của order-service và tiến trình OutboxPoller với outcome SUCCESS	66
Hình 5.11	Kết quả k6 ba kịch bản: catalog soak, checkout stress và flash-sale spike	68
Hình 5.12	Bằng chứng resilience: Kafka outbox replay và Redis degradation/recovery	69

DANH MỤC BẢNG BIỂU

Bảng 1.1	Các thành phần trong phạm vi đề tài	3
Bảng 1.2	Kiến trúc các nền tảng thương mại điện tử lớn	5
Bảng 2.1	So sánh kiến trúc Monolith, SOA và Microservices	11
Bảng 2.2	So sánh Saga Choreography và Saga Orchestration	15
Bảng 3.1	Các thư viện và phiên bản chính	25
Bảng 3.2	Danh sách Kafka topic và luồng dữ liệu tương ứng	27
Bảng 4.1	Yêu cầu chức năng theo miền nghiệp vụ	32
Bảng 4.2	Yêu cầu phi chức năng và chiến lược đáp ứng	33
Bảng 4.3	Sở hữu dịch vụ và giao tiếp	35
Bảng 4.4	Định tuyến API Gateway	39
Bảng 4.5	Bản đồ Kafka Topics	41
Bảng 4.6	Tác vụ nền trong hệ thống	51
Bảng 5.1	Môi trường và công cụ phát triển	53
Bảng 5.2	Map service với lớp lưu trữ chính	55
Bảng 5.3	Chính sách phân quyền tại API Gateway	59
Bảng 5.4	Các module chính của Admin Panel	61
Bảng 5.5	Thành phần triển khai production-like	63
Bảng 5.6	Kết quả backend readiness smoke test	67
Bảng 5.7	Tổng hợp kết quả kiểm thử hiệu năng bằng k6	67
Bảng 5.8	Tổng hợp kết quả kiểm thử khả năng chịu lỗi	69
Bảng A.1	Các URL truy cập hệ thống trên môi trường local	76

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
ACID	Atomicity, Consistency, Isolation, Durability	Tính nguyên tử, nhất quán, cô lập, bền vững
CAP	Consistency, Availability, Partition Tolerance	Định lý nhất quán, khả dụng, chịu phân mảnh
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp và triển khai liên tục
CNTT		Công nghệ thông tin
COD	Cash on Delivery	Thanh toán khi nhận hàng
CORS	Cross-Origin Resource Sharing	Chia sẻ tài nguyên giữa các nguồn gốc khác nhau
CQRS	Command Query Responsibility Segregation	Tách biệt trách nhiệm lệnh và truy vấn
ĐATN		Đồ án tốt nghiệp
DDD	Domain-Driven Design	Thiết kế hướng domain
DTO	Data Transfer Object	Đối tượng truyền dữ liệu
HMAC	Hash-based Message Authentication Code	Mã xác thực thông điệp dựa trên hàm băm
HTTP	HyperText Transfer Protocol	Giao thức truyền siêu văn bản
IPN	Instant Payment Notification	Thông báo thanh toán tức thì
JWT	JSON Web Token	Token web định dạng JSON
OIDC	OpenID Connect	Giao thức xác thực định danh mở
RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
REST	Representational State Transfer	Chuyển trạng thái đại diện
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
TMĐT		Thương mại điện tử
TTL	Time To Live	Thời gian sống của bản ghi
UUID	Universally Unique Identifier	Định danh duy nhất toàn cục

DANH MỤC THUẬT NGỮ

Thuật ngữ	Giải nghĩa
Bounded Context	Ranh giới rõ ràng trong DDD, bên trong đó mọi khái niệm domain có nghĩa nhất quán và duy nhất
Circuit Breaker	Pattern ngăn cascading failure: tự động ngắt mạch khi service downstream liên tục lỗi
Eventual Consistency	Mô hình nhất quán trong hệ phân tán: các bản sao dữ liệu sẽ hội tụ về cùng giá trị sau một khoảng thời gian
Flash Sale	Sự kiện bán hàng số lượng giới hạn trong thời gian ngắn, tạo tải đồng thời cực cao
Idempotent Consumer	Pattern đảm bảo xử lý cùng một message nhiều lần không gây side effect khác so với xử lý một lần
Microservice	Service nhỏ, độc lập, xây dựng xung quanh một Bounded Context, có thể triển khai và scale riêng biệt
Outbox Pattern	Pattern lưu event vào bảng outbox cùng transaction với business data, đảm bảo at-least-once delivery
Over-sell	Tình trạng bán nhiều hơn số lượng hàng thực có, xảy ra do race condition trong hệ thống tải cao
Rate Limiting	Kỹ thuật giới hạn số request một client gửi trong một khoảng thời gian nhất định
Saga Pattern	Pattern xử lý distributed transaction bằng chuỗi local transaction; khi lỗi thực thi compensating transaction
Service Discovery	Cơ chế tự động phát hiện địa chỉ IP:Port của service trong môi trường container động
Trusted Subsystem	Pattern: chỉ API Gateway validate JWT, forward thông tin user qua header tin cậy cho backend

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong thập kỷ qua, thương mại điện tử (TMĐT) tại Việt Nam đã chứng kiến tốc độ tăng trưởng vượt bậc. Theo báo cáo của Hiệp hội Thương mại điện tử Việt Nam (VECOM) năm 2024, doanh thu TMĐT B2C ước đạt 25 tỷ USD, tăng trưởng bình quân trên 25%/năm trong giai đoạn 2020–2024. Các sàn giao dịch lớn như Shopee, Lazada, Tiki đang phục vụ hàng chục triệu người dùng đồng thời, đặc biệt trong các sự kiện Flash Sale và Black Friday — nơi lưu lượng giao dịch có thể tăng đột biến gấp 10–50 lần so với ngày thường.

Thực tế vận hành cho thấy các hệ thống TMĐT xây dựng theo kiến trúc Monolith truyền thống gặp nhiều hạn chế nghiêm trọng khi đối mặt với bài toán quy mô lớn. Thứ nhất, toàn bộ hệ thống phải được mở rộng đồng thời dù chỉ một module đang chịu tải cao, dẫn đến lãng phí tài nguyên và chi phí vận hành. Thứ hai, mọi thay đổi dù nhỏ đều yêu cầu triển khai lại toàn bộ ứng dụng, làm chậm tốc độ phát hành tính năng mới và tăng rủi ro khi cập nhật. Thứ ba, một lỗi cục bộ có thể kéo sập toàn bộ hệ thống do thiếu cơ chế cách ly sự cố (fault isolation). Đối với các nghiệp vụ đặc thù như flash sale — nơi hàng chục nghìn người dùng đồng thời cạnh tranh một số lượng hàng giới hạn trong vài phút — Monolith không thể đáp ứng được yêu cầu về khả năng mở rộng chọn lọc và xử lý đồng thời cao.

Kiến trúc microservices ra đời như một giải pháp cho những hạn chế nêu trên. Tuy nhiên, việc thiết kế và triển khai một hệ thống microservices hoàn chỉnh đòi hỏi nắm vững nhiều mẫu thiết kế phức tạp: từ Saga Pattern cho giao dịch phân tán, Transactional Outbox cho truyền thông điệp tin cậy, đến Idempotent Consumer cho xử lý phân phối at-least-once. Đây là những vấn đề được đề cập rộng rãi trong tài liệu học thuật nhưng ít dự án thực tế tại Việt Nam triển khai một cách đầy đủ và có hệ thống. Bên cạnh đó, việc tích hợp các thành phần hạ tầng như service discovery, API gateway, message broker, distributed caching, full-text search engine và identity provider vào một hệ thống vận hành thống nhất cũng là một thách thức kỹ thuật đáng kể.

Xuất phát từ thực tiễn trên, đề tài đặt ra câu hỏi nghiên cứu: *Làm thế nào để thiết kế và triển khai một hệ thống thương mại điện tử hoàn chỉnh theo kiến trúc microservices, có khả năng mở rộng độc lập theo từng miền nghiệp vụ, chịu lỗi cục bộ, đảm bảo tính nhất quán dữ liệu trong môi trường phân tán, và đặc biệt hỗ trợ tính năng flash sale chống over-sell trong điều kiện tải cao?*

1.2 Mục tiêu đề tài

Mục tiêu tổng quát

Thiết kế, triển khai và đánh giá một hệ thống thương mại điện tử đầy đủ chức năng theo kiến trúc microservices, sử dụng Spring Boot 3.5, Spring Cloud 2025 và Java 21, qua đó minh chứng tính khả thi và hiệu quả của kiến trúc này trong việc giải quyết các bài toán đặc thù của TMĐT: khả năng mở rộng chọn lọc, xử lý đồng thời cao, tính nhất quán dữ liệu phân tán và khả năng quan sát hệ thống từ đầu đến cuối.

Mục tiêu cụ thể

Đề tài đặt ra sáu mục tiêu cụ thể, mỗi mục tiêu đều có thể đo lường và kiểm chứng thông qua mã nguồn, cấu hình hệ thống và kết quả kiểm thử:

- 1. Phân rã hệ thống theo Domain-Driven Design:** Phân tích và thiết kế 13 bounded context độc lập, mỗi context tương ứng với một microservice sở hữu cơ sở dữ liệu, Dockerfile và vòng đời triển khai riêng biệt. Các service bao phủ đầy đủ các miền nghiệp vụ cốt lõi của TMĐT: định danh, người dùng, sản phẩm, tồn kho, giỏ hàng, đơn hàng, thanh toán, khuyến mãi, thông báo, đánh giá, tìm kiếm, nội dung và flash sale.
- 2. Áp dụng các mẫu thiết kế phân tán cốt lõi:** Triển khai bốn mẫu thiết kế quan trọng trong môi trường phân tán — Saga Orchestration (với order-service làm bộ điều phối giao dịch phân tán trong luồng đặt hàng); Transactional Outbox (đảm bảo phát hành sự kiện at-least-once ngay cả khi service gặp sự cố); Idempotent Consumer (đảm bảo hiệu ứng xử lý exactly-once ở phía nhận); và CQRS-lite (tách biệt mô hình đọc với Elasticsearch phục vụ tìm kiếm toàn văn).
- 3. Tích hợp bảo mật tập trung với Keycloak:** Sử dụng Keycloak 26 làm nhà cung cấp định danh, cấu hình OAuth 2.0/OpenID Connect với JWT ký thuật toán RS256 (khóa bất đối xứng). API Gateway đóng vai trò Resource Server duy nhất theo mẫu Trusted Subsystem — xác thực tập trung, chuyển tiếp thông tin định danh qua header tin cậy cho các service backend, loại bỏ việc mỗi service phải gọi lại Keycloak để xác minh token.
- 4. Tích hợp cổng thanh toán VNPAY:** Kết nối với môi trường sandbox của VNPAY, sinh URL thanh toán với chữ ký xác thực HMAC-SHA512, xử lý Return URL và IPN callback, đồng thời triển khai cơ chế tự động hủy thanh toán khi quá thời hạn 30 phút.
- 5. Xây dựng cơ chế Flash Sale chống over-sell:** Thiết kế giải pháp chống tranh

chấp tài nguyên bằng Redis Lua atomic script, buyer set chống mua trùng, cơ chế bù trừ khi tạo đơn thất bại và reconciliation scheduler đối chiếu lại số đơn thực tế. Mục tiêu: trong điều kiện nhiều người dùng đồng thời, số đơn hàng thành công không vượt quá số lượng slot của chiến dịch.

6. **Xây dựng hệ thống quan sát hoàn chỉnh:** Triển khai Prometheus và Grafana cho thu thập và trực quan hóa số liệu (JVM heap, request rate, latency, error rate); Zipkin cho truy vết phân tán theo chuẩn W3C Trace Context; structured logging với khả năng liên kết traceId/spanId, cho phép giám sát và gỡ lỗi hệ thống từ đầu đến cuối.

1.3 Phạm vi đề tài

Phạm vi thực hiện

Hệ thống được xây dựng bao gồm 13 business microservice, ba Spring Cloud infrastructure service, các hệ thống hạ tầng hỗ trợ, Next.js storefront/Admin Panel và bộ kịch bản kiểm thử thực nghiệm. Toàn bộ backend và hạ tầng được đóng gói bằng Docker Compose; môi trường production-like bổ sung frontend container, Caddy reverse proxy và GitHub Actions/GHCR cho quy trình build/push image. Bảng 1.1 liệt kê chi tiết các thành phần trong phạm vi thực hiện của đề tài.

Bảng 1.1: Các thành phần trong phạm vi đề tài

Nhóm	Thành phần	Công nghệ
Spring Cloud	API Gateway	Spring Cloud Gateway
	Discovery Server	Netflix Eureka
	Config Server	Spring Cloud Config
Business Services	identity-service, user-service	Keycloak, PostgreSQL, Kafka
	product-service	PostgreSQL, Redis, Kafka
	inventory-service	PostgreSQL, Kafka
	cart-service	Redis
	order-service, payment-service	PostgreSQL, Kafka, VNPAY
	voucher-service, content-service	PostgreSQL
	notification-service	PostgreSQL, Kafka, Mailpit
	review-service	PostgreSQL, OpenFeign
	search-service	Elasticsearch, Kafka
	flash-sale-service	PostgreSQL, Redis, Kafka
Infrastructure	PostgreSQL 17, Redis 8	Dữ liệu & Cache
	Kafka KRaft 8.2	Message Broker
	Elasticsearch 8.18	Search Engine
	Keycloak 26.6, Mailpit	Xác thực & Email
	Prometheus, Grafana, Zipkin	Observability
Frontend	Storefront	Next.js App Router
	Admin Panel	Server Actions, ROLE_ADMIN guard
Kiểm thử & Triển khai	Unit/Integration Test	JUnit 5, Testcontainers
	Smoke/E2E Test	Bash scripts, Phase 14 demo
	Performance Test	k6
	Docker Compose/AWS	EC2 single-host, Caddy, GHCR

Ngoài phạm vi

Các hạng mục sau nằm ngoài phạm vi của đề tài, xuất phát từ giới hạn về thời gian, tài nguyên và định hướng nghiên cứu của đồ án tốt nghiệp:

- **Frontend cấp sản phẩm thương mại:** Đề tài đã triển khai Next.js storefront và Admin Panel phục vụ demo các luồng chính. Tuy nhiên, các năng lực production như tối ưu UX đa thiết bị, upload file thật qua object storage, workflow kiểm duyệt nâng cao và mobile app riêng chưa nằm trong phạm vi.
- **Kubernetes và Service Mesh:** Triển khai trên Kubernetes với Istio hoặc Linkerd là hướng mở rộng trong tương lai. Phạm vi hiện tại dừng ở Docker Compose single-host, phù hợp với quy mô của một đồ án tốt nghiệp.
- **CI/CD pipeline cấp sản xuất đầy đủ:** GitHub Actions đã hỗ trợ build/push image lên GitHub Container Registry và triển khai lên EC2 ở mức production-like. Phạm vi hiện tại chưa bao gồm canary/blue-green deployment, quét bảo mật bắt buộc, quản lý secret chuyên dụng và triển khai đa môi trường.
- **Thanh toán VNPAY thực tế:** Chỉ sử dụng môi trường sandbox của VNPAY để minh họa luồng thanh toán. Chưa đăng ký tài khoản merchant phục vụ giao dịch thật.
- **Logistics và hoàn tiền:** Các nghiệp vụ vận chuyển, giao hàng và hoàn tiền phức tạp nằm ngoài phạm vi backend e-commerce cốt lõi mà đề tài hướng tới.
- **Triển khai đa vùng:** Hệ thống được triển khai trên một máy ảo EC2 duy nhất tại khu vực ap-southeast-1 (Singapore), chưa hỗ trợ chuyển đổi dự phòng đa vùng địa lý.

1.4 Khảo sát hệ thống tương tự

Để có cơ sở thực tiễn cho các quyết định thiết kế, đề tài tiến hành khảo sát kiến trúc của một số nền tảng TMĐT lớn đang vận hành thành công trên thị trường. Mặc dù các nền tảng này vận hành theo mô hình sàn giao dịch (multi-vendor marketplace), các quyết định kỹ thuật của họ — đặc biệt về kiến trúc microservices, chiến lược dữ liệu, message queue và cơ chế xử lý đồng thời cao — là những bài học giá trị cho bất kỳ hệ thống TMĐT nào, bao gồm cả mô hình B2C mà đề tài hướng tới.

Kiến trúc các nền tảng thương mại điện tử lớn

Bảng 1.2 tổng hợp kết quả khảo sát kiến trúc của ba nền tảng TMĐT hàng đầu tại Việt Nam và Đông Nam Á, tập trung vào các khía cạnh: mô hình kiến trúc, ngôn ngữ lập trình chính, hệ quản trị cơ sở dữ liệu, message queue, giải pháp tìm kiếm và chiến lược xử lý flash sale.

Bảng 1.2: Kiến trúc các nền tảng thương mại điện tử lớn

Tiêu chí	Shopee	Tiki	Lazada
Kiến trúc	Microservices (hàng trăm service)	Microservices kết hợp Monolith	Microservices (hệ sinh thái Alibaba)
Ngôn ngữ chính	Go, Python, Java	Java (Spring), Go	Java
Cơ sở dữ liệu	MySQL sharding, Redis, TiDB	PostgreSQL, MySQL, Redis, ES	MySQL, Redis, HBase
Message Queue	Kafka, RabbitMQ	Kafka	RocketMQ, Kafka
Tìm kiếm	Elasticsearch	Elasticsearch	Elasticsearch
Flash Sale	Redis + Lua + DB constraint	Redis + Queue + DB lock	Alibaba Sentinel

Từ kết quả khảo sát, có thể rút ra năm điểm chung quan trọng trong kiến trúc kỹ thuật của các nền tảng TMĐT lớn, qua đó củng cố cho các quyết định công nghệ của đề tài: (1) kiến trúc microservices được sử dụng để phân rã nghiệp vụ phức tạp thành các dịch vụ độc lập; (2) chiến lược polyglot persistence — kết hợp nhiều loại cơ sở dữ liệu cho các mục đích khác nhau — là cách tiếp cận phổ biến; (3) message queue đóng vai trò trung tâm trong xử lý bất đồng bộ và giao tiếp giữa các dịch vụ; (4) Elasticsearch là lựa chọn thống nhất cho bài toán tìm kiếm toàn văn; và (5) Redis là thành phần then chốt trong kiến trúc xử lý flash sale, thường được kết hợp với các cơ chế bảo vệ ở tầng cơ sở dữ liệu.

Tài liệu và dự án tham khảo

Bên cạnh việc khảo sát xu hướng kiến trúc, đề tài tham khảo các tài liệu và dự án sau trong quá trình thiết kế và triển khai:

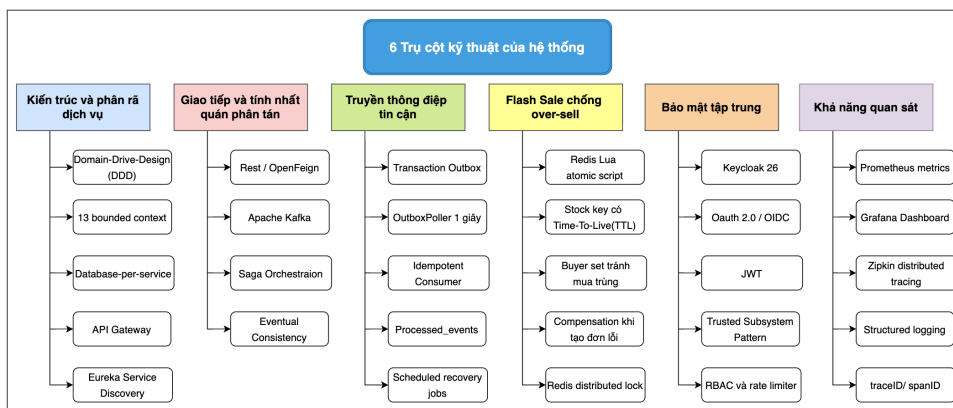
- **Microservices Patterns (Chris Richardson):** Đây là tài liệu tham khảo nền tảng về các mẫu thiết kế trong kiến trúc microservices, bao gồm Saga, Transactional Outbox, CQRS, API Gateway và Database per Service. Trang web microservices.io cung cấp bộ pattern catalog đầy đủ kèm ví dụ minh họa, là nguồn tham khảo chính cho các quyết định thiết kế phân tán trong đề tài.
- **Building Microservices (Sam Newman, O'Reilly):** Cuốn sách kinh điển về thiết kế và triển khai microservices, cung cấp nền tảng lý thuyết vững chắc về bounded context, database per service, giao tiếp đồng bộ/bất đồng bộ, resilience pattern và chiến lược triển khai. Ấn bản thứ hai (2021) cập nhật các thực tiễn mới về container hóa và observability.
- **Domain-Driven Design (Eric Evans):** Cuốn sách đặt nền móng cho phương

phápDDD, cung cấp các khái niệm cốt lõi như Bounded Context, Ubiquitous Language, Aggregate và Context Map — những công cụ được sử dụng trực tiếp để phân rã hệ thống thành 13 microservice trong đề tài.

- **Spring Boot & Spring Cloud Official Documentation:** Tài liệu chính thức của Spring cung cấp hướng dẫn chi tiết về cách cấu hình Spring Cloud Gateway, Eureka Service Discovery, Config Server, OpenFeign, Spring Kafka và Spring Security OAuth2 Resource Server. Đây là nguồn tham khảo kỹ thuật chính trong quá trình cài đặt.
- **Các dự án e-commerce mã nguồn mở trên GitHub:** Nhiều dự án e-commerce mã nguồn mở như Saleor, Medusa và Spree cung cấp ý tưởng tham khảo về cấu trúc thư mục, thiết kế REST API và tổ chức dữ liệu. Tuy nhiên, hầu hết các dự án này chưa triển khai đầy đủ các mẫu thiết kế phân tán phức tạp như Saga Orchestration, Transactional Outbox với Kafka, Idempotent Consumer hay cơ chế Redis Lua atomic counter cho flash sale — những điểm mà đề tài này hướng tới giải quyết một cách trọn vẹn.

1.5 Định hướng giải pháp

Dựa trên phân tích bài toán, khảo sát thực tiễn và nghiên cứu lý thuyết, đề tài xác định sáu định hướng kỹ thuật then chốt, chi phối toàn bộ quá trình thiết kế và triển khai hệ thống.



Hình 1.1: Sơ đồ tổng quan sáu trụ cột kỹ thuật của hệ thống

Kiến trúc và phân rã dịch vụ. Áp dụng phương pháp Domain-Driven Design với Bounded Context làm đơn vị phân rã, hệ thống được chia thành 13 microservice tương ứng với 13 miền nghiệp vụ độc lập. Mỗi service sở hữu cơ sở dữ liệu riêng và không được phép truy cập trực tiếp vào dữ liệu của service khác — mọi tương tác dữ liệu phải thông qua API hoặc sự kiện. API Gateway đóng vai trò điểm vào duy nhất cho mọi yêu cầu từ phía client, kết hợp với Eureka Service Discovery để định tuyến động và Config Server để quản lý cấu hình tập trung, đảm bảo tính nhất

quán về mặt vận hành trên toàn hệ thống.

Giao tiếp và tính nhất quán phân tán. Hệ thống kết hợp hai phương thức giao tiếp: REST + OpenFeign cho các truy vấn đồng bộ cần phản hồi tức thời (xác minh giá sản phẩm, kiểm tra tồn kho) và Apache Kafka KRaft cho các tương tác bất đồng bộ không yêu cầu phản hồi ngay (cập nhật trạng thái đơn hàng, gửi thông báo). Saga Orchestration Pattern được áp dụng để quản lý giao dịch phân tán trong luồng đặt hàng, với order-service đóng vai trò bộ điều phối trung tâm, đảm bảo tính nhất quán cuối cùng thay vì two-phase commit. Toàn bộ 13 Kafka topic được thiết kế rõ ràng về producer, consumer và cấu trúc payload.

Truyền thông điệp tin cậy. Transactional Outbox Pattern được triển khai tại order-service và payment-service để giải quyết bài toán dual-write: sự kiện được ghi vào bảng outbox hoặc payment_outbox trong cùng một giao dịch cơ sở dữ liệu với dữ liệu nghiệp vụ. OutboxPoller (chu kỳ quét 1 giây) định kỳ đọc các sự kiện chưa được phát hành bằng truy vấn FOR UPDATE SKIP LOCKED và gửi lên Kafka, đảm bảo at-least-once delivery ngay cả khi service gặp sự cố giữa chừng. Phía nhận, Idempotent Consumer sử dụng bảng processed_events để phát hiện và loại bỏ sự kiện trùng lặp, đạt được hiệu ứng exactly-once processing ở tầng ứng dụng. Sáu scheduled job chạy nền đảm bảo hệ thống tự phục hồi sau các tình huống bất thường: ReservationExpiryScheduler, PaymentTimeoutScheduler, OutboxPoller cho order và payment, CampaignScheduler và ReconciliationScheduler.

Flash sale chống over-sell. Giải pháp được thiết kế theo mô hình phòng thủ đa lớp: lớp thứ nhất sử dụng Redis Lua atomic script để thực hiện thao tác kiểm tra buyer set, kiểm tra tồn kho và giảm slot trong bộ nhớ một cách nguyên tử; lớp thứ hai sử dụng logic tạo đơn và cơ chế bù trừ để hoàn slot khi quá trình tạo đơn thất bại; lớp thứ ba sử dụng ReconciliationScheduler để đối chiếu soldCount với số đơn flash-sale thực tế trong order-service. Cơ chế distributed lock dựa trên Redis đảm bảo chỉ một instance của flash-sale-service thực thi CampaignScheduler tại mỗi thời điểm, còn Redis stock key và buyers key được đặt TTL theo thời điểm kết thúc chiến dịch.

Bảo mật tập trung. Toàn bộ hoạt động xác thực và phân quyền được xử lý tại API Gateway thông qua Keycloak 26 theo mẫu Trusted Subsystem Pattern: Gateway là điểm duy nhất xác minh JWT (RS256), sau đó chuyển tiếp thông tin định danh đã được xác thực qua ba header tin cậy — X-User-Id, X-User-Roles và X-User-Email — cho các service backend. Các service nội bộ hoàn toàn tin tưởng các header này mà không cần gọi lại Keycloak, giúp giảm độ trễ và giảm tải

cho Identity Provider. Phân quyền dựa trên mô hình RBAC với hai vai trò chính: ROLE_USER (khách hàng) và ROLE_ADMIN (quản trị viên). Các endpoint nhạy cảm được bảo vệ bổ sung bằng Redis token bucket rate limiter: 10 request/giây cho đặt hàng và 3 request/giây cho mua flash sale.

Khả năng quan sát. Hệ thống quan sát được xây dựng trên cả ba trụ cột. Về metrics: Prometheus định kỳ thu thập số liệu từ các endpoint `/actuator/prometheus`, Grafana trực quan hóa thông qua các dashboard được provision tự động cho JVM, Spring Boot service metrics và tổng quan Saga. Về distributed tracing: Zipkin tiếp nhận và hiển thị vết (trace) từ tất cả service theo chuẩn W3C Trace Context, mỗi request được gán một traceId duy nhất và mỗi đoạn xử lý trong một service là một span, cho phép tái tạo toàn bộ hành trình của request qua hệ thống. Về logging: structured logging với Logback pattern chứa traceId và spanId trong mỗi dòng log, cho phép liên kết chéo giữa logs và traces khi cần điều tra sự cố.

1.6 Bố cục báo cáo

Báo cáo được tổ chức thành năm chương chính, một chương kết luận và hai phụ lục. Nội dung các chương được sắp xếp theo logic từ tổng quan đến chi tiết, từ lý thuyết đến thực hành, từ thiết kế đến kiểm chứng:

Chương 1 — Giới thiệu đề tài trình bày bối cảnh, câu hỏi nghiên cứu, mục tiêu, phạm vi, khảo sát hệ thống tương tự và định hướng giải pháp.

Chương 2 — Cơ sở lý thuyết trình bày nền tảng lý thuyết về kiến trúc microservices, các mẫu thiết kế phân tán (Saga, Outbox, Idempotent Consumer, CQRS), cơ chế đảm bảo độ tin cậy, bảo mật phân tán và khả năng quan sát.

Chương 3 — Giải pháp công nghệ phân tích và lý giải việc lựa chọn từng công nghệ trong hệ thống, từ nền tảng phát triển (Java 21, Spring Boot, Spring Cloud), hạ tầng dữ liệu (PostgreSQL, Redis, Elasticsearch, Kafka), đến bảo mật (Keycloak), thanh toán (VNPAY), giao diện (Next.js), triển khai (Docker, AWS, CI/CD) và công cụ kiểm thử.

Chương 4 — Phân tích và Thiết kế hệ thống trình bày kết quả phân tích yêu cầu và thiết kế hệ thống, bao gồm kiến trúc tổng thể, thiết kế cơ sở dữ liệu, API, luồng sự kiện, bảo mật, máy trạng thái và các luồng nghiệp vụ chính.

Chương 5 — Cài đặt, Triển khai, Kiểm thử và Đánh giá mô tả quá trình hiện thực hóa thiết kế và triển khai production-like trên AWS EC2, đồng thời trình bày kết quả kiểm thử chức năng, hiệu năng và khả năng chịu lỗi của hệ thống.

Kết luận và Hướng phát triển tổng kết kết quả đạt được, phân tích hạn chế và đề xuất các hướng phát triển trong tương lai.

Phụ lục A cung cấp hướng dẫn cài đặt và vận hành hệ thống. **Phụ lục B** trình bày đặc tả chi tiết các use case chính.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này trình bày bảy nền tảng lý thuyết trực tiếp định hình các quyết định thiết kế then chốt của hệ thống. Mỗi mục xuất phát từ một bài toán kỹ thuật cụ thể mà hệ thống phải giải quyết, sau đó phân tích nguyên lý và đánh giá cách áp dụng: (1) kiến trúc microservices là nền tảng triển khai toàn bộ hệ thống; (2) phân rã hệ thống và cách ly dữ liệu dẫn đến bài toán nhất quán phân tán; (3) giao dịch nhiều bước xuyên service đòi hỏi Saga Orchestration; (4) giao tiếp bất đồng bộ tin cậy cần Transactional Outbox và Idempotent Consumer; (5) flash sale đồng thời cao yêu cầu thao tác nguyên tử trên Redis; (6) xác thực phân tán cần mô hình Trusted Subsystem; (7) các mẫu hỗ trợ bổ sung hoàn thiện độ tin cậy và khả năng quan sát.

2.1 Kiến trúc Microservices

2.1.1 Khái niệm và đặc điểm cốt lõi

Kiến trúc microservices là phong cách kiến trúc phần mềm cấu trúc một ứng dụng thành tập hợp các service nhỏ, triển khai độc lập, mỗi service chạy trong tiến trình riêng và giao tiếp qua cơ chế nhẹ — thường là HTTP/REST hoặc message broker. Mỗi service được tổ chức xung quanh một khả năng nghiệp vụ (business capability) cụ thể và có thể được triển khai độc lập bằng hệ thống tự động hóa [1], [2].

Sáu đặc điểm cốt lõi phân biệt microservices với các kiểu kiến trúc khác:

- **Single Responsibility:** mỗi service đảm nhiệm đúng một bounded context nghiệp vụ, không chồng lấn trách nhiệm. Ranh giới rõ ràng giúp nhóm phát triển hiểu toàn bộ service và thay đổi mà không ảnh hưởng service khác.
- **Loose Coupling, High Cohesion:** các service không biết chi tiết nội bộ của nhau (loose coupling), mọi giao tiếp đều qua API/sự kiện được công bố công khai; trong khi đó logic liên quan được nhóm lại trong cùng service (high cohesion).
- **Decentralized Data Management:** mỗi service sở hữu và quản lý kho dữ liệu riêng (Database per Service). Không có service nào được phép đọc trực tiếp vào database của service khác.
- **Independent Deployability:** mỗi service có pipeline CI/CD riêng, có thể cập nhật, rollback và scale mà không cần phối hợp với service khác.
- **Technology Heterogeneity:** mỗi service được phép chọn ngôn ngữ lập trình, framework và loại cơ sở dữ liệu phù hợp nhất với đặc thù nghiệp vụ của nó (Polyglot Persistence).
- **Design for Failure:** trong hệ thống phân tán, sự cố mạng, timeout và lỗi từng

phần là tất yếu. Mỗi service phải được thiết kế giả định service lân cận có thể không sẵn sàng và xử lý gracefully qua Circuit Breaker, Retry và fallback.

2.1.2 So sánh với Monolith và SOA

Để đánh giá đúng lý do lựa chọn microservices, cần đối chiếu với hai kiểu kiến trúc tiền nhiệm là Monolith và SOA (Service-Oriented Architecture). Bảng 2.1 tổng hợp sự khác biệt theo các chiều quan trọng:

Bảng 2.1: So sánh kiến trúc Monolith, SOA và Microservices

Tiêu chí	Monolith	SOA	Microservices
Đơn vị triển khai	Toàn bộ ứng dụng trong một artifact duy nhất	Nhóm chức năng, thường là WAR/EAR lớn	Service đơn lẻ, JAR độc lập, container riêng
Cơ sở dữ liệu	Shared database duy nhất	Shared hoặc ít database dùng chung	Database per Service — mỗi service sở hữu DB riêng
Giao tiếp nội bộ	Gọi hàm trong bộ nhớ (in-process)	SOAP/XML qua Enterprise Service Bus (ESB)	REST, gRPC hoặc message broker nhẹ (Kafka)
Công nghệ	Bị ràng buộc một stack duy nhất	Chủ yếu một ngôn ngữ, ESB là nút thắt	Tự do — polyglot theo từng service
Khả năng mở rộng	Scale toàn bộ ứng dụng, lãng phí tài nguyên	Scale theo nhóm service	Scale từng service độc lập theo nhu cầu
Tổ chức nhóm	Nhóm lớn, chung codebase, dễ xung đột	Nhóm trung bình theo layer	Nhóm nhỏ, mỗi nhóm sở hữu vài service
Độ phức tạp vận hành	Thấp — một artifact, một deployment	Trung bình — ESB là điểm phức tạp	Cao — phân tán, cần observability, service mesh
Phù hợp với	MVP, nhóm nhỏ, ứng dụng đơn giản	Enterprise legacy, tích hợp hệ thống cũ	Hệ thống lớn, nhiều nhóm, yêu cầu scale cao

Kiến trúc Monolith đơn giản trong triển khai và debug — toàn bộ code chạy trong cùng một tiến trình nên không có độ trễ mạng nội bộ, không cần service discovery và transaction ACID hoạt động tự nhiên. Tuy nhiên, khi hệ thống tăng trưởng, monolith gặp hai vấn đề không thể tránh: mọi thay đổi dù nhỏ đều yêu cầu rebuild và redeploy toàn bộ ứng dụng; scale chỉ có thể thực hiện theo chiều ngang toàn bộ ứng dụng, không thể scale riêng module nào đang là bottleneck.

SOA giải quyết một phần bằng cách chia nhỏ thành các service lớn hơn, nhưng phụ thuộc nặng vào ESB như một bus trung tâm. ESB trở thành điểm tập trung cho routing, transformation và orchestration — khi ESB gặp sự cố hoặc trở thành bottleneck, toàn bộ hệ thống bị ảnh hưởng. Ngoài ra, shared database trong SOA

vẫn tạo ra coupling ở tầng dữ liệu ngay cả khi logic đã được tách ra.

Microservices đẩy nguyên tắc tách biệt đến mức tối đa: mỗi service là một đơn vị triển khai độc lập hoàn toàn, từ code đến database. Đánh đổi là độ phức tạp vận hành tăng đáng kể — phải giải quyết các bài toán phân tán như nhất quán dữ liệu, distributed tracing và service-to-service authentication mà monolith không bao giờ gặp phải. Các phần tiếp theo của chương phân tích cụ thể từng bài toán đó.

Hệ thống được xây dựng theo kiến trúc microservices với 13 business service và 3 service hạ tầng. Mỗi business service tương ứng với một domain nghiệp vụ tách biệt: định danh (identity-service), hồ sơ người dùng (user-service), catalog sản phẩm (product-service), tìm kiếm (search-service), giỏ hàng (cart-service), đặt hàng (order-service), thanh toán (payment-service), kho hàng (inventory-service), voucher (voucher-service), đánh giá (review-service), nội dung (content-service), thông báo (notification-service) và flash sale (flash-sale-service). Ba service hạ tầng — API Gateway, Config Server và Service Registry — hỗ trợ vận hành mà không chứa logic nghiệp vụ. Mỗi service được đóng gói thành Docker container riêng với cơ sở dữ liệu độc lập, pipeline CI/CD riêng và giao tiếp với nhau qua REST (đồng bộ) hoặc Kafka (bất đồng bộ). Quyết định này dẫn trực tiếp đến các thách thức kỹ thuật được giải quyết trong các phần tiếp theo.

2.2 Phân rã hệ thống và Database per Service

Hệ thống thương mại điện tử phức tạp không thể được mô hình hóa bằng một mô hình dữ liệu thống nhất duy nhất, vì các miền nghiệp vụ khác nhau nhìn nhận cùng một khái niệm theo những cách không tương thích. Ví dụ, khái niệm *Đơn hàng* trong miền đặt hàng mang ý nghĩa về thông tin khách hàng, danh sách sản phẩm và phương thức thanh toán; trong miền kho hàng, chính đơn hàng đó lại là tập hợp các mặt hàng cần trừ tồn kho. Domain-Driven Design (DDD) giải quyết mâu thuẫn này thông qua khái niệm **Bounded Context** [3]: một ranh giới tường minh trong đó một mô hình nghiệp vụ cụ thể có ý nghĩa nhất quán và không cần giải thích thêm.

Áp dụng DDD vào hệ thống, 13 Bounded Context được xác định tương ứng với 13 microservice: mỗi service sở hữu mô hình nghiệp vụ riêng, không chia sẻ lớp domain với service khác. Ranh giới giữa các context được thể hiện qua nguyên tắc *Database per Service* [4]: mỗi service chỉ được phép đọc và ghi vào cơ sở dữ liệu mà nó sở hữu, mọi trao đổi dữ liệu chéo buộc phải đi qua API hoặc sự kiện được công bố công khai. Nguyên tắc này loại bỏ hoàn toàn sự kết dính schema giữa các service — một trong những nguyên nhân hàng đầu khiến monolith khó phát triển độc lập.

Polyglot Persistence — Database per Service mở ra khả năng mỗi service chọn loại cơ sở dữ liệu phù hợp nhất với kiểu truy cập của mình thay vì ép tất cả vào một mô hình duy nhất. Trong hệ thống, lựa chọn được thực hiện dựa trên đặc thù truy cập: **PostgreSQL 17** (10 database riêng biệt) cho các service cần tính nhất quán mạnh, ràng buộc khóa ngoại và giao dịch ACID; **Redis 8** cho cart-service (key-value với TTL) và flash-sale-service (bộ đếm nguyên tử); **Elasticsearch 8.18** là kho đọc của search-service, tối ưu cho tìm kiếm toàn văn đa tiêu chí; **Apache Kafka KRaft** đóng vai trò message broker bền vững, đảm bảo giao tiếp bất đồng bộ tin cậy giữa các service.

Database per Service tạo ra một hệ quả không thể tránh: khi một nghiệp vụ yêu cầu cập nhật dữ liệu trên nhiều service (ví dụ: đặt hàng đồng thời phải cập nhật order_db, inventory_db, voucher_db và payment_db), không thể dùng giao dịch ACID truyền thống để bao phủ tất cả vì mỗi database thuộc về một service độc lập. Đây là bài toán nhất quán phân tán [5], được giải quyết bằng các mẫu thiết kế ở các phần tiếp theo.

2.3 Saga Orchestration — giao dịch phân tán nhiều bước

Giao dịch ACID của cơ sở dữ liệu quan hệ đảm bảo rằng một tập hợp thao tác trên cùng một database hoặc cùng thành công hoặc cùng bị hủy. Tuy nhiên, giao thức Two-Phase Commit (2PC) — cách mở rộng ACID sang nhiều database — có hai vấn đề nghiêm trọng trong kiến trúc microservices: 2PC yêu cầu coordinator trung tâm giữ khóa trên tất cả participant trong suốt giao dịch, tạo điểm nghẽn hiệu năng và điểm lỗi đơn; đồng thời khả năng hỗ trợ 2PC phụ thuộc vào driver cơ sở dữ liệu và với Redis, Elasticsearch thì không tồn tại.

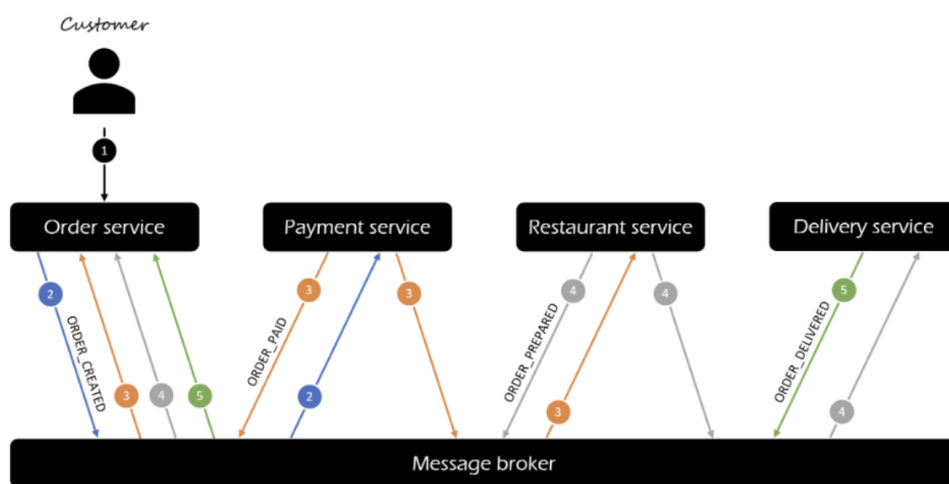
Saga Pattern [4], [6] là giải pháp thay thế: phân rã giao dịch phân tán lớn thành chuỗi các giao dịch cục bộ $T_1, T_2, \dots, T_n$, mỗi giao dịch thực thi trong phạm vi một service duy nhất và commit ngay khi hoàn thành. Với mỗi T_i thành công, hệ thống phải xác định giao dịch bù trừ C_i có khả năng hoàn tác tác động của T_i ; nếu T_k thất bại, hệ thống thực thi $C_{k-1}, C_{k-2}, \dots, C_1$ theo thứ tự ngược. Điểm khác biệt so với ACID là Saga không cung cấp *isolation*: dữ liệu tạm thời không nhất quán giữa các service nhưng hội tụ về đúng sau khi chuỗi Saga hoàn thành (*eventual consistency*).

2.3.1 Saga Choreography

Trong biến thể Choreography, không có thành phần điều phối trung tâm: mỗi service tự lắng nghe sự kiện từ Kafka và tự quyết định hành động kế tiếp dựa trên loại sự kiện nhận được. Ví dụ, nếu áp dụng Choreography cho luồng đặt hàng, inventory-service sẽ lắng nghe order-created, xử lý đặt chỗ tồn kho rồi phát inventory-updated; tiếp đó payment-service lắng nghe inventory-

updated và tự tiến hành thanh toán mà không cần ai ra lệnh. Mỗi service chỉ biết sự kiện nó quan tâm, không biết gì về các service khác trong luồng — tạo ra tính kết dính rất lỏng (loose coupling).

Ưu điểm của Choreography là phù hợp với luồng đơn giản, ít bước, ít thay đổi: không cần thành phần trung tâm, mỗi service hoạt động hoàn toàn độc lập. Nhược điểm rõ ràng khi luồng phức tạp: trạng thái Saga bị phân tán khắp các service, không có điểm nào có bức tranh toàn cảnh về luồng đang ở bước nào; xử lý bù trừ trở nên khó kiểm soát vì không có orchestrator chủ động phát lệnh theo thứ tự ngược.

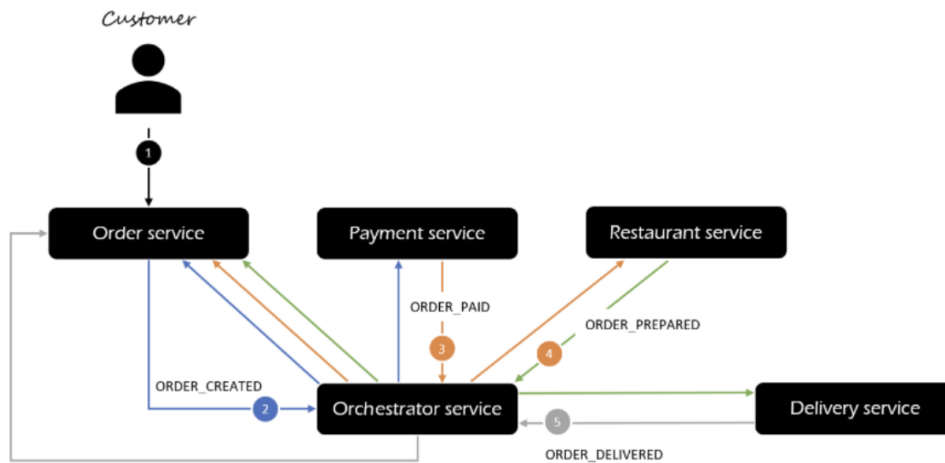


Hình 2.1: Saga Choreography Pattern — mỗi service tự phản ứng theo sự kiện

2.3.2 Saga Orchestration

Trong biến thể Orchestration, một thành phần trung tâm (orchestrator) nắm toàn bộ luồng và ra lệnh tuần tự đến từng service tham gia. Orchestrator biết chính xác Saga đang ở bước nào, quyết định bước tiếp theo và khi có sự cố, chủ động phát lệnh bù trừ theo thứ tự ngược.

Ưu điểm của Orchestration là toàn bộ trạng thái Saga được tập trung tại một nơi nên dễ theo dõi, kiểm thử và mở rộng: thêm hoặc sửa một bước nghiệp vụ chỉ cần thay đổi orchestrator thay vì sửa nhiều service. Nhược điểm là orchestrator là thành phần phức tạp hơn và cần được thiết kế cẩn thận để không trở thành điểm nghẽn hay điểm lỗi đơn của toàn luồng.



Hình 2.2: Saga Orchestration Pattern — orchestrator điều phối tập trung

Sau khi đã phân tích riêng từng biến thể, Bảng 2.2 đối chiếu trực tiếp Choreography và Orchestration theo các tiêu chí then chốt để làm cơ sở cho lựa chọn của hệ thống.

Bảng 2.2: So sánh Saga Choreography và Saga Orchestration

Đặc điểm	Choreography	Orchestration
Cơ chế điều phối	Mỗi service tự lắng nghe sự kiện và tự quyết định hành động kế tiếp	Orchestrator trung tâm ra lệnh tuần tự đến từng service
Tính kết dính	Lỏng — các service không biết về nhau	Chặt hơn — orchestrator nắm toàn bộ luồng
Trạng thái Saga	Phân tán khắp các service, khó theo dõi	Tập trung tại orchestrator, dễ kiểm tra
Xử lý bù trừ	Phức tạp — khó xác định điểm kích hoạt bù trừ	Orchestrator chủ động phát lệnh bù trừ theo thứ tự ngược
Thêm bước nghiệp vụ	Cần sửa nhiều service	Chỉ sửa orchestrator
Phù hợp khi	Luồng đơn, ít bước, ít thay đổi	Luồng phức tạp, nhiều bước, cần truy vết tập trung

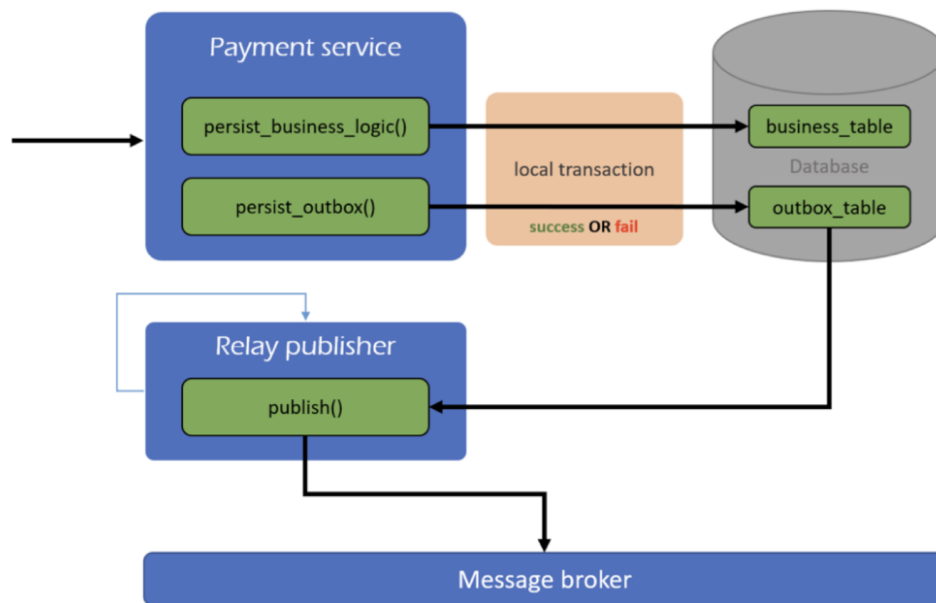
Luồng đặt hàng trong hệ thống trải qua 6–8 bước liên quan đến năm service khác nhau (product, inventory, voucher, payment và notification) ngoài chính order-service điều phối, với hai nhánh rẽ phụ thuộc vào phương thức thanh toán (COD hoặc VNPAY) và nhiều điểm có thể thất bại. Orchestration là lựa chọn phù hợp hơn vì trạng thái Saga được tập trung tại order-service, logic bù trừ được xử lý một chỗ và thêm hoặc sửa bước nghiệp vụ chỉ cần thay đổi một nơi. Order-service đóng vai trò orchestrator duy nhất và duy trì vòng đời đơn hàng qua bốn trạng thái: PENDING (đang chờ đặt chỗ tồn kho), STOCK_RESERVED (tồn kho đã đặt, chờ thanh toán), CONFIRMED (kết thúc thành công) và CANCELLED (thất bại, orchestrator phát lệnh bù trừ giải phóng tồn kho và hoàn trả voucher). Mỗi chuyển trạng thái

được kích hoạt bởi sự kiện Kafka cụ thể, đảm bảo mọi đường chuyển đổi bất hợp lệ bị từ chối ở tầng code.

2.4 Transactional Outbox và Idempotent Consumer

Sau khi order-service tạo đơn hàng trong cơ sở dữ liệu, nó cần phát sự kiện `order-created` lên Kafka để inventory-service tiến hành đặt chỗ tồn kho. Cách tiếp cận thông thường là ghi dữ liệu vào database rồi ngay sau đó gọi `kafkaTemplate.send()` — đây là vấn đề **dual-write**. Nếu service gặp sự cố sau khi commit transaction database nhưng trước khi kịp gọi Kafka, sự kiện sẽ không bao giờ được phát: đơn hàng tồn tại trong database với trạng thái `PENDING` nhưng inventory-service không nhận được yêu cầu đặt chỗ. Ngược lại, nếu Kafka nhận sự kiện nhưng database bị rollback, inventory sẽ đặt chỗ cho đơn hàng không tồn tại. Cả hai tình huống đều dẫn đến mất đồng bộ không phát hiện được bằng cách quan sát từng thành phần riêng lẻ.

Transactional Outbox Pattern [4] giải quyết dual-write bằng cách khai thác ACID của database đã có: thay vì gọi Kafka trực tiếp, service ghi bản ghi sự kiện vào bảng `outbox` trong **cùng một giao dịch database** với dữ liệu nghiệp vụ. Một tiến trình nền (`OutboxPoller`) chạy định kỳ quét bảng `outbox`, đọc các sự kiện chưa gửi và phát lên Kafka; sau khi Kafka xác nhận, bản ghi được đánh dấu `processed = true`. Sơ đồ luồng được minh họa ở Hình 2.3. `OutboxPoller` sử dụng `SELECT ... FOR UPDATE SKIP LOCKED`: `FOR UPDATE` khóa các hàng được chọn ngăn instance khác lấy trùng; `SKIP LOCKED` bỏ qua hàng đã bị khóa thay vì chờ — cho phép nhiều instance xử lý song song. Cơ chế này cung cấp **at-least-once delivery**: nếu service gặp sự cố sau khi Kafka nhận nhưng trước khi `outbox` được đánh dấu, Poller sẽ phát lại sự kiện sau khi khởi động lại.



Hình 2.3: Luồng hoạt động của Transactional Outbox Pattern

Idempotent Consumer đảm bảo rằng việc xử lý cùng một thông điệp nhiều lần không tạo ra tác động phụ khác biệt so với chỉ xử lý một lần. Cơ chế dựa trên bảng `processed_events` với khóa chính là `event_id`: consumer nhận thông điệp với `event_id` duy nhất; trong cùng một transaction, cố gắng INSERT `event_id` vào `processed_events` và thực thi logic nghiệp vụ. Nếu INSERT thành công (event chưa xử lý), logic được commit; nếu INSERT vi phạm unique constraint (event đã xử lý), transaction rollback và thông điệp bị bỏ qua hoàn toàn. Kết hợp lại: Transactional Outbox đảm bảo sự kiện không bao giờ bị mất (at-least-once); Idempotent Consumer loại bỏ tác động trùng lặp. Sự kết hợp này tạo ra **hiệu ứng exactly-once processing** ở tầng ứng dụng, ngay cả khi hạ tầng Kafka chỉ đảm bảo at-least-once delivery.

2.5 Xử lý đồng thời cao — Redis Lua atomic và mô hình phòng thủ đa lớp

Flash sale tạo ra tình huống đồng thời cực cao: hàng trăm đến hàng nghìn người dùng đồng thời cố gắng mua một số lượng hàng giới hạn. Vấn đề nằm ở **race condition kinh điển**: giả sử còn 1 slot, hai request A và B đồng thời đọc `stock = 1`, cả hai thấy còn hàng, cả hai tiến hành mua — kết quả: 1 slot được bán cho 2 người (over-sell). Hai cách giải quyết truyền thống trong cơ sở dữ liệu quan hệ đều không phù hợp cho tải cao: pessimistic lock (`FOR UPDATE`) tạo hàng đợi tuần tự, hiệu năng sụp đổ khi có 500 VU đồng thời; optimistic lock (version column) tỷ lệ retry tăng tương ứng số VU, làm tăng tải thay vì giảm.

Redis single-threaded model cung cấp nguyên tử tự nhiên: mỗi lệnh được thực thi hoàn chỉnh trước khi lệnh tiếp theo bắt đầu, không có sự xen ngang giữa các

lệnh từ bất kỳ kết nối nào. Tuy nhiên, bài toán flash sale đòi hỏi thực hiện *nhiều* lệnh như một khối không thể tách rời: kiểm tra người dùng đã mua chưa, kiểm tra stock, giảm stock và ghi nhận người mua — nếu thực thi tuần tự các lệnh riêng biệt, khoảng trống giữa các lệnh vẫn tạo ra cửa sổ cho race condition.

Redis Lua Script [7] giải quyết triệt để: khi client gửi script Lua lên Redis, server thực thi toàn bộ script như một khối nguyên tử duy nhất — không một lệnh nào khác được thực thi trong suốt thời gian script chạy. Script flash sale thực hiện đầy đủ trong một lần gọi: (1) kiểm tra `flash-sale:{id}:buyers` xem `user_id` đã mua chưa — nếu rồi trả về `-1`; (2) đọc `flash-sale:{id}:stock` — nếu ≤ 0 trả về `0`; (3) DECR stock và SADD `user_id` vào buyers set, trả về `1`. Không có bất kỳ khoảng trống nào giữa các bước, không có race condition nào có thể xảy ra kể cả khi có hàng nghìn request đồng thời.

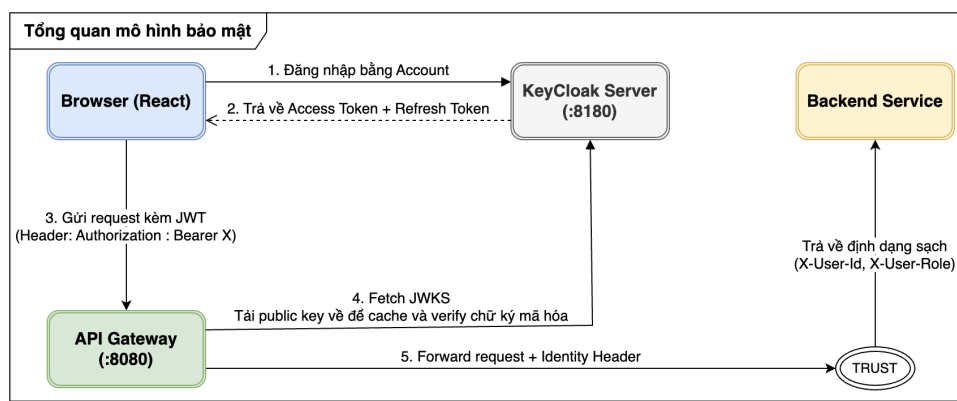
Mô hình phòng thủ đa lớp bổ sung cho Lua script để bảo vệ trước sự cố hạ tầng: *Lớp 1 — Nguyên tử Redis*: Lua script xử lý phần lớn request trong bộ nhớ với độ trễ dưới mili-giây, loại bỏ ngay các request không đủ điều kiện trước khi đến tầng database hay Kafka. *Lớp 2 — Kafka Compensation*: nếu publish Kafka thất bại sau khi Lua script xác nhận slot, service lập tức thực thi compensation: INCR stock và SREM `user_id` khỏi buyers set. *Lớp 3 — ReconciliationScheduler*: chạy mỗi 5 phút, đối chiếu `sold_count` trong database với số đơn flash-sale thực tế; nếu phát hiện drift, bù trừ counter Redis về giá trị đúng. Ba lớp hoạt động độc lập và bổ sung cho nhau: lớp 1 nhanh nhưng không bảo vệ khỏi lỗi hạ tầng; lớp 2 xử lý lỗi ngay lập tức; lớp 3 bắt sai lệch tích lũy mà hai lớp trên bỏ sót.

2.6 Bảo mật tập trung

JSON Web Token (JWT) [8] là tiêu chuẩn mở cho phép truyền thông tin định danh đã được ký số giữa các bên mà không cần tra cứu trạng thái session trên server. Hệ thống sử dụng **RS256** (RSA + SHA-256) thay vì HS256 (HMAC): với HS256, cả bên ký lẫn bên xác minh phải chia sẻ cùng một khóa bí mật — không phù hợp với kiến trúc nhiều service vì mỗi service phải biết secret key, tạo rủi ro bảo mật nếu một service bị xâm phạm. Với RS256, Keycloak giữ private key để ký token; các service chỉ cần public key (lấy từ JWKS endpoint) để xác minh chữ ký — private key không bao giờ rời khỏi Keycloak.

Trong kiến trúc microservices, nếu mỗi service tự xác minh JWT với Keycloak, mỗi request sinh thêm một lời gọi mạng từ từng service, tạo bottleneck và điểm lỗi đơn. **Trusted Subsystem Pattern** giải quyết bằng cách chỉ định một điểm thực thi xác thực duy nhất — API Gateway — và cho phép tất cả service nội bộ tin tưởng thông tin định danh do Gateway cung cấp mà không xác minh lại. Luồng cụ thể:

1. Client gửi request kèm Access Token trong header `Authorization: Bearer`.
2. API Gateway **loại bỏ** (strip) hoàn toàn các header `X-User-Id`, `X-User-Roles`, `X-User-Email` từ request gốc của client — ngăn chặn mọi nỗ lực giả mạo định danh từ phía client.
3. Gateway xác minh chữ ký JWT bằng public key lấy từ JWKS endpoint của Keycloak, kiểm tra `exp` (hạn token) và `iss` (nhà phát hành).
4. Sau khi xác minh thành công, Gateway **chèn** thông tin định danh đã được xác thực vào ba header: `X-User-Id` (sub claim), `X-User-Roles` (realm_access.roles), `X-User-Email` (email claim).
5. Request được chuyển tiếp đến service đích; service đọc trực tiếp các header này mà không cần JWT library.



Hình 2.4: Luồng xác thực Trusted Subsystem Pattern: Gateway xác minh JWT và inject header định danh

Pattern này chỉ an toàn khi kết hợp với cách ly mạng tầng hạ tầng: nếu service backend có thể bị gọi trực tiếp từ bên ngoài, kẻ tấn công có thể bỏ qua Gateway và tự chèn header tùy ý. Trong hệ thống, tất cả service business (cổng 8081–8093) chỉ kết nối vào mạng *internal* của Docker, không expose cổng ra host; API Gateway (cổng 8080) là điểm duy nhất kết nối cả mạng *internal* lẫn *public*, không có đường nào để client gọi trực tiếp vào business service mà không qua Gateway.

2.7 Các mẫu thiết kế hỗ trợ: CQRS-lite, Circuit Breaker và Observability

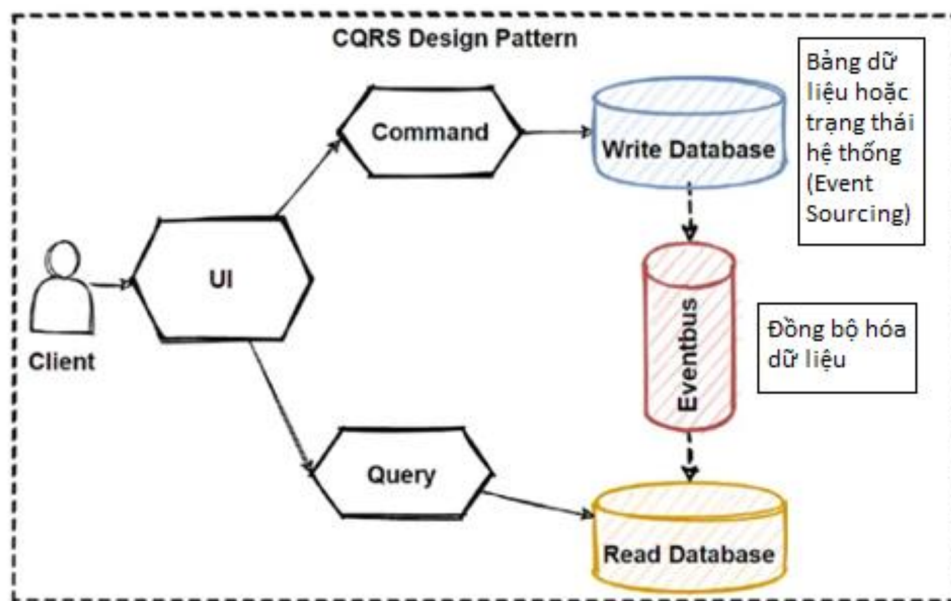
2.7.1 CQRS-lite — tách biệt mô hình đọc và ghi cho tìm kiếm

CQRS (Command Query Responsibility Segregation) [9] là mẫu thiết kế tách hoàn toàn mô hình ghi (Command) và mô hình đọc (Query) của hệ thống, mỗi mô hình được lưu trữ trên kho dữ liệu tối ưu cho kiểu thao tác của mình. Hệ thống áp dụng biến thể **CQRS-lite** cho chức năng tìm kiếm sản phẩm:

- **Mô hình ghi:** product-service quản lý catalog trên PostgreSQL với schema quan hệ chuẩn hóa, đảm bảo tính nhất quán và toàn vẹn dữ liệu (khóa ngoại,

constraints, giao dịch ACID). Đây là nguồn dữ liệu chân thực (source of truth).

- **Mô hình đọc:** search-service duy trì một bản sao denormalized trong Elasticsearch index `products`, được tối ưu cho full-text search, lọc đa tiêu chí (category, brand, price range) và xếp hạng mức độ liên quan.
- **Đồng bộ bất đồng bộ:** khi quản trị viên tạo/sửa/xóa sản phẩm, product-service phát sự kiện `product-created/updated/deleted` lên Kafka. Search-service tiêu thụ sự kiện và cập nhật Elasticsearch, chấp nhận độ trễ đồng bộ ngắn (thường dưới 1 giây) để đổi lấy hiệu năng tìm kiếm vượt trội.



Hình 2.5: Kiến trúc CQRS-lite: PostgreSQL làm mô hình ghi, Elasticsearch làm mô hình đọc

Lý do lựa chọn CQRS-lite thay vì query trực tiếp PostgreSQL: full-text search với phân tích ngữ nghĩa (stemming, tokenization), lọc kết hợp nhiều trường và xếp hạng mức độ liên quan là những khả năng mà Elasticsearch cung cấp tự nhiên nhưng sẽ cần query SQL cực kỳ phức tạp (LIKE, GIN index, tsvector) để mô phỏng trên PostgreSQL với hiệu năng kém hơn đáng kể.

2.7.2 Circuit Breaker — ngăn sự cố dây chuyền giữa các service

Khi một service gọi đồng bộ đến service khác qua OpenFeign và service đích đang gặp sự cố hoặc phản hồi chậm, nếu không có cơ chế bảo vệ, thread pool của service gọi sẽ bị chiếm giữ bởi các request đang chờ, dẫn đến cạn kiệt tài nguyên và sự cố lan truyền sang các service lân cận — hiện tượng *cascading failure*.

Circuit Breaker (Resilience4j) [10], [11] bảo vệ bằng cách giám sát tỷ lệ lỗi của mỗi Feign call và hoạt động theo ba trạng thái:



Hình 2.6: Sơ đồ ba trạng thái Circuit Breaker: CLOSED, OPEN và HALF_OPEN

- **CLOSED:** mọi request được chuyển tiếp bình thường. Nếu tỷ lệ lỗi vượt ngưỡng (50% trong 10 lần gọi gần nhất), circuit chuyển sang OPEN.
- **OPEN:** tất cả request bị từ chối ngay lập tức (fail-fast) mà không gọi thật đến service đích, trả về kết quả fallback. Sau 10 giây, circuit chuyển sang HALF_OPEN.
- **HALF_OPEN:** cho phép một số lượng hạn chế request thật đến service đích để kiểm tra. Nếu thành công, circuit trở về CLOSED; nếu thất bại, quay lại OPEN.

Trong hệ thống, Circuit Breaker được cấu hình cho toàn bộ bảy lời gọi Feign: cart-service → product-service (xác minh sản phẩm khi thêm giỏ), order-service → product-service (snapshot giá), order-service → voucher-service (reserve/commit/release), order-service → cart-service (xóa giỏ sau khi đặt hàng), payment-service → order-service (lấy payment context), review-service → order-service (xác minh lịch sử mua) và flash-sale-service → order-service (đếm đơn phục vụ reconciliation). Cần lưu ý các tương tác trong luồng Saga như đặt chỗ tồn kho không dùng Feign mà đi qua sự kiện Kafka, nên không nằm trong phạm vi bảo vệ của Circuit Breaker. Kết hợp với Retry (tối đa 3 lần, delay 500ms) và Time Limiter (timeout 3 giây), Resilience4j đảm bảo sự cố tại một service không làm tê liệt service phụ thuộc.

Rate Limiting tại API Gateway giới hạn số lượng request mỗi client có thể gửi trong một khoảng thời gian, bảo vệ hệ thống khỏi lạm dụng và đảm bảo công bằng giữa người dùng. Thuật toán Token Bucket được sử dụng: mỗi client được gán "thùng" với sức chứa tối đa B token, token bổ sung với tốc độ r token/giây, mỗi

request tiêu thụ 1 token — cho phép burst ngắn hạn nhưng giới hạn tốc độ trung bình dài hạn. Hai endpoint được giới hạn đặc biệt: `POST /api/orders` (10 req/s, burst 20) và `POST /api/flash-sales/*/purchase` (3 req/s, burst 5). Trạng thái token bucket lưu trên Redis, cho phép rate limiting nhất quán khi Gateway chạy nhiều instance.

2.7.3 Khả năng quan sát — ba trụ cột giám sát hệ thống phân tán

Khi một request thất bại trong hệ thống 16 service, không thể biết lỗi phát sinh từ đâu mà không có công cụ quan sát đúng. Observability được xây dựng trên ba trụ cột bổ sung lẫn nhau:

Metrics (Prometheus + Grafana): thu thập định kỳ các giá trị số phản ánh tình trạng hệ thống — request rate, latency percentile (p50/p95/p99), error rate, JVM memory, Kafka consumer lag. Metrics hiệu quả cho cảnh báo ngưỡng và lập kế hoạch năng lực, nhưng không giải thích nguyên nhân của từng sự kiện cụ thể.

Distributed Tracing (Zipkin): gán một `TraceId` duy nhất cho mỗi request, truyền qua tất cả service mà request đi qua. Mỗi đoạn xử lý tại một service tạo ra một `Span` với thời gian bắt đầu, kết thúc và các nhãn mô tả. Tập hợp tất cả `Span` của một `Trace` tái tạo chính xác hành trình của request qua hệ thống, xác định service nào đang gây độ trễ. Micrometer Tracing tự động propagate context qua HTTP header (`traceparent` theo chuẩn W3C) và qua Kafka message header.

Log tương quan theo trace context: toàn bộ service dùng chung một Logback pattern thống nhất, nhúng tên service cùng `traceId` và `spanId` vào mỗi dòng log (các giá trị này được Micrometer Tracing tự động điền vào MDC). Nhờ `traceId` xuất hiện đồng nhất trên mọi dòng log của cùng một request, có thể lọc và ghép toàn bộ log liên quan đến một request xuyên suốt các service, đồng thời đối chiếu trực tiếp giữa dòng log và trace tương ứng trên Zipkin khi điều tra sự cố.

Scheduled Jobs và Distributed Lock — nhiều tác vụ phải chạy định kỳ theo thời gian: `OutboxPoller` phát sự kiện mỗi 1 giây, `ReservationExpiryScheduler` hủy đơn VNPAY hết hạn mỗi 60 giây, `ReconciliationScheduler` đối chiếu số liệu flash sale mỗi 5 phút. Khi service chạy nhiều instance, cần đảm bảo chỉ một instance thực thi scheduler tại mỗi thời điểm. Giải pháp là Distributed Lock trên Redis với `SET NX EX`: `NX` đảm bảo chỉ một instance giành được khóa; `EX` đảm bảo khóa tự động giải phóng nếu instance đang giữ khóa gặp sự cố, tránh deadlock vĩnh viễn. Đối với `OutboxPoller`, `FOR UPDATE SKIP LOCKED` đóng vai trò distributed lock ở tầng database: mỗi instance xử lý một tập con bản ghi outbox mà không xung đột, tối đa hóa thông lượng khi scale.



Hinhve/chuong2/observability.png

Hình 2.7: Ba trụ cột Observability: Metrics, Distributed Tracing và Log tương quan theo trace context

CHƯƠNG 3. GIẢI PHÁP CÔNG NGHỆ

Chương này trình bày các giải pháp công nghệ được chọn để hiện thực hóa hệ thống, trả lời câu hỏi *công nghệ nào, vì sao và đánh đổi gì*. Mỗi mục phân tích vai trò thực tế trong hệ thống, lý do lựa chọn so với lựa chọn thay thế và hạn chế cần nhận thức.

3.1 Nền tảng phát triển

Hệ thống sử dụng **Java 21** (LTS, hỗ trợ đến 2031) cho toàn bộ 16 service backend, tận dụng các cải tiến của nền tảng ở mức phù hợp: Record Classes giảm boilerplate cho một số DTO; Pattern Matching và switch expression giúp logic điều kiện ngắn gọn hơn. Bản thân Java 21 cũng mở sẵn khả năng dùng Virtual Threads (Project Loom) để tăng thông lượng I/O, là hướng tối ưu có thể bật khi triển khai ở quy mô lớn hơn. Toàn bộ mã nguồn tổ chức theo **Maven multi-module** với một `pom.xml` cha quản lý phiên bản tập trung cho 17 module (16 service + common); module common chứa `ApiResponse` wrapper, exception chuẩn hóa và DTO hợp đồng sự kiện Kafka dùng chung, đảm bảo producer và consumer đồng thuận về cấu trúc thông điệp.

Spring Boot 3.5 [12] là framework chính cho 16 service. Bốn lý do lựa chọn: **(1) Auto-configuration** — khai báo dependency, Spring Boot tự cấu hình `DataSource`, `KafkaTemplate`, `ElasticsearchClient`, loại bỏ phần lớn cấu hình thủ công; **(2) Hệ sinh thái đồng bộ** — Spring Data JPA, Spring Data Redis, Spring Data Elasticsearch, Spring Kafka, Spring Security và Micrometer Tracing tích hợp liền mạch; **(3) Spring Boot Actuator** tự động expose `/actuator/health` và `/actuator/prometheus` làm nền cho observability stack; **(4) Embedded server** — Tomcat cho business service, Netty cho Gateway phản ứng bất đồng bộ, giúp mỗi service đóng gói thành JAR tự chứa, dễ container hóa. Lựa chọn thay thế Quarkus và Micronaut có thời gian khởi động nhanh hơn nhờ compile-time DI, nhưng Spring Boot vượt trội về độ hoàn thiện hệ sinh thái và tài liệu — yếu tố quan trọng hơn trong thời gian phát triển giới hạn. Các thư viện hỗ trợ chính: **Lombok** giảm boilerplate qua `@Data/@Builder`, **Jackson** xử lý JSON nhất quán cho REST và Kafka payload, **Flyway** quản lý schema migration SQL theo thứ tự, **SpringDoc OpenAPI** tự động sinh tài liệu API.

Bảng 3.1: Các thư viện và phiên bản chính

Mục đích	Thư viện / Công cụ	Phiên bản
Ngôn ngữ / Runtime	Java (OpenJDK)	21 LTS
Framework chính	Spring Boot	3.5.x
Spring Cloud	Spring Cloud Dependencies	2025.0.x
Giảm boilerplate	Lombok	1.18.x
JSON serialization	Jackson Databind	(qua Spring Boot BOM)
Database migration	Flyway	11.x
API documentation	SpringDoc OpenAPI	2.8.x

3.2 Hệ sinh thái Spring Cloud

Spring Cloud Gateway [13] là điểm vào duy nhất cho mọi request từ client, xây dựng trên Spring WebFlux và Reactor Netty với mô hình phi đồng bộ non-blocking, xử lý hàng nghìn kết nối đồng thời với số thread hạn chế. Mỗi route ánh xạ đường dẫn đến service đích qua Eureka (`lb://product-service`) rồi áp dụng chuỗi filter. Hệ thống có hai filter tùy chỉnh: **AuthHeaderFilter** thực thi Trusted Subsystem Pattern — loại bỏ các header `X-User-Id/Roles/Email` từ client (ngăn giả mạo), xác minh JWT bằng public key JWKS từ Keycloak, chèn lại thông tin đã xác thực cho service đích; **GuestSessionFilter** tạo và duy trì `sessionId` qua cookie cho người dùng chưa đăng nhập để bảo toàn giỏ hàng. Bên cạnh đó, Gateway dùng filter **RequestRateLimiter** tích hợp sẵn của Spring Cloud Gateway (thuật toán Redis token bucket) cho `POST /api/orders` (10 req/s, burst 20) và `POST /api/flash-sales/*/purchase` (3 req/s, burst 5). CORS cấu hình tập trung qua biến môi trường `CORS_ALLOWED_ORIGINS`.

Netflix Eureka là Service Registry trung tâm: service đăng ký địa chỉ khi khởi động và gửi heartbeat mỗi 30 giây; Gateway truy vấn Eureka để lấy danh sách instance và thực hiện client-side load balancing Round-Robin. Eureka theo mô hình AP (CAP Theorem), phù hợp với TMĐT nơi tính sẵn sàng quan trọng hơn nhất quán tuyệt đối; cơ chế self-preservation giữ nguyên registry thay vì xóa hàng loạt khi heartbeat sụt giảm.

Spring Cloud Config Server quản lý cấu hình tập trung: mỗi service khi khởi động lấy cấu hình qua HTTP theo tên và profile (`order-service-docker.yml...`), cho phép thay đổi tại một điểm mà không cần rebuild image. Trong đồ án dùng native profile (đọc từ classpath container) thay vì Git backend; hạn chế là cần rebuild Config Server image khi thay đổi cấu hình.

OpenFeign là HTTP client khai báo: lập trình viên định nghĩa interface với an-

notation mô tả endpoint đích, OpenFeign sinh implementation tại runtime và tích hợp Eureka để phân giải địa chỉ thực tế. Bẫy Feign call trong hệ thống: cart-service → product-service (snapshot sản phẩm khi thêm giỏ); order-service → product-service (snapshot giá), order-service → voucher-service (đặt chỗ/commit/hoàn voucher), order-service → cart-service (xóa giỏ sau khi đặt hàng); payment-service → order-service (lấy payment context); review-service → order-service (xác minh lịch sử mua); và flash-sale-service → order-service (đếm đơn phục vụ reconciliation). Riêng việc đặt chỗ tồn kho không dùng Feign mà đi qua sự kiện Kafka `order-created` trong luồng Saga.

Resilience4j bảo vệ toàn bộ Feign call khỏi cascading failure qua ba cơ chế phối hợp. *Circuit Breaker* giám sát tỷ lệ lỗi trên sliding window 10 lần: vượt 50%, circuit chuyển OPEN (fail-fast, trả fallback ngay); sau 10 giây chuyển HALF_OPEN thử lại; nếu thành công về CLOSED. *Retry* thử lại tối đa 3 lần với delay 500ms cho lỗi có thể phục hồi, chỉ kích hoạt khi circuit CLOSED hoặc HALF_OPEN. *Time Limiter* áp timeout 3 giây mỗi call; kết hợp ba cơ chế, worst-case là $3 \times 3 = 9$ giây trước khi trả fallback, không có request bị chặn vô thời hạn.

3.3 Apache Kafka — Message Broker

Apache Kafka [14] là trực giao tiếp bất đồng bộ trung tâm, vận hành ở chế độ **KRaft** (Kafka Raft metadata mode) — không phụ thuộc ZooKeeper, giảm số thành phần hạ tầng và đơn giản hóa triển khai trong Docker Compose. Hệ thống sử dụng 13 Kafka topic, mỗi topic phục vụ một luồng nghiệp vụ cụ thể, tổng hợp ở Bảng 3.2.

Bảng 3.2: Danh sách Kafka topic và luồng dữ liệu tương ứng

Topic	Producer	Consumer
user-registered	identity-service	user-service
order-created	order-service	inventory-service
inventory-updated	inventory-service	order-service
inventory-failed	inventory-service	order-service
payment-requested	order-service	payment-service
payment-success	payment-service	order-service
payment-failed	payment-service	order-service
order-confirmed	order-service	inventory-service, notification-service
order-cancelled	order-service	inventory-service, notification-service
flash-sale-order-requested	flash-sale-service	order-service
product-created	product-service	search-service
product-updated	product-service	search-service
product-deleted	product-service	search-service

Kafka được chọn thay vì RabbitMQ vì ba ưu thế trong bối cảnh hệ thống này. Thứ nhất, Kafka lưu trữ thông điệp bền vững trên đĩa với retention có thể cấu hình — OutboxPoller có thể đọc lại sự kiện từ quá khứ nếu cần tái xử lý hoặc debug sau sự cố. Thứ hai, cơ chế partition và consumer group đảm bảo thứ tự sự kiện trong cùng partition trong khi vẫn cho phép xử lý song song — yếu tố quan trọng để tuân tự hóa sự kiện của cùng một đơn hàng. Thứ ba, throughput cao nhờ sequential disk I/O và zero-copy transfer, phù hợp với lưu lượng lớn trong sự kiện flash sale.

Hạn chế của Kafka so với RabbitMQ: độ trễ end-to-end cao hơn do cơ chế batch và polling, kém phù hợp cho các tương tác cần phản hồi tức thì — lý do các call đồng bộ trong hệ thống vẫn dùng OpenFeign. Kafka cũng phức tạp hơn về cấu hình và vận hành, tiêu tốn nhiều tài nguyên hơn trên môi trường single-host.

3.4 Hạ tầng dữ liệu

3.4.1 PostgreSQL 17

PostgreSQL [15] là RDBMS chính, phục vụ 10 trong 13 business service theo nguyên tắc Database per Service. Trên một instance PostgreSQL duy nhất, mỗi service sở hữu một logical database riêng biệt (`order_db`, `payment_db`, `product_db`...), đảm bảo cách ly dữ liệu và cho phép mỗi service quản lý schema độc lập thông qua Flyway migration.

PostgreSQL được chọn thay vì MySQL vì bốn lý do cụ thể. **UUID native:** PostgreSQL hỗ trợ kiểu `uuid` làm khóa chính cho hầu hết bảng trong hệ thống — phù

hợp với môi trường phân tán nơi ID phải duy nhất toàn cầu mà không cần phối hợp trung tâm. **JSONB**: một số bảng (ví dụ snapshot thông tin sản phẩm trong đơn hàng) lưu trữ dữ liệu bán cấu trúc dưới dạng JSONB, cho phép truy vấn JSON với index hiệu quả. **FOR UPDATE SKIP LOCKED**: mệnh đề này — cốt lõi của cơ chế OutboxPoller cho phép nhiều instance xử lý song song mà không tranh chấp — được PostgreSQL hỗ trợ đầy đủ. **MVCC**: Multi-Version Concurrency Control xử lý tốt tình huống đọc-ghi đồng thời cao trong các service như order và payment mà không cần khóa đọc.

Hạn chế: 10 database chạy trên một instance PostgreSQL single-host, không có replication hay failover — điểm cần cải thiện khi chuyển sang Kubernetes với managed database.

3.4.2 Redis 8

Redis đảm nhiệm bốn vai trò với đặc điểm truy cập khác nhau, tận dụng mô hình single-threaded và thao tác in-memory với độ trễ dưới mili-giây:

(1) Flash sale atomic operations: flash-sale-service sử dụng Redis làm bộ đếm tồn kho (`flash-sale:{id}:stock`) và tập hợp người mua (`flash-sale:{id}:buyers`), khai thác Lua Script để thực thi kiểm tra và giảm slot một cách nguyên tử trong một bước duy nhất. Đây là vai trò đặc trưng nhất và quan trọng nhất của Redis trong hệ thống.

(2) Distributed Lock: flash-sale-service sử dụng lệnh `SET NX EX` để tranh giành quyền thực thi các scheduler (`CampaignScheduler`, `ReconciliationScheduler`) khi chạy nhiều instance, đảm bảo chỉ một instance xử lý mỗi thời điểm và tránh deadlock vĩnh viễn qua TTL tự động.

(3) Cart storage: cart-service lưu giỏ hàng với key `cart:{userId}` hoặc `cart:{sessionId}`, tận dụng TTL để tự động dọn dẹp giỏ hàng bỏ hoang mà không cần cron job.

(4) Rate Limiting: API Gateway lưu trạng thái token bucket trên Redis, định danh theo `X-User-Id` cho các endpoint đã xác thực (đặt hàng, flash sale) và theo địa chỉ IP cho endpoint công khai, cho phép rate limiting hoạt động nhất quán khi Gateway chạy nhiều instance.

So sánh lựa chọn thay thế: Memcached thiếu Lua scripting và cấu trúc dữ liệu phong phú (Sorted Set, Set, Hash) nên không đáp ứng được vai trò flash sale; Hazelcast in-process cache loại bỏ network overhead nhưng phức tạp hơn khi đồng bộ trạng thái giữa instance và tiêu tốn heap JVM của chính service.

3.4.3 Elasticsearch 8

Elasticsearch [16] phục vụ read model trong CQRS-lite của search-service. Index `products` lưu bản sao denormalized của thông tin sản phẩm, được tối ưu cho full-text search đa ngôn ngữ, lọc kết hợp nhiều tiêu chí (category, brand, price range) và xếp hạng mức độ liên quan. Đồng bộ từ PostgreSQL qua Kafka: mỗi sự kiện `product-created/updated/deleted` từ product-service được search-service tiêu thụ và ánh xạ vào Elasticsearch document tương ứng, chấp nhận độ trễ đồng bộ ngắn để đổi lấy hiệu năng tìm kiếm.

So với PostgreSQL full-text search (`tsvector`): Elasticsearch vượt trội về tốc độ trên tập dữ liệu lớn, hỗ trợ tìm kiếm mờ (fuzzy search), gợi ý tự động (autocomplete) và phân tích tần suất từ khóa — những tính năng quan trọng cho trải nghiệm tìm kiếm sản phẩm trong TMĐT. So với Apache Solr: Elasticsearch có REST API hiện đại hơn và tích hợp tốt với Spring Data Elasticsearch.

Hạn chế: Elasticsearch tiêu tốn nhiều RAM (JVM heap mặc định 512MB–1GB), cần cấu hình `ES_JAVA_OPTS` cẩn thận trong môi trường single-host để không ảnh hưởng đến các container khác.

3.5 Keycloak — Bảo mật và quản lý định danh

Keycloak 26 [17] được chọn làm Identity Provider (IdP) tập trung, chịu trách nhiệm quản lý người dùng, xác thực và cấp phát token theo chuẩn OAuth 2.0 [18] và OpenID Connect. Với Keycloak, hệ thống không cần tự triển khai logic quản lý mật khẩu, phiên làm việc hay làm mới token — tất cả được Keycloak xử lý theo chuẩn đã được kiểm chứng rộng rãi.

Keycloak đảm nhiệm ba chức năng cụ thể. **Cấp phát token:** sau khi người dùng xác thực thành công, Keycloak cấp ba loại token — Access Token (JWT RS256, thời hạn 5 phút), Refresh Token (thời hạn 30 phút) và ID Token. Access Token được ký bằng private key của Keycloak; public key được công bố tại JWKS endpoint (`/realms/ecommerce/protocol/openid-connect/certs`) để API Gateway xác minh cục bộ mà không cần gọi lại Keycloak cho mỗi request. **Làm mới token:** khi Access Token hết hạn, client dùng Refresh Token để nhận Access Token mới mà không cần đăng nhập lại. **Quản trị người dùng:** quản trị viên tạo, sửa, khóa tài khoản và quản lý vai trò (`ROLE_USER`, `ROLE_ADMIN`) qua Keycloak Admin Console.

So sánh lựa chọn: tự xây dựng hệ thống xác thực (Spring Security + database người dùng) thiếu các tính năng bảo mật đã kiểm chứng như brute-force detection, MFA, social login và token refresh an toàn; Auth0 và Okta là dịch vụ thương mại,

phụ thuộc bên thứ ba và giới hạn số người dùng ở free tier; Keycloak là mã nguồn mở, self-hosted, không giới hạn người dùng và phù hợp với môi trường học thuật.

Hạn chế: Keycloak là ứng dụng Java nặng, tiêu tốn đáng kể tài nguyên RAM và CPU trên single-host — được quản lý bằng `mem_limit` trong Docker Compose và chấp nhận như đánh đổi để có giải pháp bảo mật toàn diện.

3.6 Next.js — Tầng giao diện

Next.js 16 [19] (App Router, React 19, TypeScript, Tailwind CSS) phục vụ hai nhóm màn hình: Storefront cho khách hàng và Admin Panel cho quản trị viên. Access Token lưu trong `httpOnly` cookie và chỉ gắn vào header `Authorization` khi server-side code gọi API Gateway (pattern BFF — Backend for Frontend), ngăn JavaScript phía client truy cập token và giảm nguy cơ XSS. Admin Panel (`/admin/*`) được bảo vệ bởi middleware kiểm tra cookie và vai trò `ROLE_ADMIN`; các mutation dùng Server Actions để token không xuất hiện trong JavaScript trên trình duyệt.

App Router của Next.js 16 cho phép kết hợp linh hoạt Server Components (render HTML phía server, giảm JavaScript bundle) và Client Components (tương tác động phía client). Storefront tận dụng Server Components cho các trang catalog, chi tiết sản phẩm và kết quả tìm kiếm — cải thiện SEO và thời gian tải lần đầu. Các tính năng realtime như cập nhật giỏ hàng và trạng thái flash sale dùng Client Components với React state. Next.js được chọn thay vì React SPA thuần vì hỗ trợ SSR/SSG, routing tích hợp và không cần reverse proxy riêng cho frontend.

3.7 Docker — Đóng gói và triển khai

Docker [20] đóng gói toàn bộ hệ thống — 16 service backend, 1 frontend Next.js cùng các thành phần hạ tầng (PostgreSQL, Redis, Kafka, Elasticsearch, Keycloak, Prometheus, Grafana, Zipkin, Mailpit) — thành container độc lập, đảm bảo môi trường đồng nhất giữa phát triển và production. Docker Compose điều phối các container với `healthcheck`, `depends_on` và `network` nội bộ.

Hai file cấu hình tách biệt môi trường: `docker-compose.yml` cho phát triển local (build từ source, expose đầy đủ cổng) và `docker-compose.prod.yml` cho production-like (kéo image từ GitHub Container Registry, đặt `mem_limit` và `mem_reservation` phù hợp tài nguyên EC2). Caddy đóng vai trò reverse proxy trước API Gateway, tự động cấp và gia hạn chứng chỉ TLS qua Let's Encrypt. Docker Compose được chọn thay vì Kubernetes vì phù hợp với mô hình single-host của đề án; hạn chế rõ ràng là không có auto-scaling, self-healing hay rolling update — các tính năng cần Kubernetes khi hệ thống ra production thật.

3.8 Các công nghệ và công cụ hỗ trợ

VNPAY [21] là cổng thanh toán tích hợp, hỗ trợ thẻ ATM nội địa, thẻ tín dụng quốc tế và QR code, cung cấp môi trường sandbox miễn phí cho phát triển. Payment-service ký tham số bằng HMAC-SHA512 để tạo URL thanh toán; sau khi người dùng hoàn tất, VNPAY gọi lại qua cả Return URL (cập nhật giao diện) và IPN endpoint (ghi nhận kết quả phía server), đảm bảo kết quả không bị mất khi người dùng đóng trình duyệt sớm. Phạm vi đồ án giới hạn ở sandbox.

GitHub Actions tự động build Docker image và publish lên GitHub Container Registry (GHCR) mỗi khi push lên nhánh `main`; **AWS EC2** single-host kéo image mới và khởi động lại container qua `docker-compose.prod.yml`, kiểm chứng khả năng triển khai liên tục trong phạm vi đồ án.

Prometheus [22] thu thập metrics định kỳ 15 giây từ `/actuator/prometheus` của từng service (JVM, request rate/latency/error, Kafka consumer lag, Circuit Breaker state); **Grafana** trực quan hóa qua ba dashboard provision tự động: `jvm-overview`, `spring-boot-overview` và `ecommerce-saga-overview` (trạng thái đơn hàng theo thời gian). **Zipkin** [23] theo dõi hành trình request xuyên service: Micrometer Tracing sinh `TraceId/SpanId` và propagate qua HTTP header `traceparent` (W3C) và Kafka header, cho phép tái tạo toàn bộ chuỗi xử lý khi điều tra sự cố.

Mailpit là SMTP server giả lập cho môi trường phát triển, bắt giữ toàn bộ email từ `notification-service` và hiển thị qua giao diện web tại `http://localhost:8025`; thay thế bằng SMTP thật trên production qua biến môi trường mà không cần thay đổi mã nguồn.

JUnit 5 + Mockito kiểm thử đơn vị, mock Feign client/Kafka template/repository để kiểm thử logic nghiệp vụ cô lập. **Testcontainers** khởi tạo container PostgreSQL thật trong test thay vì H2 in-memory — cần thiết vì H2 không hỗ trợ `uuid`, `jsonb` và `FOR UPDATE SKIP LOCKED`; dùng chủ yếu cho `OutboxPoller` và `Idempotent Consumer`. **k6** [24] kiểm thử hiệu năng với bốn kịch bản: smoke test (9/9 pass), catalog stress test (p95 = 61 ms, 200 VU), checkout load test (100% success, 50 VU) và flash sale spike test (500 VU đồng thời, 0 over-sell) — kết quả dùng làm bằng chứng định lượng trong Chương 5.

CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày toàn bộ kết quả phân tích và thiết kế hệ thống, từ việc xác định yêu cầu, mô hình hóa kiến trúc tổng thể, thiết kế cơ sở dữ liệu, thiết kế API và giao tiếp, đến thiết kế luồng sự kiện, bảo mật, máy trạng thái và các luồng nghiệp vụ chính. Đây là chương có mật độ sơ đồ và bảng biểu cao nhất, phản ánh toàn bộ quá trình chuyển đổi từ yêu cầu nghiệp vụ sang thiết kế kỹ thuật chi tiết. Các quyết định thiết kế trong chương này dựa trên nền tảng lý thuyết đã trình bày ở Chương 2 và được hiện thực hóa bằng các công nghệ đã phân tích ở Chương 3.

4.1 Phân tích yêu cầu hệ thống

Yêu cầu chức năng. Hệ thống được xây dựng để đáp ứng tập hợp đầy đủ các chức năng của một nền tảng thương mại điện tử, phục vụ hai nhóm người dùng chính là khách hàng và quản trị viên. Bảng 4.1 tổng hợp các yêu cầu chức năng được nhóm theo miền nghiệp vụ.

Bảng 4.1: Yêu cầu chức năng theo miền nghiệp vụ

Miền nghiệp vụ	Đối tượng	Yêu cầu chức năng
Định danh & Người dùng	Khách hàng	Đăng ký tài khoản, đăng nhập, làm mới token
	Khách hàng	Quản lý thông tin cá nhân, quản lý địa chỉ giao hàng
Sản phẩm & Danh mục	Khách hàng	Xem danh sách sản phẩm, lọc theo danh mục/thương hiệu, xem chi tiết
	Quản trị viên	Tạo, sửa, xóa sản phẩm, danh mục, thương hiệu
Tìm kiếm	Khách hàng	Tìm kiếm toàn văn theo từ khóa, gợi ý sản phẩm liên quan
Giỏ hàng	Khách hàng	Thêm, sửa số lượng, xóa sản phẩm; hỗ trợ giỏ hàng khách vắng lai
Đơn hàng	Khách hàng	Tạo đơn hàng (COD/VNPAY), xem danh sách đơn hàng, xem chi tiết trạng thái
	Hệ thống	Tự động xử lý luồng Saga: trừ tồn kho, xác nhận thanh toán, hủy đơn quá hạn
Thanh toán	Khách hàng	Thanh toán online qua VNPAY, xem kết quả thanh toán
	Hệ thống	Xử lý IPN callback từ VNPAY, tự động timeout thanh toán quá hạn
Khuyến mãi	Khách hàng	Xem danh sách voucher khả dụng, áp dụng voucher khi đặt hàng

Miền nghiệp vụ	Đối tượng	Yêu cầu chức năng
	Quản trị viên	Tạo, sửa, xóa voucher; cấu hình điều kiện sử dụng
Flash Sale	Khách hàng	Xem danh sách chiến dịch, mua hàng flash sale với số lượng giới hạn
	Quản trị viên	Tạo, kích hoạt, kết thúc chiến dịch; cấu hình sản phẩm và số lượng
Đánh giá	Khách hàng	Tạo, sửa, xóa đánh giá sản phẩm (chỉ sau khi mua hàng thành công)
Nội dung	Quản trị viên	Quản lý banner, bài viết blog
Thông báo	Hệ thống	Gửi email xác nhận đơn hàng và hủy đơn

Yêu cầu phi chức năng. Bên cạnh các yêu cầu chức năng, hệ thống phải đáp ứng các yêu cầu phi chức năng then chốt của một hệ thống phân tán, được tổng hợp trong Bảng 4.2.

Bảng 4.2: Yêu cầu phi chức năng và chiến lược đáp ứng

Yêu cầu	Mô tả	Chiến lược thiết kế
Khả năng mở rộng	Hệ thống có thể mở rộng độc lập từng dịch vụ theo tải	Phân rã 13 service; database per service
Tính sẵn sàng	Hệ thống tiếp tục hoạt động khi một service gặp sự cố	Circuit Breaker; stateless service; graceful degradation
Tính nhất quán	Dữ liệu nhất quán cuối cùng giữa các service	Saga Orchestration; Outbox + Idempotent Consumer
Bảo mật	Xác thực và phân quyền cho mọi API endpoint	Keycloak OAuth2/OIDC; JWT RS256; Gateway Trusted Subsystem
Hiệu năng	Đáp ứng tải cao trong flash sale; phản hồi nhanh cho catalog	Redis cache; Redis Lua atomic; k6 load test
Khả năng quan sát	Giám sát và truy vết request end-to-end	Prometheus + Grafana; Zipkin; structured logging
Khả năng triển khai	Dễ dàng khởi tạo toàn bộ hệ thống	Docker Compose; healthcheck; Flyway migration tự động

4.2 Mô hình hóa yêu cầu — Biểu đồ Use Case

Hệ thống có bốn tác nhân chính tương tác theo các vai trò và mục đích khác nhau. Khách hàng chưa đăng nhập (Guest) có thể duyệt sản phẩm, tìm kiếm và thao tác với giỏ hàng tạm thời. Khách hàng đã đăng nhập (Customer) có toàn quyền thực hiện các nghiệp vụ mua sắm: đặt hàng, thanh toán, tham gia flash sale và đánh giá sản phẩm. Quản trị viên (Admin) quản lý toàn bộ danh mục, sản phẩm, tồn kho, khuyến mãi, nội dung và chiến dịch flash sale. Hệ thống bên ngoài (External System) bao gồm VNPAY xử lý thanh toán và SMTP Server gửi email thông báo. Ngoài ra, các tác vụ nền (Scheduler) tự động xử lý các công việc định kỳ như hết hạn đặt chỗ tồn kho và timeout thanh toán.



Hình 4.1: Biểu đồ Use Case tổng thể hệ thống

4.3 Kiến trúc tổng thể

Hệ thống được tổ chức thành bốn tầng kiến trúc rõ ràng, mỗi tầng có trách nhiệm và phạm vi ảnh hưởng riêng biệt. Tầng Client bao gồm trình duyệt web, ứng dụng di động hoặc các công cụ kiểm thử API như Postman — đây là nơi phát sinh yêu cầu vào hệ thống. Tầng Edge và Spring Cloud chứa API Gateway (cổng vào duy nhất), Eureka Server (đăng ký và khám phá dịch vụ), Config Server (quản lý cấu hình tập trung) và Keycloak (xác thực và cấp token). Tầng Business Services gồm 13 dịch vụ nghiệp vụ, mỗi dịch vụ đảm nhiệm một miền chức năng độc lập và sở hữu cơ sở dữ liệu riêng. Tầng Infrastructure chứa các hệ thống hỗ trợ: PostgreSQL, Redis, Kafka, Elasticsearch, Mailpit, Prometheus, Grafana và Zipkin.

Giao tiếp giữa các tầng tuân theo nguyên tắc: Client chỉ giao tiếp với tầng Edge (qua API Gateway); các dịch vụ trong tầng Business giao tiếp với nhau qua REST/OpenFeign hoặc bất đồng bộ qua Kafka; mọi truy cập đến cơ sở dữ liệu đều thông qua dịch vụ sở hữu dữ liệu đó — không có truy cập trực tiếp từ dịch vụ khác.

Hinhve/chuong4/kien-truc-4-tang-he-thong.png

Hình 4.2: Sơ đồ thành phần bốn tầng của hệ thống

4.4 Phân rã miền nghiệp vụ và sở hữu dịch vụ

Áp dụng nguyên tắc Domain-Driven Design, hệ thống được phân rã thành 13 Bounded Context, mỗi context tương ứng với một microservice. Bảng 4.3 mô tả chi tiết trách nhiệm, dữ liệu sở hữu và phương thức giao tiếp của từng dịch vụ. Nguyên tắc cốt lõi là mỗi dịch vụ sở hữu toàn bộ logic nghiệp vụ và dữ liệu trong phạm vi Bounded Context của mình; mọi truy cập dữ liệu chéo phải thông qua API công khai hoặc sự kiện được công bố trên Kafka.

Bảng 4.3: Sở hữu dịch vụ và giao tiếp

Dịch vụ	Cổng	Bounded Context	Dữ liệu sở hữu	Giao tiếp
identity-service	8081	Định danh & Đăng ký	Keycloak users	Kafka (pub)
user-service	8082	Hồ sơ người dùng	PostgreSQL (user_db)	Kafka (sub)
product-service	8083	Sản phẩm & Danh mục	PostgreSQL (product_db), Redis cache	REST, Kafka (pub)
inventory-service	8084	Tồn kho & Đặt chỗ	PostgreSQL (inventory_db)	Kafka (pub/-sub)
cart-service	8085	Giỏ hàng	Redis (cart keys)	REST (Feign)
order-service	8086	Đơn hàng (Saga orchestrator)	PostgreSQL (order_db), Outbox	REST, Kafka (pub/sub)
payment-service	8087	Thanh toán & VNPAY	PostgreSQL (payment_db), Outbox	REST, Kafka (pub/sub)
voucher-service	8088	Khuyến mãi	PostgreSQL (voucher_db)	REST

Dịch vụ	Cổng	Bounded Context	Dữ liệu sở hữu	Giao tiếp
notification-service	8089	Thông báo & Email	PostgreSQL (notification_db)	Kafka (sub)
review-service	8090	Đánh giá sản phẩm	PostgreSQL (review_db)	REST (Feign)
search-service	8091	Tìm kiếm toàn văn	Elasticsearch indices	Kafka (sub)
content-service	8092	Nội dung (Banner/Blog)	PostgreSQL (content_db)	REST
flash-sale-service	8093	Flash Sale	PostgreSQL (flash_sale_db), Redis stock	REST, Kafka (pub/sub)

Hinhve/chuong4/context-map.png

Hình 4.3: Bản đồ ngữ cảnh (Context Map) 13 Bounded Context

4.5 Thiết kế cơ sở dữ liệu

4.5.1 PostgreSQL

Mỗi business service sử dụng PostgreSQL sở hữu một database riêng với schema được quản lý bởi Flyway migration. Dưới đây là thiết kế cho các database cốt lõi. Các bảng sử dụng UUID làm khóa chính, đảm bảo tính duy nhất toàn cục và tránh phụ thuộc vào sequence tự tăng của database — điều quan trọng trong môi trường phân tán nơi dữ liệu có thể được tạo từ nhiều service khác nhau.

Database đơn hàng — `order_db`

Đây là database quan trọng nhất, lưu trữ toàn bộ vòng đời của đơn hàng cùng với cơ chế Outbox và Idempotency. Bảng `orders` lưu thông tin đơn hàng với các trường chính: mã đơn, mã khách hàng, phương thức thanh toán, trạng thái, tổng tiền, mã voucher (nếu có) và thời điểm hết hạn đặt chỗ. Bảng `order_items` lưu

chi tiết từng mặt hàng trong đơn: mã sản phẩm, số lượng và đơn giá tại thời điểm đặt. Bảng `outbox` lưu các sự kiện cần phát hành lên Kafka, với các trường: mã sự kiện, loại aggregate, mã aggregate, loại sự kiện, payload JSON, cờ `processed` và thời điểm xử lý. Bảng `processed_events` lưu mã các sự kiện đã được xử lý, phục vụ cơ chế Idempotent Consumer.

Database thanh toán — `payment_db`

Bảng `payments` lưu thông tin thanh toán: mã thanh toán, mã đơn hàng, phương thức, số tiền, trạng thái và thời điểm hết hạn. Bảng `payment_outbox` lưu sự kiện thanh toán ở trạng thái `PENDING`, số lần thử, lỗi cuối cùng và thời điểm phát hành, đảm bảo khả năng phát hành sự kiện thanh toán một cách tin cậy.

Database sản phẩm — `product_db`

Bảng `products` lưu thông tin sản phẩm (tên, mô tả, giá, SKU, trạng thái hoạt động), liên kết với `categories` (danh mục phân cấp) và `brands` (thương hiệu).

Database tồn kho — `inventory_db`

Bảng `inventory` lưu hai giá trị chính cho từng SKU: `quantity` là tổng số lượng còn trong kho và `reserved_quantity` là số lượng đang được đặt chỗ. Số lượng khả dụng được suy ra bằng `quantity - reserved_quantity`. Khi nhận sự kiện `order-created`, `inventory-service` tăng `reserved_quantity`; khi nhận `order-confirmed`, `service` giảm đồng thời `quantity` và `reserved_quantity`; khi nhận `order-cancelled`, `service` giảm `reserved_quantity` để giải phóng đặt chỗ.

Các database còn lại

Các database `user_db`, `voucher_db`, `notification_db`, `review_db`, `content_db` và `flash_sale_db` lưu trữ dữ liệu nghiệp vụ tương ứng với mô hình quan hệ chuẩn hóa, mỗi database do một service duy nhất quản lý.

4.5.2 Elasticsearch

`Search-service` sử dụng Elasticsearch để lưu trữ và truy vấn thông tin sản phẩm phục vụ tìm kiếm toàn văn. Mỗi sản phẩm được lưu dưới dạng một document trong index `products` với cấu trúc được tối ưu cho tìm kiếm: trường `name` và `description` được phân tích toàn văn (text analyzer với stemming tiếng Việt nếu có), trường `category` và `brand` được lưu dưới dạng keyword để lọc chính xác, trường `price` lưu dưới dạng số để lọc theo khoảng giá. Document được đồng bộ từ PostgreSQL của `product-service` thông qua Kafka event mỗi khi có thay đổi, đảm bảo eventual consistency giữa hai mô hình.

Hinhve/chuong4/luong-outbox-idempotency.png

Hình 4.4: Luồng hoạt động của Transactional Outbox và Idempotent Consumer

Hinhve/chuong4/erd-main.png

Hình 4.5: Mô hình ERD các database chính: order_db, payment_db, product_db, inventory_db

4.5.3 Redis

Redis được sử dụng cho ba mục đích với cấu trúc khóa được thiết kế rõ ràng. Đối với giỏ hàng, khóa có định dạng `cart:{userId}` hoặc `cart:{sessionId}` lưu trữ danh sách sản phẩm dưới dạng Hash, với TTL 7 ngày cho người dùng đã đăng nhập và 30 phút cho khách vắng lai. Đối với cache sản phẩm, khóa `product:{productId}` lưu thông tin sản phẩm dưới dạng JSON string với TTL 30 phút, được tự động invalidate khi sản phẩm được cập nhật. Đối với flash sale, khóa `flash-sale:{campaignId}:stock` lưu số lượng slot còn lại dưới dạng integer, được khởi tạo khi chiến dịch được kích hoạt và tự động hết hạn theo TTL bằng với thời gian còn lại của chiến dịch; khóa `flash-sale:{campaignId}:buyers` lưu tập hợp (Set) mã người dùng đã mua thành công để chống trùng lặp.



Hình 4.6: Mô hình khóa Redis: Cart, Cache, Flash Sale

4.6 Thiết kế API và giao tiếp đồng bộ

Định tuyến API Gateway. API Gateway định tuyến yêu cầu từ client đến các dịch vụ backend dựa trên đường dẫn. Ngoài chức năng định tuyến, Gateway còn áp dụng các chính sách bảo mật và giới hạn tốc độ khác nhau cho từng nhóm endpoint. Bảng 4.4 tổng hợp các route chính.

Bảng 4.4: Định tuyến API Gateway

Đường dẫn	Dịch vụ đích	Xác thực	Rate Limit
/api/auth/**	identity-service	Không (public)	10 req/s, burst 20
/api/products/**	product-service	Tùy chọn (GET public)	Không giới hạn
/api/search/**	search-service	Tùy chọn	Không giới hạn
/api/cart/**	cart-service	Tùy chọn (guest session)	Không giới hạn
/api/users/**	user-service	Bắt buộc	Không giới hạn
/api/orders/**	order-service	Bắt buộc	10 req/s, burst 20
/api/payments/**	payment-service	Bắt buộc (trừ VNPAY IPN)	Không giới hạn
/api/vouchers/**	voucher-service	Bắt buộc (admin cho CRUD)	Không giới hạn
/api/reviews/**	review-service	Bắt buộc (tạo), tùy chọn (xem)	Không giới hạn
/api/flash-sales/*/purchase	flash-sale-service	Bắt buộc	3 req/s, burst 5

Đường dẫn	Dịch vụ đích	Xác thực	Rate Limit
/api/flash-sales/**	flash-sale-service	Bắt buộc (admin cho CRUD)	Không giới hạn
/api/content/**	content-service	Tùy chọn (GET public)	Không giới hạn
/api/notifications/**	notification-service	Bắt buộc	Không giới hạn

Giao tiếp Feign giữa các dịch vụ. Các tương tác đồng bộ giữa các dịch vụ được thực hiện thông qua OpenFeign client. Cart-service gọi product-service để xác minh tính hợp lệ của sản phẩm khi thêm vào giỏ hàng. Order-service — với vai trò Saga orchestrator — gọi product-service để kiểm tra giá và trạng thái sản phẩm, gọi voucher-service để xác minh và đặt chỗ voucher khi tạo đơn, và gọi cart-service để xóa giỏ hàng sau khi đơn được xác nhận. Payment-service gọi order-service để lấy payment context; review-service gọi order-service để kiểm tra người dùng đã có đơn hàng đã xác nhận chứa sản phẩm cần đánh giá hay chưa; flash-sale-service gọi order-service để đếm số đơn phục vụ reconciliation. Việc đặt chỗ tồn kho không dùng Feign mà đi qua sự kiện Kafka `order-created` trong luồng Saga. Các Feign client được cấu hình timeout 2–3 giây và retry 2–3 lần (tùy client), kết hợp với Circuit Breaker để ngăn cascading failure.

Hình ảnh: `chuong4/feign-dependencies.png`

Hình 4.7: Sơ đồ phụ thuộc REST/Feign giữa các dịch vụ

4.7 Thiết kế luồng sự kiện — Kafka Topics

Giao tiếp bất đồng bộ giữa các dịch vụ được thực hiện thông qua 13 Kafka topic, được tổ chức thành 11 nhóm nghiệp vụ. Mỗi topic có producer và consumer được xác định rõ ràng, cùng với cấu trúc payload cụ thể. Kafka được cấu hình với cơ chế tự động tạo topic (`auto.create.topics.enable=true`) cho hầu hết các topic, ngoại trừ `flash-sale-order-requested` được tạo thủ công với 3

partition để kiểm soát thứ tự xử lý.

Bảng 4.5 mô tả toàn bộ các topic, luồng sự kiện và vai trò của từng dịch vụ tham gia.

Bảng 4.5: Bản đồ Kafka Topics

Topic	Producer	Consumer	Mục đích
user-registered	identity-service	user-service	Tạo hồ sơ người dùng sau đăng ký
product-created	product-service	search-service	Index sản phẩm mới vào Elasticsearch
product-updated	product-service	search-service	Cập nhật index khi sửa sản phẩm
product-deleted	product-service	search-service	Xóa index khi xóa sản phẩm
order-created	order-service	inventory-service	Yêu cầu đặt chỗ tồn kho
inventory-updated	inventory-service	order-service	Xác nhận đặt chỗ thành công
inventory-failed	inventory-service	order-service	Thông báo không đủ tồn kho
payment-requested	order-service	payment-service	Yêu cầu xử lý thanh toán COD
payment-success	payment-service	order-service	Thanh toán thành công, xác nhận đơn
payment-failed	payment-service	order-service	Thanh toán thất bại, hủy đơn
order-confirmed	order-service	inventory-service, notification-service	Xác nhận tồn kho, gửi email
order-cancelled	order-service	inventory-service, notification-service	Hoàn tồn kho, gửi email (voucher hoàn qua Feign)
flash-sale-order-requested	flash-sale-service	order-service	Tạo đơn hàng flash sale

Hìnhve/chuong4/kafka-flow.png

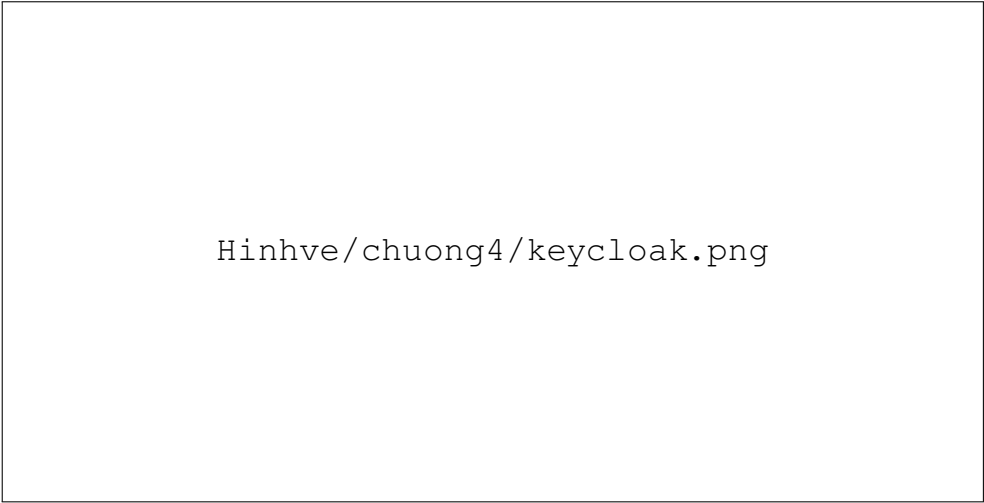
Hình 4.8: Sơ đồ luồng sự kiện Kafka giữa các dịch vụ

4.8 Thiết kế bảo mật

Hệ thống áp dụng mô hình bảo mật tập trung với API Gateway là điểm thực thi duy nhất. Luồng xác thực diễn ra qua bốn bước. Bước thứ nhất, client gửi thông tin đăng nhập đến Keycloak (qua identity-service hoặc trực tiếp) và nhận được bộ token JWT. Bước thứ hai, client gửi kèm Access Token trong header `Authorization: Bearer <token>` cho mọi request đến API Gateway. Bước thứ ba, Gateway xác minh chữ ký JWT sử dụng public key lấy từ endpoint JWKS của Keycloak, kiểm tra thời hạn và các claims cần thiết. Bước thứ tư, sau khi xác minh thành công, Gateway trích xuất thông tin định danh từ JWT (userId, roles, email) và chèn vào ba header `X-User-Id`, `X-User-Roles`, `X-User-Email` trước khi chuyển tiếp request đến dịch vụ backend.

Các dịch vụ backend không thực hiện lại việc xác minh JWT mà hoàn toàn tin tưởng vào các header do Gateway cung cấp (Trusted Subsystem Pattern). Điều kiện để mô hình này an toàn là các dịch vụ backend chỉ lắng nghe trên mạng nội bộ của Docker, không có cổng nào được mở ra bên ngoài. Phân quyền được thực hiện dựa trên vai trò: các route quản trị như `/api/orders/admin/**`, `/api/users/admin/**`, `/api/inventory/admin/**`, `/api/reviews/admin/**` và các mutation `product/voucher/flash-sale/content` yêu cầu `ROLE_ADMIN`; các endpoint nghiệp vụ người dùng yêu cầu token hợp lệ hoặc được mở public tùy chức năng.

Các endpoint đặc biệt nhạy cảm được bảo vệ bổ sung bằng Redis token bucket rate limiter. Endpoint đặt hàng (`POST /api/orders`) được giới hạn 10 request/giây với khả năng burst 20 request để ngăn chặn đặt hàng tự động. Endpoint mua flash sale (`POST /api/flash-sales/*/purchase`) được giới hạn 3 request/giây với burst 5 request, đảm bảo công bằng giữa những người tham gia.

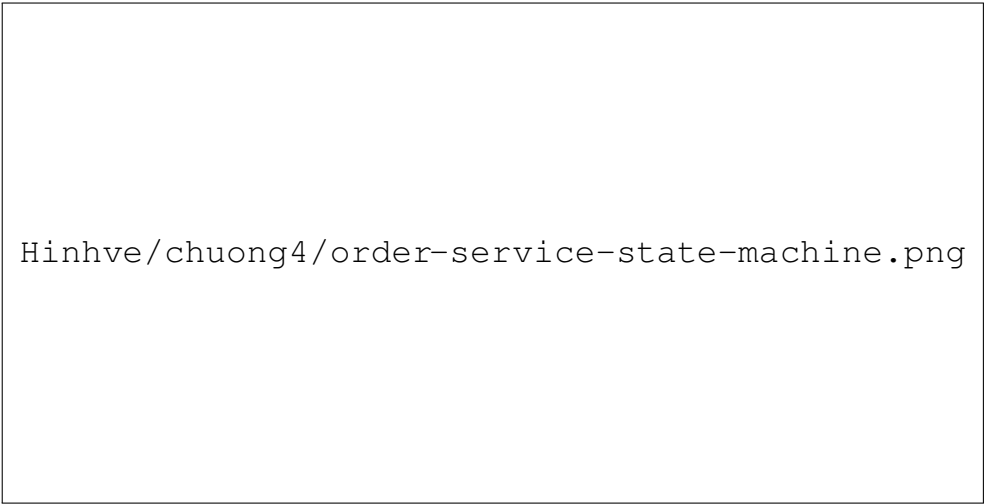


Hinhve/chuong4/keycloak.png

Hình 4.9: Luồng xác thực và phân quyền qua API Gateway

4.9 Thiết kế máy trạng thái

Máy trạng thái đơn hàng. Đơn hàng trong hệ thống trải qua một vòng đời được kiểm soát chặt chẽ với bốn trạng thái chính. Trạng thái `PENDING` là trạng thái khởi tạo, ngay sau khi `order-service` tạo đơn hàng và ghi sự kiện `order-created` vào `outbox`. Trạng thái `STOCK_RESERVED` đạt được khi `inventory-service` xác nhận đã đặt chỗ tồn kho thành công. Từ đây, luồng xử lý rẽ nhánh tùy theo phương thức thanh toán: với `COD`, đơn hàng được xác nhận ngay lập tức; với `VNPAY`, đơn hàng chờ khách hàng thanh toán trong vòng 30 phút. Trạng thái `CONFIRMED` là trạng thái kết thúc thành công, đạt được khi thanh toán `COD` được xác nhận hoặc `VNPAY` callback thành công. Trạng thái `CANCELLED` là trạng thái kết thúc thất bại, xảy ra khi tồn kho không đủ, thanh toán thất bại hoặc hết thời gian chờ thanh toán.



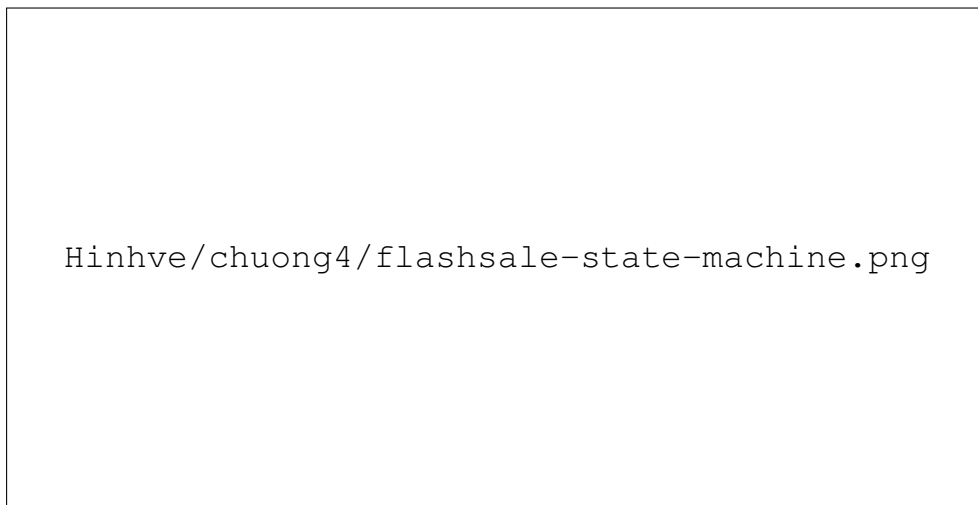
Hinhve/chuong4/order-service-state-machine.png

Hình 4.10: Sơ đồ máy trạng thái đơn hàng

Máy trạng thái thanh toán. Thanh toán `VNPAY` có bốn trạng thái. `PENDING` — URL thanh toán đã được tạo, khách hàng đang trong quá trình thanh toán. `COMPLETED` — `VNPAY` xác nhận thanh toán thành công qua IPN callback hoặc

Return URL. FAILED — thanh toán thất bại do khách hàng hủy, thẻ không hợp lệ hoặc số dư không đủ. TIMEOUT — quá thời gian chờ 30 phút, được phát hiện và xử lý bởi PaymentTimeoutScheduler. Một ràng buộc quan trọng là mỗi đơn hàng chỉ có thể có tối đa một thanh toán ở trạng thái PENDING tại mỗi thời điểm, được đảm bảo bằng partial unique index trên order_id khi status = 'PENDING'.

Máy trạng thái chiến dịch Flash Sale. Chiến dịch flash sale vận hành qua bốn trạng thái. SCHEDULED — chiến dịch đã được tạo và lên lịch, đang chờ đến thời điểm bắt đầu. ACTIVE — chiến dịch đang diễn ra, người dùng có thể tham gia mua hàng; ở trạng thái này, CampaignScheduler đã khởi tạo Redis stock key và TTL. ENDED — chiến dịch đã kết thúc theo thời gian, không nhận thêm yêu cầu mua mới. CANCELLED — chiến dịch bị quản trị viên hủy trước hoặc trong thời gian diễn ra. Quá trình chuyển trạng thái từ SCHEDULED sang ACTIVE được thực hiện tự động bởi CampaignScheduler (chu kỳ 5 giây) dựa trên thời gian hệ thống. Việc chuyển sang ENDED cũng được CampaignScheduler xử lý khi thời gian kết thúc đã qua; khi hủy chiến dịch, service xóa các Redis key liên quan để không tiếp nhận mua mới.



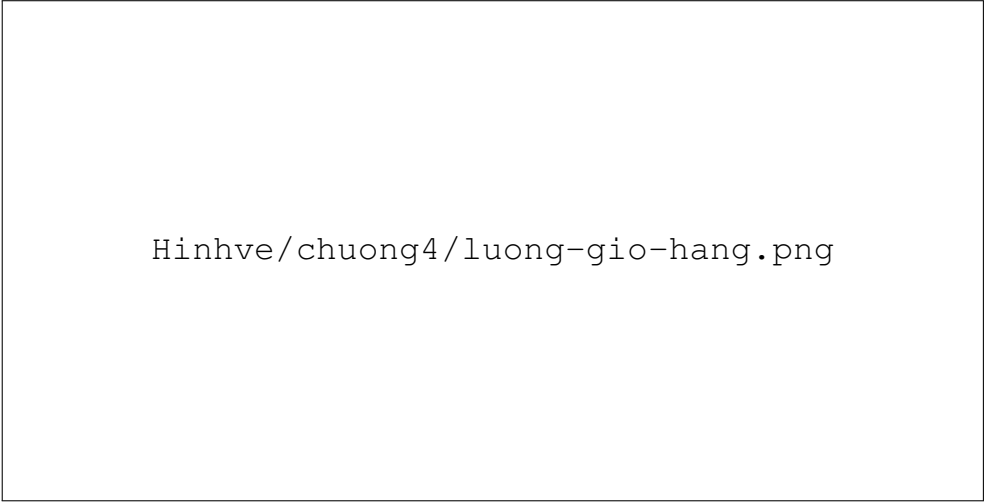
Hình 4.11: Sơ đồ máy trạng thái thanh toán và chiến dịch Flash Sale

4.10 Thiết kế luồng nghiệp vụ chính

4.10.1 Luồng thao tác giỏ hàng

Giỏ hàng trong hệ thống được thiết kế để hỗ trợ cả người dùng đã đăng nhập và khách vãng lai thông qua cơ chế session. Khi người dùng thêm sản phẩm vào giỏ hàng qua POST/api/cart/items, cart-service gọi product-service qua OpenFeign để xác minh sản phẩm tồn tại và đang hoạt động, sau đó lưu thông tin vào Redis với key được tạo từ userId (nếu đã đăng nhập) hoặc sessionId (nếu là khách). Khi người dùng đăng nhập, session cart được tự động hợp nhất (merge) vào user cart thông qua API POST/api/cart/merge. Dữ liệu giỏ hàng được

đặt TTL 7 ngày cho người dùng đã đăng nhập và 30 phút cho khách vắng lai, tự động được dọn dẹp khi hết hạn.

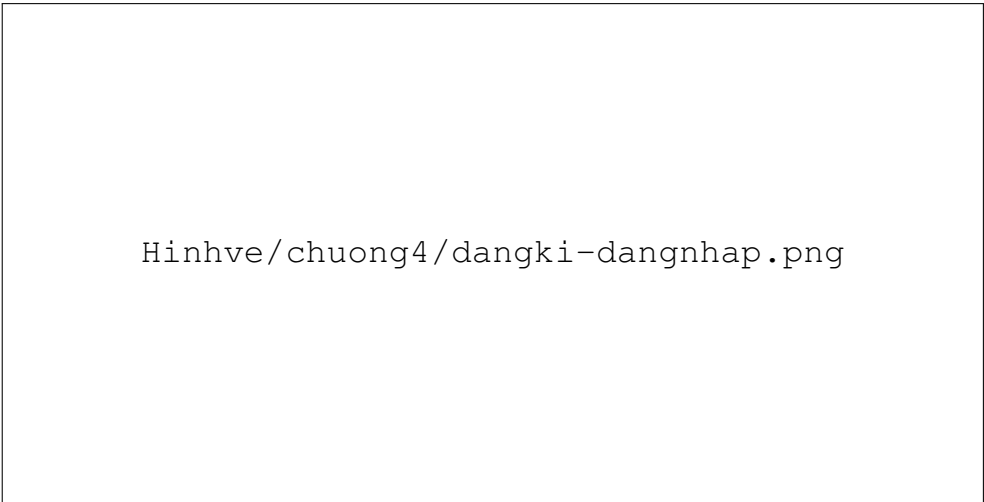


Hinhve/chuong4/luong-gio-hang.png

Hình 4.12: Biểu đồ tuần tự luồng thao tác giỏ hàng

4.10.2 Luồng đăng ký người dùng

Khi người dùng gửi yêu cầu đăng ký qua `POST/api/auth/register`, identity-service tiếp nhận và gọi Keycloak Admin API để tạo tài khoản mới trong realm `ecommerce`. Sau khi tạo thành công, identity-service phát hành sự kiện `user-registered` lên Kafka, chứa thông tin: `userId`, `email`, `username` và `timestamp`. User-service tiêu thụ sự kiện này và tạo bản ghi User profile tương ứng trong `user_db`, bao gồm các trường mặc định như họ tên (từ email), trạng thái hoạt động và thời điểm tạo. Luồng này minh họa mẫu giao tiếp bất đồng bộ: identity-service không cần biết user-service tồn tại, và user-service có thể được triển khai hoặc thay đổi độc lập.



Hinhve/chuong4/dangki-dangnhap.png

Hình 4.13: Biểu đồ tuần tự luồng đăng kí và đăng nhập người dùng

4.10.3 Luồng đặt hàng COD — Saga Orchestration

Đây là luồng phức tạp nhất, thể hiện đầy đủ Saga Orchestration Pattern với order-service đóng vai trò orchestrator. Luồng diễn ra qua các bước sau:

Bước 1 — Tạo đơn hàng. Khách hàng gửi yêu cầu `POST/api/orders` với payload chứa danh sách sản phẩm (`productId`, `quantity`), phương thức thanh toán COD, địa chỉ giao hàng và mã voucher (nếu có). Order-service thực hiện hai lời gọi Feign đồng bộ: gọi product-service để xác minh sản phẩm tồn tại và đang hoạt động, lấy đơn giá hiện tại; và gọi voucher-service để xác minh và đặt chỗ voucher (nếu có). Việc kiểm tra và đặt chỗ tồn kho không thực hiện đồng bộ ở bước này mà được xử lý bất đồng bộ ở Bước 2 qua sự kiện Kafka. Nếu các lời gọi đều hợp lệ, order-service tạo bản ghi đơn hàng với trạng thái `PENDING` và đồng thời ghi sự kiện `order-created` vào bảng `outbox` trong cùng một giao dịch.

Bước 2 — Đặt chỗ tồn kho. `OutboxPoller` (chu kỳ 1 giây) của order-service phát hành sự kiện `order-created` lên Kafka. Inventory-service tiêu thụ sự kiện, thực hiện đặt chỗ cho từng sản phẩm trong đơn hàng bằng cách tăng `reserved_quantity` trong khi `quantity` chưa đổi. Nếu tất cả sản phẩm đều đủ số lượng, inventory-service phát hành `inventory-updated` lên Kafka. Nếu bất kỳ sản phẩm nào không đủ, phát hành `inventory-failed`.

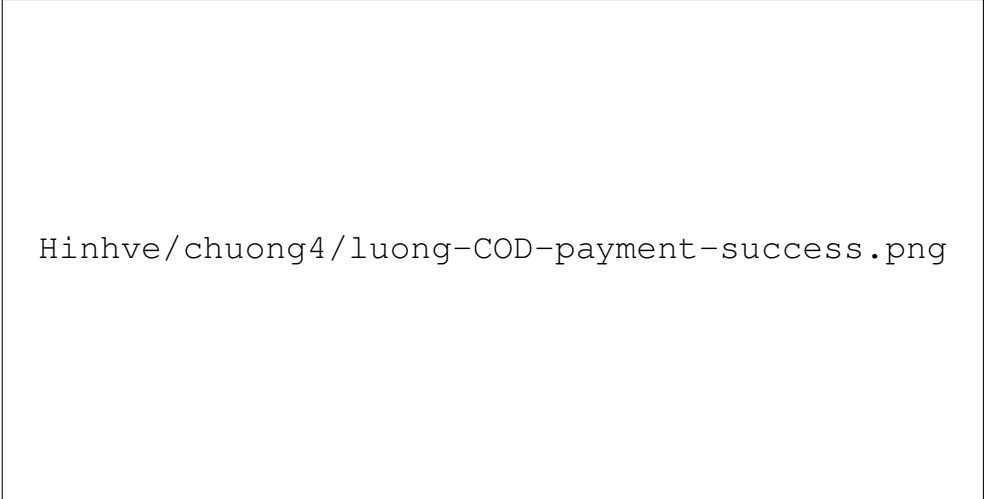


Hình 4.14: Biểu đồ tuần tự Bước 1–2 Saga: Tạo đơn hàng và đặt chỗ tồn kho qua Transactional Outbox

Bước 3 — Xác nhận đơn COD. Order-service tiêu thụ `inventory-updated`, chuyển trạng thái đơn hàng sang `STOCK_RESERVED`. Với phương thức COD, quá trình thanh toán được xử lý tự động: order-service phát hành `order-confirmed` ngay lập tức. Inventory-service tiêu thụ `order-confirmed`, thực hiện `confirm reservation` (trừ hẵn số lượng tồn kho). Notification-service tiêu thụ `order-confirmed`, gửi email xác nhận đơn hàng đến khách hàng. Đơn hàng

đạt trạng thái cuối cùng `CONFIRMED`.

Bước 4 — Xử lý thất bại. Nếu `inventory-service` phát hành `inventory-failed`, `order-service` chuyển trạng thái đơn hàng sang `CANCELLED` với lý do không đủ tồn kho, phát hành `order-cancelled`. `Inventory-service` và `voucher-service` tiêu thụ `order-cancelled` để hoàn trả đặt chỗ và voucher. `Notification-service` gửi email thông báo hủy đơn.

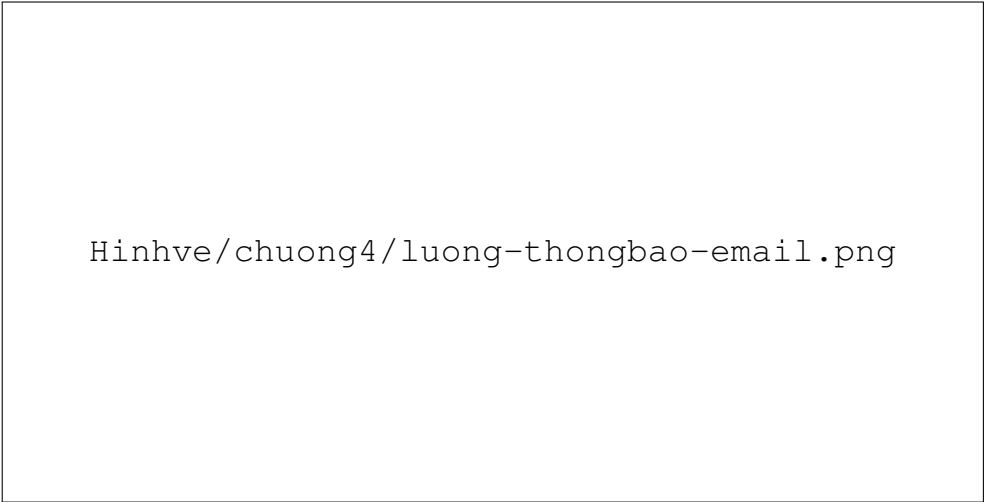


Hinhve/chuong4/luong-COD-payment-success.png

Hình 4.15: Biểu đồ tuần tự luồng đặt hàng COD — Saga Orchestration

4.10.4 Luồng thông báo email

`Notification-service` đóng vai trò trung tâm trong việc gửi thông báo đến khách hàng, hoạt động hoàn toàn theo cơ chế event-driven. Dịch vụ lắng nghe hai sự kiện chính từ Kafka: `order-confirmed` để gửi email xác nhận đơn hàng kèm chi tiết sản phẩm và tổng tiền; và `order-cancelled` để gửi email thông báo hủy đơn kèm lý do. Mỗi email được tạo từ template với các trường động được điền từ payload của sự kiện, sau đó gửi qua SMTP server — trong môi trường phát triển là Mailpit, trong môi trường production là dịch vụ email thật.



Hinhve/chuong4/luong-thongbao-email.png

Hình 4.16: Biểu đồ tuần tự luồng gửi thông báo email qua `notification-service`

4.10.5 Luồng đặt hàng VNPAY

Luồng VNPAY mở rộng luồng COD với các bước bổ sung cho thanh toán trực tuyến. Sau khi đơn hàng đạt trạng thái `STOCK_RESERVED`, order-service thiết lập `reservation_expired_at=NOW()+30` phút. Khách hàng gọi `POST/api/payments/vnpay/create` để tạo URL thanh toán. Payment-service sinh URL với chữ ký HMAC-SHA512 và trả về cho client. Khách hàng được chuyển hướng đến trang thanh toán VNPAY.

Khi VNPAY gọi IPN callback với kết quả thanh toán, payment-service xác minh chữ ký và cập nhật trạng thái. Nếu thành công, payment-service phát hành `payment-success` lên Kafka; order-service tiêu thụ và chuyển đơn hàng sang `CONFIRMED`. Nếu thất bại, phát hành `payment-failed` và đơn hàng bị hủy với cơ chế hoàn trả tương tự. Nếu sau 30 phút không có callback, Reservation-ExpiryScheduler (order-service) và PaymentTimeoutScheduler (payment-service) cùng hoạt động từ hai phía để đảm bảo đơn hàng và thanh toán đều được chuyển sang trạng thái kết thúc phù hợp.

Hìnhve/chuong4/luong-thanhtoan-thatbai.png

Hình 4.17: Biểu đồ tuần tự luồng xử lý thanh toán thất bại và hoàn trả tài nguyên

4.10.6 Luồng Flash Sale

Luồng flash sale được thiết kế đặc biệt để xử lý tình huống đồng thời cao với nhiều lớp bảo vệ. Khi khách hàng gọi `POST/api/flash-sales/{campaignId}/purchase`, flash-sale-service thực hiện tuần tự các bước kiểm tra. Đầu tiên, kiểm tra thời gian hiệu lực của chiến dịch (phải ở trạng thái `ACTIVE` và trong khoảng thời gian cho phép). Tiếp theo, thực thi Redis Lua script nguyên tử để kiểm tra buyer set và giảm số lượng slot: script kiểm tra khóa `flash-sale:{campaignId}:buyers` để chống mua trùng, sau đó kiểm tra khóa `flash-sale:{campaignId}:stock`; nếu stock lớn hơn 0 thì giảm 1 và thêm user vào buyer set, ngược lại trả về mã thất bại. Nếu Redis trả về thất bại,

Hinhve/chuong4/luong-VNPay-callback-
thanhcong.png

Hình 4.18: Biểu đồ tuần tự luồng đặt hàng VNPay với xử lý IPN callback

service ném lỗi nghiệp vụ (HTTP 400) với thông điệp tương ứng — sản phẩm đã bán hết hoặc người dùng đã mua trước đó.

Nếu Redis trả về thành công, flash-sale-service phát hành sự kiện `flash-sale-order-requested` lên Kafka rồi tăng `soldCount` trong database (nếu bước tăng `soldCount` lỗi sau khi Kafka đã ACK, `ReconciliationScheduler` sẽ bù lại sau). Order-service tạo đơn hàng flash sale với giá đã giảm theo chiến dịch và xử lý tương tự luồng COD. Nếu bước tạo đơn thất bại, flash-sale-service dùng compensation script để hoàn lại slot Redis và loại người dùng khỏi buyer set. `ReconciliationScheduler` định kỳ đối chiếu `soldCount` với số đơn flash-sale thực tế trong order-service để phát hiện và bù trừ slot mồi.

`CampaignScheduler` (chu kỳ 5 giây, với Redis distributed lock) đảm nhiệm ba nhiệm vụ nền: kích hoạt chiến dịch đến hạn (chuyển `SCHEDULED` sang `ACTIVE`, seed Redis stock), khôi phục khóa Redis bị mất (đọc `soldCount` từ database để tính lại stock còn lại), và kết thúc chiến dịch hết hạn (chuyển `ACTIVE` sang `ENDED`).

4.10.7 Luồng tìm kiếm sản phẩm (CQRS-lite)

Luồng tìm kiếm minh họa áp dụng CQRS-lite với hai đường dữ liệu độc lập. Đường ghi (Command): khi quản trị viên tạo hoặc cập nhật sản phẩm qua product-service, dữ liệu được lưu vào PostgreSQL (`product_db`). Đồng thời, product-service phát hành sự kiện `product-created` hoặc `product-updated` trực tiếp lên Kafka. Search-service tiêu thụ sự kiện và cập nhật document tương ứng trong Elasticsearch index. Đường đọc (Query): khi khách hàng tìm kiếm sản phẩm, request được gửi trực tiếp đến search-service, truy vấn Elasticsearch và trả về kết quả đã được tối ưu cho tìm kiếm toàn văn, không thông qua PostgreSQL. Hai đường

Hinhve/chuong4/luong-flash-sale.png

Hình 4.19: Biểu đồ tuần tự luồng mua hàng Flash Sale với cơ chế ba lớp bảo vệ dữ liệu hoạt động độc lập, chấp nhận độ trễ đồng bộ nhỏ (thường dưới 1 giây), đảm bảo eventual consistency.

Hinhve/chuong4/luong-tim-kiem-sp.png

Hình 4.20: Biểu đồ tuần tự luồng tìm kiếm sản phẩm theo mô hình CQRS-lite

4.10.8 Luồng đánh giá sản phẩm

Luồng đánh giá sản phẩm minh họa cơ chế xác minh quyền đánh giá dựa trên lịch sử mua hàng. Khi khách hàng gửi yêu cầu `POST/api/reviews`, `review-service` gọi `order-service` qua OpenFeign để kiểm tra người dùng đã có đơn hàng ở trạng thái `CONFIRMED` chứa sản phẩm cần đánh giá hay chưa. Chỉ khi điều kiện này được thỏa mãn, đánh giá mới được tạo và lưu vào `review_db`. Cơ chế này ngăn chặn đánh giá giả mạo từ người dùng chưa từng mua sản phẩm, đảm bảo tính xác thực của hệ thống đánh giá.

Hinhve/chuong4/luong-review-sp.png

Hình 4.21: Biểu đồ tuần tự luồng đánh giá sản phẩm

4.11 Thiết kế tác vụ nền — Scheduled Jobs

Hệ thống có sáu tác vụ nền (scheduled jobs) chạy định kỳ để xử lý các tình huống mà cơ chế event-driven không bao phủ được, đặc biệt là các tình huống liên quan đến timeout và phục hồi. Bảng 4.6 tổng hợp toàn bộ các tác vụ.

Bảng 4.6: Tác vụ nền trong hệ thống

Tác vụ	Dịch vụ	Chu kỳ	Nhiệm vụ
ReservationExpiry Scheduler	order-service	60 giây	Hủy đơn VNPAY quá hạn 30 phút, giải phóng voucher
PaymentTimeout Scheduler	payment-service	60 giây	Chuyển payment PENDING quá hạn sang TIMEOUT
OutboxPoller (Order)	order-service	1 giây	Phát hành sự kiện chưa gửi từ bảng outbox
OutboxPoller (Payment)	payment-service	1 giây	Phát hành sự kiện chưa gửi từ bảng payment_outbox
Campaign Scheduler	flash-sale-service	5 giây	Kích hoạt campaign, khôi phục Redis key, kết thúc campaign hết hạn
Reconciliation Scheduler	flash-sale-service	300 giây	Đối chiếu soldCount với số đơn flash-sale thực tế và bù slot mờ côi

Hai scheduler xử lý hết hạn reservation và timeout payment hoạt động song song để xử lý cùng một tình huống từ hai phía, tạo cơ chế dự phòng chéo: nếu một scheduler gặp sự cố hoặc bỏ sót, scheduler còn lại vẫn đảm bảo tài nguyên (tồn kho,

voucher) được giải phóng. OutboxPoller với chu kỳ 1 giây đảm bảo độ trễ phát hành sự kiện thấp trong điều kiện bình thường, đồng thời phục hồi các sự kiện bị bỏ lỡ sau khi service khởi động lại. CampaignScheduler sử dụng Redis distributed lock (SETNX với TTL) để đảm bảo chỉ một instance của flash-sale-service thực thi tại mỗi thời điểm, tránh xung đột trong môi trường nhiều replica.

CHƯƠNG 5. TRIỂN KHAI VÀ KIỂM THỬ

Chương này trình bày quá trình hiện thực hóa và kiểm chứng thiết kế đã phân tích ở Chương 4. Nội dung được chia thành hai phần có quan hệ nhân quả. Phần *Triển khai* (Mục 5.1–5.7) mô tả cách hệ thống được xây dựng, đóng gói và đưa vào vận hành: từ môi trường phát triển, cài đặt các thành phần backend cốt lõi, bảo mật, giao diện người dùng, đóng gói Docker và triển khai production-like trên AWS EC2, đến lớp giám sát vận hành. Phần *Kiểm thử* (Mục 5.8–5.10) chứng minh hệ thống chạy đúng và ổn định bằng các bằng chứng định lượng: smoke test theo suite, kiểm thử hiệu năng bằng k6 và kiểm thử khả năng chịu lỗi. Nguyên tắc xuyên suốt của chương là mọi phát biểu về kết quả đều phải đối chiếu được với mã nguồn, cấu hình hoặc artifact kiểm thử thực tế trong thư mục `.test/results`.

5.1 Môi trường phát triển và cấu trúc dự án

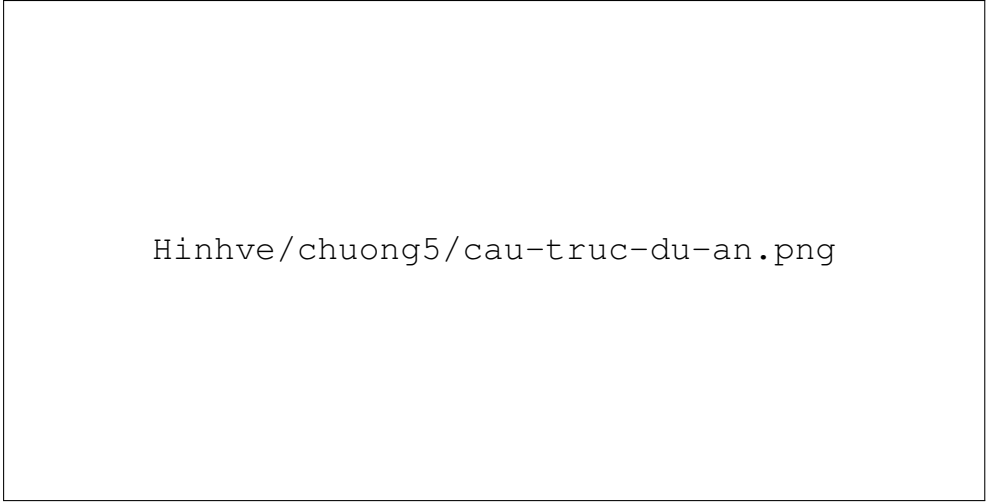
Hệ thống được phát triển theo mô hình monorepo nhằm thuận tiện cho việc quản lý dependency, build, kiểm thử và chạy toàn bộ stack trong phạm vi đồ án. Backend sử dụng Java 21, Spring Boot 3.5.13, Spring Cloud 2025.0.0 và Maven Wrapper để cố định phiên bản công cụ build giữa các máy. Frontend sử dụng Next.js App Router, TypeScript và Server Actions. Toàn bộ môi trường runtime được chuẩn hóa bằng Docker Compose, giúp lập trình viên dựng lại nguyên trạng hệ thống chỉ bằng một lệnh.

Bảng 5.1: Môi trường và công cụ phát triển

Nhóm	Công cụ/Phiên bản	Vai trò trong đồ án
Ngôn ngữ back-end	Java 21	Phát triển 13 business service và 3 infrastructure service
Framework back-end	Spring Boot 3.5.13, Spring Cloud 2025.0.0	REST API, Gateway, Eureka, Config, OpenFeign, Kafka, Security, Actuator
Build backend	Maven Wrapper	Build multi-module, quản lý dependency và plugin tập trung
Frontend	Next.js App Router, TypeScript	Storefront, Admin Panel, BFF/proxy, Server Actions
Container	Docker Compose	Khởi động local stack và production-like stack trên EC2
Kiểm thử	JUnit 5, Testcontainers, k6, Bash script	Unit, integration, smoke, performance và resilience test
Triển khai	AWS EC2, Caddy, GitHub Actions, GHCR	Production-like single-host deployment và build/push image

Về cấu trúc mã nguồn, repository gồm module `common` (chứa DTO sự kiện, hằng số Kafka topic và tiện ích dùng chung), 13 business service như `identi`

ty, user, product, inventory, cart, order, payment, voucher, notification, review, search, content và flash-sale; ba service hạ tầng api-gateway, discovery-server, config-server; và thư mục frontend. Ngoài ra, repository còn chứa các thư mục hạ tầng vận hành như prometheus, grafana, keycloak, init-db và aws. Mỗi service backend có Dockerfile riêng theo mô hình multi-stage build; các service stateful dùng Flyway migration để quản lý schema.



Hinhve/chuong5/cau-truc-du-an.png

Hình 5.1: Cấu trúc thư mục monorepo và các module chính của hệ thống

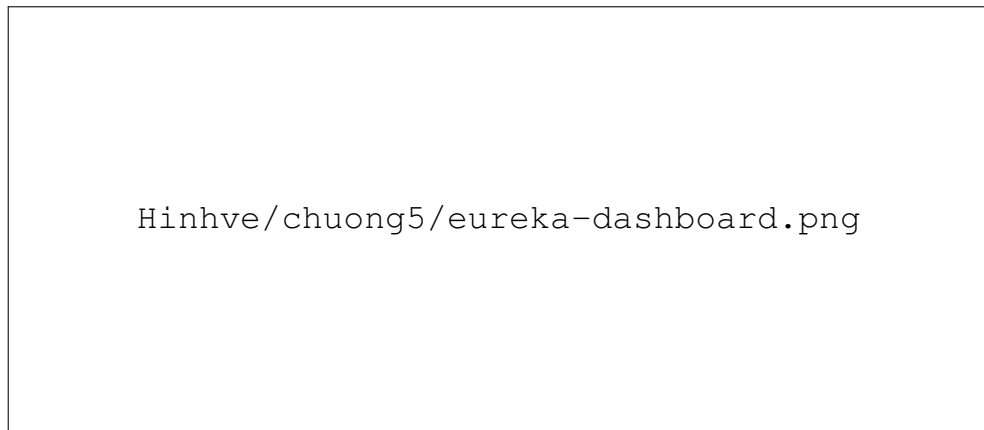
5.2 Cài đặt các thành phần backend cốt lõi

5.2.1 Lớp điều phối Spring Cloud

Ba service hạ tầng tạo thành lớp điều phối cho toàn bộ backend. api-gateway là điểm vào duy nhất cho client, chịu trách nhiệm định tuyến, xác thực JWT, gắn trusted header và rate limiting. discovery-server sử dụng Eureka để các service đăng ký và phát hiện lẫn nhau theo tên logic thay vì hard-code địa chỉ, nhờ đó các lời gọi OpenFeign có thể load-balance phía client. config-server cung cấp cấu hình tập trung từ classpath, giúp các service lấy cấu hình theo profile khi khởi động và giảm trùng lặp cấu hình giữa các service.

5.2.2 Database per Service và Flyway

Các service stateful sở hữu database logic riêng trên PostgreSQL. Cách tổ chức này đảm bảo nguyên tắc Database per Service: service khác không đọc/ghi trực tiếp vào database không thuộc quyền sở hữu của mình mà phải đi qua API hoặc sự kiện. Schema được quản lý bằng Flyway migration (các file V1__*.sql, V2__*.sql... theo thứ tự phiên bản) và Hibernate chạy với chế độ validate; nhờ đó sai lệch giữa entity và schema được phát hiện sớm ngay khi service khởi động thay vì khi truy vấn dữ liệu trong lúc chạy.



Hình 5.2: Eureka dashboard: 16 service đăng ký thành công và ở trạng thái UP

Bảng 5.2: Map service với lớp lưu trữ chính

Service	Lưu trữ chính	Ghi chú
identity-service	Keycloak	Tạo user trong realm, phát sự kiện đăng ký
user-service	PostgreSQL	Hồ sơ người dùng, địa chỉ, dữ liệu admin user
product-service	PostgreSQL, Redis	Catalog, category, brand, cache product list/detail
inventory-service	PostgreSQL	Tồn kho, reserved quantity, stock movement
cart-service	Redis	Giỏ hàng user/guest với TTL
order-service	PostgreSQL, Kafka	Order, order item, outbox, processed events
payment-service	PostgreSQL, Kafka	Payment, payment outbox, processed events
search-service	Elasticsearch	Read model phục vụ full-text search
flash-sale-service	PostgreSQL, Redis, Kafka	Campaign, Redis Lua reservation, reconciliation

5.2.3 Saga Orchestration, Outbox và Idempotent Consumer

Luồng đặt hàng là phần thể hiện rõ nhất các mẫu thiết kế phân tán. `order-service` đóng vai trò orchestrator: tạo đơn, đặt chỗ voucher, phát sự kiện `order-created`, nhận kết quả từ `inventory/payment` và quyết định xác nhận hoặc hủy đơn. Để tránh lỗi dual-write (ghi database thành công nhưng phát Kafka thất bại hoặc ngược lại), `order-service` ghi sự kiện vào bảng `outbox` trong cùng *transaction* với dữ liệu đơn hàng. Một tiến trình nền `OutboxPoller` quét bảng này theo chu kỳ 1 giây, lấy batch chưa xử lý bằng `FOR UPDATE SKIP LOCKED`, phát Kafka rồi đánh dấu `processed=true`.

Đoạn mã 5.1 mô tả cốt lõi của `OutboxPoller`. Mệnh đề `FOR UPDATE SKIP LOCKED` cho phép nhiều instance chạy song song mà không tranh cùng một batch: instance nào đã khóa hàng nào thì instance khác bỏ qua hàng đó, tránh phát trùng sự kiện.

```

1 @Scheduled(fixedDelay = 1000)
2 @Transactional
3 public void pollAndPublish() {
4     List<Outbox> batch = outboxRepository
5         .findUnprocessedWithLock(PageRequest.of(0, 50)); // FOR
6         UPDATE SKIP LOCKED
7     for (Outbox event : batch) {
8         kafkaTemplate.send(event.getTopic(),
9                             event.getAggregateId(),
10                            event.getPayload());
11         event.setProcessed(true);
12         event.setProcessedAt(Instant.now());
13     }
14 }

```

Đoạn mã 5.1: OutboxPoller — đọc batch và phát Kafka

Ở phía consumer, các service như order, inventory và payment sử dụng bảng `processed_events` để lưu `eventId` đã xử lý. Nếu Kafka gửi lại cùng một event, consumer kiểm tra bảng này, phát hiện đã xử lý và bỏ qua thay vì thực thi nghiệp vụ lần hai. Cách làm này chấp nhận ngữ nghĩa at-least-once delivery ở tầng message broker nhưng đạt hiệu ứng exactly-once processing ở tầng ứng dụng — một lựa chọn phù hợp vì Kafka KRaft chế độ đơn broker trong đồ án không bảo đảm exactly-once ở tầng hạ tầng.

5.2.4 Thanh toán VNPAY

`payment-service` tích hợp VNPAY sandbox theo ba bước chính. Khi người dùng chọn thanh toán online, service tạo payment ở trạng thái PENDING, sinh URL thanh toán và ký toàn bộ tham số bằng HMAC-SHA512. Khi VNPAY gọi IPN hoặc người dùng quay lại Return URL, service xác minh chữ ký, kiểm tra số tiền, cập nhật trạng thái payment và ghi sự kiện vào `payment_outbox`. `PaymentEventOutboxPoller` phát sự kiện `payment-success` hoặc `payment-failed` để `order-service` hoàn tất Saga. Nếu payment quá hạn, `PaymentTimeoutScheduler` chuyển trạng thái sang TIMEOUT.

Điểm cốt lõi về bảo mật của tích hợp này là quy trình ký và xác minh chữ ký. Tham số được sắp xếp theo thứ tự từ điển, encode rồi nối thành chuỗi `signing data`; chuỗi này được ký bằng khóa bí mật theo HMAC-SHA512 và biểu diễn dạng hexa (Đoạn mã 5.2). Khi nhận callback, service tính lại chữ ký từ chính các tham số nhận được và so sánh với `vnp_SecureHash`; chỉ khi trùng khớp, giao dịch mới được công nhận, nhờ đó chống được việc giả mạo kết quả thanh toán.

```

1 public static String hmacSha512(String secret, String data) {

```

```

2      try {
3          Mac hmac = Mac.getInstance("HmacSHA512");
4          hmac.init(new SecretKeySpec(
5              secret.getBytes(StandardCharsets.UTF_8), "
HmacSHA512"));
6          byte[] bytes = hmac.doFinal(data.getBytes(
StandardCharsets.UTF_8));
7          StringBuilder result = new StringBuilder(bytes.length *
2);
8          for (byte b : bytes) {
9              result.append(String.format("%02x", b)); // hex
encoding
10             }
11             return result.toString();
12         } catch (Exception ex) {
13             throw new IllegalStateException("Cannot sign VNPay
payload", ex);
14         }
15     }

```

Đoạn mã 5.2: Ký tham số VNPAY bằng HMAC-SHA512

5.2.5 Flash sale high-concurrency

flash-sale-service xử lý mua flash sale bằng Redis Lua script để thao tác kiểm tra buyer set, kiểm tra stock và giảm slot trong một lệnh nguyên tử. Redis key chính có dạng flash-sale:{campaignId}:stock và flash-sale:{campaignId}:buyers, được đặt TTL theo thời điểm kết thúc chiến dịch. Sau khi giữ slot thành công, service phát event flash-sale-order-requested; nếu tạo đơn thất bại, compensation script hoàn lại slot và xóa buyer khỏi set.

Đoạn mã 5.3 là phần cốt lõi của Lua script. Vì Redis thực thi mỗi script như một thao tác nguyên tử, ba bước kiểm tra duplicate buyer, kiểm tra stock và cập nhật diễn ra trong một khối không thể xen ngang, loại bỏ race condition vốn là nguyên nhân gây over-sell trong kịch bản nhiều người mua tranh nhau cùng một slot.

```

1 local buyers_key = KEYS[1]  -- flash-sale:{id}:buyers
2 local stock_key  = KEYS[2]  -- flash-sale:{id}:stock
3 local user_id    = ARGV[1]
4
5 if redis.call('SISMEMBER', buyers_key, user_id) == 1 then
6     return -1  -- already purchased by this user
7 end
8 local stock = tonumber(redis.call('GET', stock_key) or '0')
9 if stock <= 0 then

```

```

10     return 0    -- sold out
11 end
12 redis.call('DECR', stock_key)
13 redis.call('SADD', buyers_key, user_id)
14 return 1      -- reservation success

```

Đoạn mã 5.3: Redis Lua script — atomic reservation cho flash sale

Hai scheduler hỗ trợ flash-sale vận hành ổn định. CampaignScheduler chạy 5 giây/lần, dùng Redis distributed lock để chỉ một instance kích hoạt chiến dịch đến hạn, khôi phục Redis stock key bị mất và kết thúc campaign hết hạn. ReconciliationScheduler chạy 300 giây/lần, đối chiếu soldCount trong flash-sale-service với số order flash-sale thực tế ở order-service, từ đó phát hiện và bù slot mờ côi khi cần.

5.3 Cài đặt bảo mật và phân quyền

Bảo mật được thực thi tập trung tại API Gateway theo Trusted Subsystem Pattern. Client gửi access token dạng `Authorization:Bearer<token>` tới Gateway; Gateway xác minh chữ ký RS256 qua JWKS endpoint của Keycloak, kiểm tra thời hạn token, trích xuất `userId/email/roles` rồi gắn vào ba trusted header: `X-User-Id`, `X-User-Email`, `X-User-Roles`. Backend service phía sau chỉ tin các header này trong mạng nội bộ Docker và không mở cổng trực tiếp ra ngoài.

Một chi tiết bảo mật quan trọng là chống giả mạo trusted header. Trước khi gắn danh tính từ JWT, Gateway luôn *xóa sạch* mọi header danh tính do client tự đưa vào, bảo đảm client không thể tự khai báo `X-User-Id` để mạo danh người dùng khác. Đoạn mã 5.4 thể hiện logic này: chuỗi xử lý strip header trước, sau đó chỉ khi có JWT hợp lệ mới gắn lại danh tính trích xuất từ token.

```

1 public Mono<Void> filter(ServerWebExchange exchange,
   GatewayFilterChain chain) {
2     // Strip client-supplied identity headers first to prevent
   spoofing
3     ServerWebExchange stripped = stripIdentityHeaders(exchange)
   ;
4     return ReactiveSecurityContextHolder.getContext()
   .map(ctx -> ctx.getAuthentication())
5     .filter(auth -> auth != null && auth.getPrincipal()
   instanceof Jwt)
6     .map(auth -> (Jwt) auth.getPrincipal())
7     .map(jwt -> withIdentityHeaders(stripped, jwt)) // X-
   User-Id/Email/Roles
8     .defaultIfEmpty(stripped)
9     .flatMap(chain::filter);
10

```



```
11 }
```

Đoạn mã 5.4: AuthHeaderFilter — xóa header giả mạo và gắn trusted header từ JWT

Trên nền tảng xác thực đó, Gateway áp dụng chính sách phân quyền theo nhóm endpoint (Bảng 5.3). Các endpoint công khai phục vụ duyệt catalog và nội dung; các endpoint nghiệp vụ yêu cầu token hợp lệ; các endpoint quản trị yêu cầu vai trò `ROLE_ADMIN`.

Bảng 5.3: Chính sách phân quyền tại API Gateway

Nhóm endpoint	Quyền yêu cầu	Ghi chú
<code>GET/api/products/**</code> , <code>GET/api/search/**</code> , <code>GET/api/content/**</code>	Public	Cho phép người dùng duyệt catalog, search và nội dung CMS
<code>/api/cart/**</code>	Public hoặc token	Hỗ trợ guest cart bằng session id và user cart bằng token
<code>POST/api/orders</code> , <code>user profile</code> , <code>review create</code>	Authenticated	Yêu cầu token hợp lệ và trusted header
<code>/api/orders/admin/**</code> , <code>/api/users/admin/**</code> , <code>/api/inventory/admin/**</code> , <code>/api/reviews/admin/**</code>	<code>ROLE_ADMIN</code>	Endpoint quản trị bổ sung trong Phase 14
<code>Product/voucher/flash-sale/content mutations</code>	<code>ROLE_ADMIN</code>	<code>POST/PUT/DELETE</code> được bảo vệ theo method và path
<code>POST/api/payments/vnpay/ipn</code>	Public có HMAC	Public vì do VNPAY gọi, nhưng bắt buộc xác minh chữ ký

Ngoài xác thực và RBAC, Gateway áp dụng Redis token bucket rate limiter cho các endpoint nhạy cảm, với `KeyResolver` ưu tiên định danh theo `X-User-Id`, sau đó đến `X-Forwarded-For` và cuối cùng là địa chỉ remote. Luồng đặt hàng được giới hạn 10 request/giây, burst 20; luồng mua flash sale được giới hạn 3 request/giây, burst 5. Rate limiter không thay thế kiểm soát nghiệp vụ nhưng giúp giảm tải và hạn chế automation trong các luồng có rủi ro cao.

Hinhve/chuong5/security-flow.png

Hình 5.3: Luồng bảo mật: xác thực JWT tại Gateway và chèn trusted header cho backend

5.4 Cài đặt frontend Storefront và Admin Panel

Frontend được xây dựng trong thư mục `frontend/` bằng Next.js App Router. Ứng dụng có hai khu vực chính: storefront cho khách hàng và Admin Panel cho quản trị viên. Storefront hỗ trợ duyệt sản phẩm, xem chi tiết, tìm kiếm, giỏ hàng, checkout, xem đơn hàng, thanh toán VNPAY và các màn hình flash-sale, content, review phục vụ demo. Admin Panel nằm dưới route `/admin/`, dùng chung codebase và container với storefront.

Hinhve/chuong5/storefront-overview.png

Hình 5.4: Giao diện Storefront: trang chủ, danh sách sản phẩm, giỏ hàng và checkout

Các mutation trong Admin Panel được thực hiện bằng Server Actions. Action gọi `adminFetch` hoặc `adminMutate`, phía server đọc access token từ `httpOnly` cookie và gọi API Gateway. Sau khi mutation thành công, action `revalidatePath` để làm mới dữ liệu và redirect về trang danh sách. Thiết kế này giúp token không xuất hiện trong JavaScript phía trình duyệt, đồng thời vẫn tận dụng được cơ chế caching/revalidation của Next.js. Bảng 5.4 liệt kê các module quản trị và endpoint backend tương ứng.

Bảng 5.4: Các module chính của Admin Panel

Module	Route frontend	Endpoint backend chính
Dashboard	/admin/dashboard	/api/orders/admin/analytics/*, /api/inventory/low-stock
Products	/admin/products	/api/products
Inventory	/admin/inventory	/api/inventory/admin, /api/inventory/stock-in, /api/inventory/stock-out
Orders	/admin/orders	/api/orders/admin
Users	/admin/users	/api/users/admin
Vouchers	/admin/vouchers	/api/vouchers
Flash sales	/admin/flash-sales	/api/flash-sales
Content/Reviews	/admin/content/*, /admin/reviews	/api/content/*, /api/reviews/admin

Hinhve/chuong5/admin-panel-overview.png

Hình 5.5: Admin Panel: dashboard, quản lý sản phẩm, tồn kho, đơn hàng và flash sale

Giới hạn của frontend trong phạm vi đồ án được ghi nhận rõ: ảnh sản phẩm/banner nhập bằng URL thay vì upload file thật; review-service mới hỗ trợ list/delete, chưa có workflow approve/reject đầy đủ; chức năng ban/unban user chưa tích hợp Keycloak Admin API; dashboard analytics tổng hợp trực tiếp từ order-service, phù hợp demo nhưng chưa phải kiến trúc analytics quy mô lớn. Các giới hạn này được phân tích đầy đủ hơn trong phần Kết luận và hướng phát triển.

5.5 Đóng gói Docker và vận hành local

Mỗi service backend được đóng gói bằng Dockerfile multi-stage build: stage build dùng image Maven để biên dịch, stage runtime dùng image JRE Alpine

gọn nhẹ chỉ chứa file JAR. Cách này giảm đáng kể kích thước image cuối và tách biệt môi trường build khỏi môi trường chạy (Đoạn mã 5.5).

```

1 FROM maven:3.9.11-eclipse-temurin-21 AS build
2 WORKDIR /workspace
3 COPY pom.xml .
4 COPY common/pom.xml common/pom.xml
5 # ... copy pom.xml of all modules to leverage dependency cache
   layer
6 COPY common/src common/src
7 COPY content-service/src content-service/src
8 RUN mvn -q -pl content-service -am package -DskipTests
9
10 FROM eclipse-temurin:21-jre-alpine
11 WORKDIR /app
12 COPY --from=build /workspace/content-service/target/*.jar app.
   jar
13 EXPOSE 8092
14 ENTRYPOINT ["java", "-jar", "app.jar"]

```

Đoạn mã 5.5: Dockerfile multi-stage tiêu biểu (content-service)

Môi trường local dùng `docker-compose.yml` để khởi động toàn bộ backend và hạ tầng. Các container được đặt chung network `ecommerce-network`; các stateful service dùng Docker volume để giữ dữ liệu giữa các lần restart. Healthcheck được khai báo cho PostgreSQL, Redis, Kafka, Elasticsearch, Keycloak, Eureka và các service Spring Boot. Nhờ kết hợp healthcheck với `depends_on: condition: service_healthy`, Compose khởi động theo thứ tự phụ thuộc thực sự thay vì chỉ dựa vào thứ tự khai báo (Đoạn mã 5.6).

```

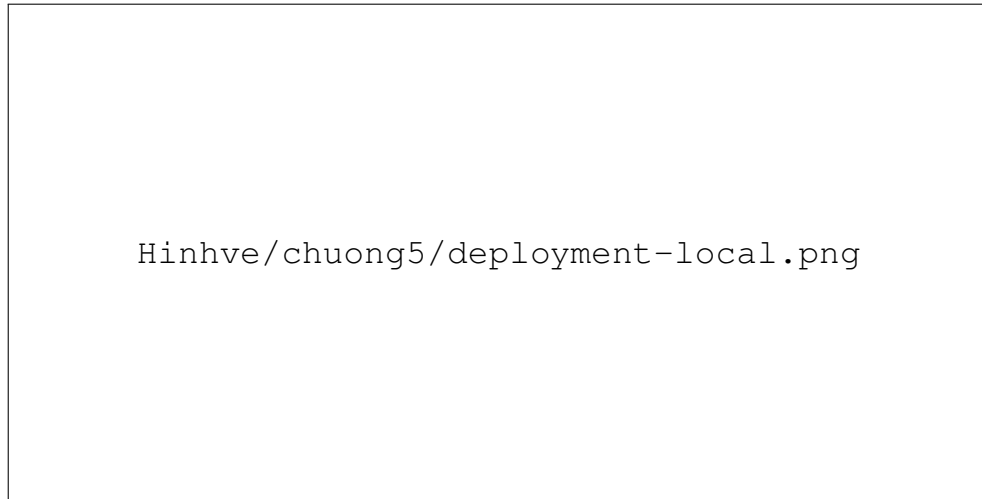
1 postgres:
2   healthcheck:
3     test: ["CMD-SHELL", "pg_isready -U postgres"]
4     interval: 5s
5     timeout: 3s
6     retries: 5
7 redis:
8   healthcheck:
9     test: ["CMD", "redis-cli", "ping"]
10    interval: 5s
11    timeout: 3s
12    retries: 5

```

Đoạn mã 5.6: Khai báo healthcheck cho PostgreSQL và Redis trong docker-compose

Các endpoint vận hành local quan trọng gồm API Gateway `http://loca`

localhost:8080, Eureka <http://localhost:8761>, Keycloak <http://localhost:8180>, Prometheus <http://localhost:9090>, Zipkin <http://localhost:9411>, Mailpit <http://localhost:8025> và Elasticsearch <http://localhost:9200>. Grafana mặc định dùng cổng 3000 trong local backend stack; khi chạy frontend cùng lúc cần remap hoặc dùng cấu hình production nơi Grafana được đưa sang cổng 3001.



Hình 5.6: Sơ đồ triển khai Docker Compose ở môi trường local

5.6 Triển khai production-like trên AWS EC2

Môi trường triển khai thực nghiệm sử dụng một máy ảo AWS EC2 single-host tại khu vực ap-southeast-1. Đây không phải kiến trúc high availability, nhưng phù hợp với phạm vi DATN: đủ để chứng minh hệ thống có thể build image, chạy container, cấu hình reverse proxy/HTTPS, giám sát runtime và thực hiện các bài kiểm thử hiệu năng/resilience trên môi trường gần thực tế.

Bảng 5.5: Thành phần triển khai production-like

Thành phần	Vai trò
AWS EC2	Máy chủ single-host chạy Docker Compose production stack
Docker Compose production override	Dùng GHCR image, cấu hình memory limit/reservation, remap Grafana 3001
Caddy	Reverse proxy, HTTPS tự động qua Let's Encrypt và nlp.io
GitHub Actions	Build/push backend và frontend image lên GHCR, hỗ trợ deploy lên EC2
GHCR	Registry lưu Docker image theo service

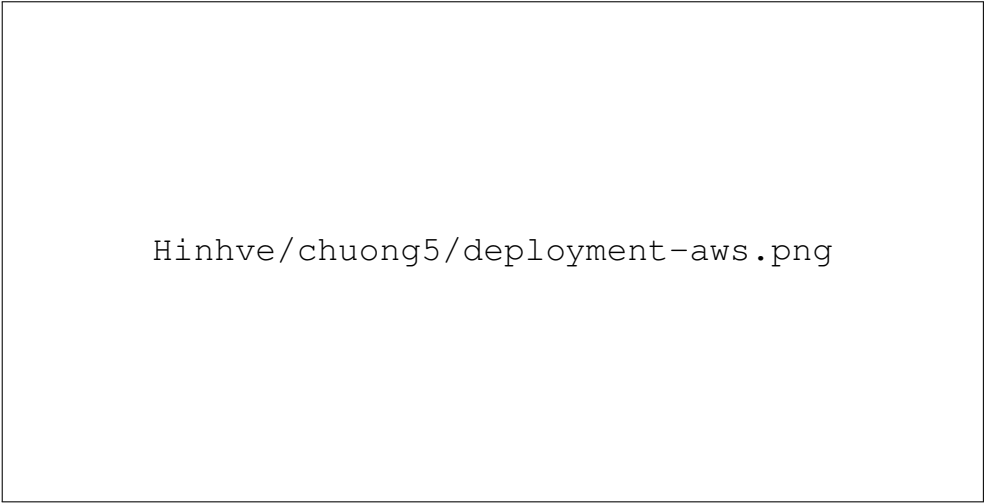
File `docker-compose.prod.yml` không build image trực tiếp trên EC2 mà tham chiếu image theo dạng `ghcr.io/{owner}/{service}:{tag}`. Cách này giảm thời gian deploy trên máy chủ, tách build khỏi runtime và giúp cùng một image có thể được kéo lại khi cần rollback. Production override cũng đặt `mem_limit` và `mem_reservation` cho từng container (ví dụ 768m

cho service nặng, 128m–256m cho service nhẹ), giúp stack ổn định hơn trong giới hạn tài nguyên của EC2. Caddy định tuyến các subdomain `app.*`, `api.*`, `keycloak.*` và `grafana.*`; Grafana dùng host port 3001 để không xung đột với frontend.

Quy trình CI/CD được tự động hóa bằng GitHub Actions. Pipeline `build-and-push.yml` được kích hoạt khi có thay đổi mã nguồn trên nhánh `main`: nó build toàn bộ service bằng Maven Wrapper, gắn tag image theo short SHA của commit rồi push lên GHCR (Đoạn mã 5.7). Việc gắn tag theo SHA cho phép truy vết chính xác image nào tương ứng với commit nào và quay lại phiên bản trước khi cần.

```
1 name: Build & Push Images
2 on:
3   push:
4     branches: [main]
5     paths: ['**/pom.xml', '**/src/**', '**/Dockerfile.prod']
6 jobs:
7   build-maven:
8     runs-on: ubuntu-latest
9     steps:
10      - uses: actions/checkout@v4
11      - uses: actions/setup-java@v4
12        with: { distribution: temurin, java-version: 21, cache:
13          maven }
14      - name: Build all services
15        run: ./mvnw -B -T 1C clean package -DskipTests
16      - name: Compute short SHA
17        run: echo "sha-short=$(git rev-parse --short HEAD)" >>
18          $GITHUB_OUTPUT
```

Đoạn mã 5.7: Trích GitHub Actions workflow build/push image lên GHCR



Hinhve/chuong5/deployment-aws.png

Hình 5.7: Sơ đồ triển khai production-like trên AWS EC2 với Caddy và GHCR



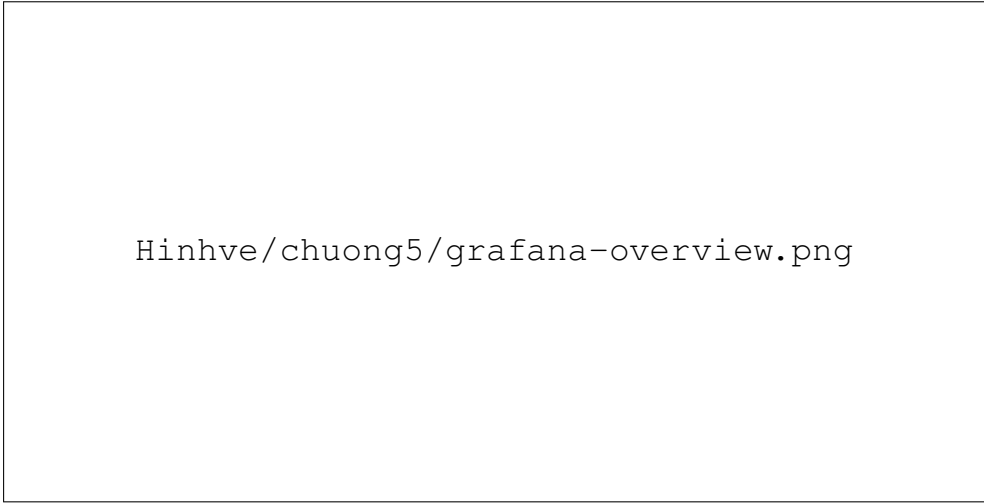
Hinhve/chuong5/github-actions.png

Hình 5.8: GitHub Actions build/push image lên GHCR thành công

5.7 Giám sát và quan sát hệ thống

Sau khi triển khai, khả năng quan sát (observability) là yếu tố then chốt để vận hành và phục vụ kiểm thử. Hệ thống áp dụng cả ba trụ cột quan sát. Về *metrics*, mỗi service Spring Boot expose endpoint `/actuator/prometheus`; Prometheus thu thập định kỳ và Grafana trực quan hóa qua các dashboard theo nhóm (Spring Boot Overview, JVM Overview, Saga Overview). Về *tracing*, Micrometer Tracing kết hợp Zipkin cho phép lần theo một request đi qua nhiều service, bao gồm cả các trace của tiến trình nền như OutboxPoller. Về *logging*, các service ghi structured log ra stdout để Docker thu gom.

Lớp quan sát này không chỉ phục vụ vận hành mà còn là nguồn bằng chứng hỗ trợ cho phần kiểm thử: biểu đồ p95 latency, request rate và JVM heap của Grafana được dùng để quan sát hệ thống trong lúc chạy tải k6, còn Zipkin giúp xác nhận đường đi và thời gian của các luồng nghiệp vụ quan trọng.



Hinhve/chuong5/grafana-overview.png

Hình 5.9: Grafana dashboard: Spring Boot Overview, JVM Overview và Saga Overview



Hinhve/chuong5/zipkin-trace.png

Hình 5.10: Zipkin: trace của order-service và tiến trình OutboxPoller với outcome SUCCESS

5.8 Chiến lược kiểm thử

Kiểm thử được tổ chức theo mô hình kim tự tháp nhiều tầng. Tầng thấp nhất là unit test và integration test (dùng Testcontainers để dựng PostgreSQL/Redis/Kafka thật) cho logic nghiệp vụ quan trọng. Tầng tiếp theo là backend readiness smoke test, chạy theo các suite tách nhỏ để tránh quá tải tài nguyên và giúp khoanh vùng lỗi nhanh. Tầng cao hơn là kiểm thử hiệu năng bằng k6 trên môi trường AWS EC2 và kiểm thử khả năng chịu lỗi bằng cách chủ động dừng service/hạ tầng rồi quan sát khả năng tự phục hồi.

Smoke test được chia thành chín suite theo miền chức năng và chạy tuần tự. Toàn bộ chín suite đều đạt (Bảng 5.6), với tổng cộng hơn 400 lượt kiểm tra thành công.

Bảng 5.6: Kết quả backend readiness smoke test

Suite	Kết quả	Artifact
Maven	PASS, 5 pass, 0 fail	01-maven-PASS.md
Infra	PASS, 27 pass, 0 fail	02-infra-PASS.md
Auth/user	PASS, 30 pass, 0 fail	03-auth-user-PASS-after-fix.md
Catalog/search	PASS, 49 pass, 0 fail	04-catalog-search-PASS.md
Cart/voucher	PASS, 57 pass, 0 fail	05-cart-voucher-PASS-after-fix.md
Order COD/review	PASS, 67 pass, 0 fail	06-order-cod-review-PASS.md
VNPAY	PASS, 55 pass, 0 fail	07-vnpay-PASS.md
Flash-sale	PASS, 62 pass, 0 fail	08-flash-sale-PASS-after-fix.md
Security	PASS, 76 pass, 0 fail, 1 warn	09-security-PASS-trimmed-memory.md

Kết quả tổng hợp cho thấy 9/9 split suites đạt. Một số lỗi phát hiện trong quá trình chuẩn bị kiểm thử — như healthcheck bị 401, guest cart session id bị reject, lỗi serializer cache sản phẩm và cấu hình Metaspace cho flash-sale test — đều đã được sửa trước khi ghi nhận kết quả final, đúng tinh thần dùng smoke test để phát hiện sớm lỗi cấu hình.

5.9 Kết quả kiểm thử hiệu năng

Các bài kiểm thử hiệu năng được thực hiện trên môi trường AWS EC2 chạy Docker Compose production-like stack; load generator là máy cá nhân chạy k6, trở tới base URL `http://13.213.118.96:8080`. Dataset thực nghiệm gồm 100 sản phẩm, 500 user/token và một flash-sale campaign có 100 slot. Nguyên tắc đưa số liệu vào báo cáo là mọi metric phải đối chiếu được với raw artifact trong thư mục `.test/results`. Bảng 5.7 tổng hợp ba kịch bản; các tiểu mục sau phân tích chi tiết từng kịch bản.

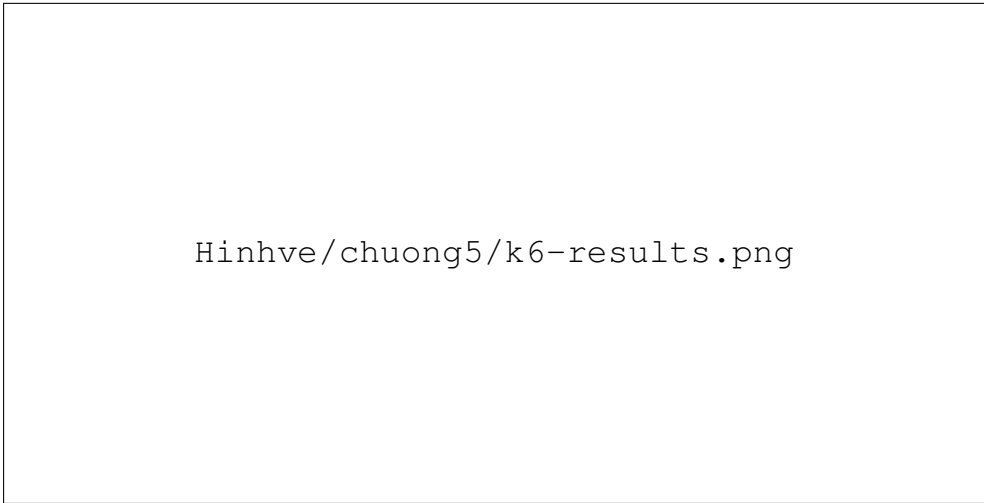
Bảng 5.7: Tổng hợp kết quả kiểm thử hiệu năng bằng k6

Kịch bản	Thời lượng	Max VU	Requests	p95	Lỗi
Catalog soak	34m03s	200	220.956	61,68 ms	0,00%
Checkout stress	7m00s	50	13.734	160,92 ms	0,00%
Flash-sale spike	1m30s	500	78.448	1,04 s (tổng thể)	0 duplicate buyer

Catalog soak mô phỏng người dùng duyệt danh sách sản phẩm trong 34 phút 03 giây với tải tối đa 200 VU theo hồ sơ $100 \rightarrow 200 \rightarrow 0$. Kịch bản tạo 55.239 iterations và 220.956 HTTP requests, p95 đạt 61,68 ms, p99 đạt 85,48 ms, error rate 0,00% với 220.956/220.956 check thành công. So với ngưỡng đặt ra (p95 < 800 ms, p99 < 2000 ms, lỗi < 1%), kết quả vượt xa kỳ vọng, cho thấy read path catalog/search/cache đáp ứng tốt trên single-host trong phạm vi dataset đồ án.

Checkout stress mô phỏng tạo đơn hàng với tải ổn định 50 VU trong 7 phút. Kịch bản ghi nhận 6.817 iterations, 13.734 HTTP requests, order success check 100,00%, HTTP failure rate 0,00% và p95 latency 160,92 ms. Đây là luồng nặng hơn catalog vì mỗi đơn đi qua product, inventory, voucher, order, Kafka và notification, nhưng độ trễ vẫn nằm trong mức phù hợp với mục tiêu thực nghiệm, xác nhận luồng Saga đặt hàng hoạt động ổn định dưới tải.

Flash-sale spike mô phỏng 500 VU trong 90 giây tranh mua 100 slot. Kết quả có đúng 100 purchase thành công trên 100 slot, soldCount của campaign bằng 100, số đơn confirmed cho campaign bằng 100 và truy vấn duplicate buyer trả về 0 dòng. Bài test ghi nhận 36.134 phản hồi sold-out; p95 tổng thể là 1,04 giây, còn p95 của nhóm expected responses là 335,55 ms. Điều này chứng minh trong kịch bản thực nghiệm đã chạy, Redis Lua atomic reservation kết hợp buyer set, compensation và reconciliation không gây over-sell ngay cả dưới tải đột biến.



Hình ảnh kết quả kiểm thử k6 ba kịch bản: catalog soak, checkout stress và flash-sale spike.

Hình 5.11: Kết quả k6 ba kịch bản: catalog soak, checkout stress và flash-sale spike

5.10 Kết quả kiểm thử khả năng chịu lỗi

Để đánh giá khả năng chịu lỗi, bốn kịch bản được thực hiện bằng cách chủ động gây lỗi vào service hoặc hạ tầng. Mục tiêu không phải chứng minh hệ thống có high availability hoàn chỉnh, mà là kiểm chứng các cơ chế đã thiết kế: circuit breaker/fallback, outbox replay, graceful degradation và Saga compensation (Bảng 5.8).

Bảng 5.8: Tổng hợp kết quả kiểm thử khả năng chịu lỗi

Kịch bản	Kết quả chính	Trạng thái
Kill order-service	Trong 30 giây downtime, order-created đạt 47,15%, HTTP failure 25,94%; sau restart gateway/order health trả 200	Completed
Kill Kafka	Khi Kafka down, order vẫn được tạo, outbox row ở trạng thái pending; sau Kafka restart, event được process và order kết thúc CONFIRMED	PASS
Kill Redis	Product list vẫn trả 200 trong Redis outage; cart phụ thuộc Redis degrade 500 trong outage và recover 200 sau restart	PASS
Inventory failure compensation	Order 4170422f-...-c7cd1d79266a chuyển CANCELLED, inventory còn quantity=0, reserved_quantity=0	PASS

Kịch bản **Kill Kafka** là bằng chứng trực tiếp cho Transactional Outbox: service nghiệp vụ vẫn ghi được dữ liệu và ghi event pending vào database khi broker không khả dụng; sau khi broker khởi động lại, các sự kiện ORDER_CREATED, PAYMENT_REQUESTED và ORDER_CONFIRMED lần lượt được đánh dấu đã xử lý và Saga tiếp tục đến trạng thái cuối CONFIRMED. Kịch bản **Kill Redis** cho thấy hệ thống phân biệt rõ read path có fallback (product list vẫn 200) với chức năng phụ thuộc trực tiếp Redis (cart trả 500 trong outage, recover 200 sau khi Redis trở lại). Cách ghi nhận này trung thực với bản chất hệ thống: một số luồng có graceful degradation, một số luồng phải fail fast để tránh sai dữ liệu.

Hìnhve/chuong5/resilience-evidence.png

Hình 5.12: Bằng chứng resilience: Kafka outbox replay và Redis degradation/recovery

Bốn kịch bản trên xác nhận rằng các cơ chế chịu lỗi đã thiết kế hoạt động đúng như kỳ vọng trong phạm vi single-host. Việc đối chiếu mức độ hoàn thành các mục tiêu của đồ án và các hạn chế còn tồn tại được trình bày tập trung trong phần Kết luận và hướng phát triển.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết luận

Đồ án đã hoàn thành mục tiêu xây dựng một hệ thống thương mại điện tử theo kiến trúc microservices, đi trọn vẹn từ phân tích yêu cầu, thiết kế, hiện thực hóa, đóng gói, triển khai cho đến kiểm thử trên môi trường gần thực tế. Hệ thống gồm 13 service nghiệp vụ và 3 service hạ tầng Spring Cloud, xây dựng trên Java 21, Spring Boot 3.5 và Spring Cloud 2025, đáp ứng đầy đủ sáu nhóm mục tiêu đặt ra ở Chương 1.

Về kiến trúc, hệ thống được phân rã thành các Bounded Context độc lập theo nguyên tắc Domain-Driven Design, mỗi service sở hữu cơ sở dữ liệu riêng và chỉ giao tiếp qua API hoặc sự kiện Kafka, với API Gateway làm cổng vào duy nhất, Eureka khám phá dịch vụ và Config Server quản lý cấu hình tập trung. Các bài toán khó của giao dịch phân tán được giải quyết bằng những mẫu thiết kế phù hợp: Saga Orchestration điều phối luồng đặt hàng, Transactional Outbox kết hợp Idempotent Consumer bảo đảm xử lý exactly-once ở tầng ứng dụng, và CQRS-lite với Elasticsearch cho tìm kiếm toàn văn. Bảo mật được tập trung tại Gateway theo Trusted Subsystem Pattern dựa trên Keycloak và JWT RS256, kết hợp rate limiter cho các luồng nhạy cảm. Hệ thống cũng tích hợp thanh toán VNPAY và một cơ chế flash sale chống over-sell đa lớp dựa trên Redis Lua atomic reservation.

Điểm nổi bật nhất về mặt kỹ thuật là cơ chế flash sale đa lớp và bảo đảm exactly-once ở tầng ứng dụng — cả hai đều được kiểm chứng bằng số liệu thực nghiệm thay vì chỉ mô tả lý thuyết. Hệ thống được đóng gói bằng Docker, triển khai production-like trên AWS EC2 với quy trình build/push image tự động qua GitHub Actions/GHCR, và được trang bị một lớp quan sát hoàn chỉnh gồm Prometheus/Grafana cho số liệu, Zipkin cho truy vết phân tán và structured logging. Kết quả kiểm thử trên môi trường này cho thấy hệ thống vận hành ổn định: catalog soak đạt p95 61,68 ms với 220.956 request không lỗi, checkout stress giữ tỷ lệ tạo đơn 100%, flash-sale spike 500 VU mua đúng 100/100 slot không có duplicate buyer, và các kịch bản resilience xác nhận khả năng tự phục hồi qua outbox replay và graceful degradation. Toàn bộ hệ thống được hoàn thiện bằng giao diện Next.js gồm storefront và Admin Panel, đủ để demo một sản phẩm hoàn chỉnh.

Bên cạnh các kết quả đạt được, đồ án vẫn còn một số hạn chế cần ghi nhận trung thực. Môi trường triển khai mới ở mức single-host EC2 nên chưa có high availability; quy trình CI/CD mới dừng ở build/push/deploy cơ bản; frontend phục vụ tốt cho demo nhưng chưa hỗ trợ upload file qua object storage hay quản lý người

dùng đầy đủ qua Keycloak Admin API; event schema đang quản lý bằng module dùng chung thay vì schema registry; và hệ thống chưa có log aggregation cùng alerting ở mức production. Những hạn chế này chính là tiền đề cho các hướng phát triển tiếp theo.

Hướng phát triển

Hướng phát triển của đồ án tập trung vào ba nhóm chính. *Trước hết là hoàn thiện nền tảng vận hành:* chuyển từ single-host Docker Compose sang Kubernetes nhiều node để đạt high availability và tự co giãn theo tải, đồng thời nâng cấp CI/CD với security scan, quản lý secret chuyên dụng và chiến lược triển khai canary/blue-green, bổ sung log aggregation và alerting cho khả năng quan sát ở mức production.

Tiếp theo là mở rộng chức năng nghiệp vụ: tích hợp object storage (S3 hoặc MinIO) cho phép upload ảnh thật thay vì nhập URL, kết nối Admin Panel với Keycloak Admin API để quản lý người dùng hoàn chỉnh, tách một analytics service riêng theo mô hình CQRS, và xây dựng dịch vụ gợi ý sản phẩm dựa trên lịch sử mua hàng và hành vi duyệt.

Cuối cùng là nâng cấp nền tảng kỹ thuật cho quy mô lớn hơn: áp dụng schema registry và contract testing để quản lý hợp đồng sự kiện giữa các service, triển khai service mesh cho mTLS và observability ở tầng mạng, và mở rộng sang ứng dụng di động bằng cách tái sử dụng API Gateway cùng luồng xác thực OAuth 2.0 sẵn có của Keycloak.

TÀI LIỆU THAM KHẢO

- [1] M. Fowler and J. Lewis. “Microservices: A definition of this new architectural term,” Accessed: Jun. 1, 2026. [Online]. Available: <https://martinfowler.com/articles/microservices.html>.
- [2] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA: O’Reilly Media, 2021.
- [3] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley Professional, 2003.
- [4] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications, 2018.
- [5] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O’Reilly Media, 2017.
- [6] H. Garcia-Molina and K. Salem, “Sagas,” in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, 1987, pp. 249–259.
- [7] Redis Ltd. “Redis documentation — scripting with lua,” Accessed: Jun. 1, 2026. [Online]. Available: <https://redis.io/docs/latest/develop/interact/programmability/eval-intro/>.
- [8] M. B. Jones, J. Bradley, and N. Sakimura. “RFC 7519: JSON web token (JWT),” Internet Engineering Task Force (IETF), Accessed: Jun. 1, 2026. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [9] M. Fowler. “CQRS — command query responsibility segregation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://martinfowler.com/bliki/CQRS.html>.
- [10] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC: Pragmatic Bookshelf, 2018.
- [11] Resilience4j. “Resilience4j user guide,” Accessed: Jun. 1, 2026. [Online]. Available: <https://resilience4j.readme.io/docs>.
- [12] VMware Tanzu. “Spring boot reference documentation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/index.html>.
- [13] VMware Tanzu. “Spring cloud documentation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://spring.io/projects/spring-cloud>.
- [14] N. Narkhede, G. Shapira, and T. Palino, *Kafka: The Definitive Guide*. Sebastopol, CA: O’Reilly Media, 2017.

- [15] The PostgreSQL Global Development Group. “Postgresql 17 documentation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://www.postgresql.org/docs/17/>.
- [16] Elasticsearch B.V. “Elasticsearch guide,” Accessed: Jun. 1, 2026. [Online]. Available: <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>.
- [17] Red Hat, Inc. “Keycloak server administration guide,” Accessed: Jun. 1, 2026. [Online]. Available: <https://www.keycloak.org/documentation>.
- [18] D. Hardt. “RFC 6749: The OAuth 2.0 authorization framework,” Internet Engineering Task Force (IETF), Accessed: Jun. 1, 2026. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6749>.
- [19] Vercel, Inc. “Next.js documentation — app router,” Accessed: Jun. 1, 2026. [Online]. Available: <https://nextjs.org/docs>.
- [20] Docker, Inc. “Docker documentation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.docker.com/>.
- [21] VNPAY. “Vnpay payment gateway — tài liệu tích hợp dành cho lập trình viên,” Accessed: Jun. 1, 2026. [Online]. Available: <https://sandbox.vnpayment.vn/apis/>.
- [22] Prometheus Authors. “Prometheus documentation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://prometheus.io/docs/introduction/overview/>.
- [23] OpenZipkin. “Zipkin — distributed tracing system,” Accessed: Jun. 1, 2026. [Online]. Available: <https://zipkin.io/>.
- [24] Grafana Labs. “Grafana k6 documentation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://grafana.com/docs/k6/latest/>.

PHỤ LỤC

A. HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH

Phụ lục này cung cấp các lệnh vận hành chính dùng để biên dịch, khởi động, kiểm tra và dừng hệ thống trong môi trường local và môi trường AWS EC2 dạng production-like.

A.1. Yêu cầu hệ thống

- **Java:** OpenJDK 21.
- **Docker:** Docker Desktop 27+ hoặc Docker Engine kèm Docker Compose plugin.
- **Maven:** Không cần cài riêng vì dự án dùng Maven Wrapper `./mvnw`.
- **Node.js:** Node.js 20+ cho frontend Next.js.
- **RAM:** Tối thiểu 12GB khả dụng cho Docker, khuyến nghị 16GB.
- **Disk:** Tối thiểu 10GB cho Docker images, volumes và build artifacts.

A.2. Biên dịch backend

Từ thư mục gốc của dự án, thực thi:

```
./mvnw clean package -DskipTests
```

Lệnh này biên dịch toàn bộ 17 module Maven theo thứ tự phụ thuộc và đóng gói các service thành file JAR thực thi. Khi cần chạy test Maven đầy đủ, có thể bỏ tham số `-DskipTests` hoặc chạy theo từng module/suite để giảm tải tài nguyên.

A.3. Chạy backend và hạ tầng local

```
docker compose up -d --build
```

Lệnh trên khởi động PostgreSQL, Redis, Kafka, Elasticsearch, Keycloak, Mailpit, Prometheus, Grafana, Zipkin, ba Spring Cloud service và 13 business service. Các lần chạy đầu tiên có thể mất vài phút do cần build image và tải dependency.

A.4. Chạy frontend local

Frontend nằm trong thư mục `frontend/`. Khi cần chạy storefront hoặc Admin Panel ở chế độ development:

```
cd frontend
npm install
npm run dev
```

Mặc định Next.js dùng `http://localhost:3000`. Nếu Grafana local cũng đang dùng cổng 3000, cần đổi cổng một trong hai thành phần hoặc dùng cấu hình

production-like nơi Grafana được remap sang host port 3001.

A.5. Kiểm tra trạng thái local

```
# Xem trạng thái container
docker compose ps
```

```
# Xem log Gateway hoặc order-service
docker compose logs -f api-gateway
docker compose logs -f order-service
```

```
# Kiểm tra health Gateway
curl http://localhost:8080/actuator/health/readiness
```

```
# Kiểm tra Eureka registry
curl -u eureka:eureka http://localhost:8761/eureka/apps
```

A.6. Các URL local quan trọng

Bảng A.1: Các URL truy cập hệ thống trên môi trường local

Thành phần	URL	Ghi chú
Frontend	http://localhost:3000	Storefront/Admin Panel khi chạy Next.js dev
API Gateway	http://localhost:8080	Cổng vào backend
Swagger UI	http://localhost:8080/swagger-ui.html	Tài liệu API qua Gateway
Eureka Dashboard	http://localhost:8761	user: eureka, pass: eureka
Keycloak Admin	http://localhost:8180	Quản trị realm
Grafana	http://localhost:3000 hoặc 3001	Phụ thuộc cấu hình chạy frontend/-Grafana
Zipkin	http://localhost:9411	Distributed tracing
Prometheus	http://localhost:9090	Metrics
Mailpit	http://localhost:8025	Kiểm tra email
Elasticsearch	http://localhost:9200	Search engine

A.7. Chạy smoke test backend

```
.test/scripts/smoke-backend.sh --suite maven
```

```
.test/scripts/smoke-backend.sh --suite infra
.test/scripts/smoke-backend.sh --suite auth-user
.test/scripts/smoke-backend.sh --suite catalog-search
.test/scripts/smoke-backend.sh --suite cart-voucher
.test/scripts/smoke-backend.sh --suite order-cod-review
.test/scripts/smoke-backend.sh --suite vnpay
.test/scripts/smoke-backend.sh --suite flash-sale
.test/scripts/smoke-backend.sh --suite security
```

Kết quả final của các suite đã được lưu trong thư mục `.test/results` và được tổng hợp ở Chương 5.

A.8. Chạy kiểm thử hiệu năng bằng k6

Ví dụ chạy catalog soak hoặc checkout stress với gateway production-like:

```
k6 run -e BASE_URL=http://13.213.118.96:8080 .test/load/catalog-
  browse.js
k6 run -e BASE_URL=http://13.213.118.96:8080 .test/load/checkout-
  stress.js
```

Khi đưa số liệu vào báo cáo, chỉ dùng các artifact raw đã lưu trong `.test/results` để tránh ghi số liệu không đối chiếu được.

A.9. Vận hành EC2 production-like

Khi cần bật EC2 để demo:

```
source aws/config.env
aws ec2 start-instances --instance-ids $INSTANCE_ID
aws ec2 wait instance-running --instance-ids $INSTANCE_ID

# Doi 2-3 phut cho services khoi dong
ssh -i $SSH_KEY_PATH ubuntu@$ELASTIC_IP "docker compose ps"
```

Khi demo xong, nên tắt instance để tiết kiệm chi phí/credit:

```
source aws/config.env
aws ec2 stop-instances --instance-ids $INSTANCE_ID
```

A.10. Dừng hệ thống local

```
# Dừng và giữ dữ liệu
docker compose down
```

```
# Dừng và xóa toàn bộ dữ liệu volume
docker compose down -v
```

B. ĐẶC TẢ USE CASE

Phụ lục này cung cấp đặc tả chi tiết cho các use case nghiệp vụ chính của hệ thống. Mỗi use case được mô tả theo cấu trúc: tên use case, tác nhân chính, điều kiện tiên quyết, luồng sự kiện chính, luồng sự kiện thay thế và điều kiện hậu kiểm.

B.1. Đăng ký tài khoản

Tên use case	Đăng ký tài khoản người dùng
Tác nhân	Khách hàng (Guest)
Điều kiện tiên quyết	Người dùng chưa có tài khoản trong hệ thống
Luồng chính	1. Người dùng gửi yêu cầu đăng ký với email, mật khẩu và thông tin cá nhân 2. Identity-service gọi Keycloak Admin API tạo user 3. Identity-service phát hành sự kiện <code>user-registered</code> lên Kafka 4. User-service tiêu thụ sự kiện, tạo hồ sơ người dùng 5. Hệ thống trả về thông báo đăng ký thành công
Luồng thay thế	A1. Email đã tồn tại: trả về lỗi 409 Conflict A2. Mật khẩu không đạt yêu cầu: trả về lỗi 400 Bad Request
Hậu kiểm	User tồn tại trong Keycloak, hồ sơ user được tạo trong <code>user_db</code>

B.2. Đặt hàng COD

Tên use case	Đặt hàng thanh toán khi nhận hàng (COD)
Tác nhân	Khách hàng đã đăng nhập
Điều kiện tiên quyết	Người dùng đã đăng nhập, giỏ hàng không rỗng, sản phẩm còn tồn kho
Luồng chính	1. Khách hàng gửi yêu cầu tạo đơn hàng (paymentMethod=COD, danh sách sản phẩm, địa chỉ) 2. Order-service kiểm tra sản phẩm, tồn kho, voucher qua Feign 3. Order-service tạo Order (PENDING) và OutboxEvent trong cùng transaction 4. Inventory-service đặt chỗ tồn kho khi nhận order-created 5. Order-service xác nhận đơn hàng (CONFIRMED) khi nhận inventory-updated 6. Inventory-service xác nhận trừ tồn kho thực tế 7. Notification-service gửi email xác nhận đơn hàng
Luồng thay thế	A1. Tồn kho không đủ: Inventory phát hành inventory-failed, Order chuyển CANCELLED A2. Voucher không hợp lệ: Order-service trả lỗi, không tạo đơn
Hậu kiểm	Order status = CONFIRMED, tồn kho đã bị trừ, email đã được gửi

B.3. Thanh toán VNPAY

Tên use case	Thanh toán trực tuyến qua VNPAY
Tác nhân	Khách hàng đã đăng nhập, Hệ thống VNPAY
Điều kiện tiên quyết	Đơn hàng đã được tạo với paymentMethod=VNPAY, trạng thái STOCK_RESERVED
Luồng chính	1. Khách hàng yêu cầu tạo URL thanh toán 2. Payment-service tạo payment (PENDING), sinh URL với chữ ký HMAC-SHA512 3. Khách hàng được chuyển hướng đến trang thanh toán VNPAY 4. Khách hàng nhập thông tin thẻ và xác nhận thanh toán 5. VNPAY gọi IPN callback với kết quả thành công 6. Payment-service xác minh chữ ký, cập nhật payment COMPLETED 7. Order-service nhận payment-success, chuyển order CONFIRMED 8. Inventory xác nhận tồn kho, Notification gửi email
Luồng thay thế	A1. Thanh toán thất bại: VNPAY trả về mã lỗi, payment chuyển FAILED, order CANCELLED A2. Quá thời hạn 30 phút: Scheduler tự động hủy đơn và giải phóng tồn kho A3. Chữ ký không hợp lệ: Payment-service từ chối, không cập nhật trạng thái
Hậu kiểm	Payment status = COMPLETED, Order status = CONFIRMED

B.4. Tham gia Flash Sale

Tên use case	Mua hàng trong sự kiện Flash Sale
Tác nhân	Khách hàng đã đăng nhập
Điều kiện tiên quyết	Chiến dịch đang ở trạng thái ACTIVE, trong thời gian diễn ra, còn tồn kho
Luồng chính	1. Khách hàng gửi yêu cầu mua hàng <code>POST/api/flash-sales/{id}/purchase</code> 2. Flash-sale-service kiểm tra thời gian hiệu lực của chiến dịch 3. Thực thi Redis Lua script: kiểm tra stock, kiểm tra chưa mua, giảm stock 4. Nếu Redis trả về thành công, phát hành <code>flash-sale-order-requested</code> lên Kafka 5. Order-service tạo đơn hàng với giá flash sale 6. Xử lý Saga tương tự luồng COD
Luồng thay thế	A1. Hết hàng (Redis stock = 0): trả về HTTP 409 Conflict A2. Đã mua trước đó: Redis buyers set phát hiện, trả về HTTP 409 A3. Chiến dịch chưa bắt đầu/đã kết thúc: trả về HTTP 400 A4. Tạo đơn thất bại sau khi giữ slot: compensation hoàn Redis stock và buyer set
Hậu kiểm	Đơn hàng flash sale được tạo, không vượt quá số lượng, không trùng người mua

B.5. Tìm kiếm sản phẩm

Tên use case	Tìm kiếm sản phẩm toàn văn
Tác nhân	Khách hàng (Guest hoặc đã đăng nhập)
Điều kiện tiên quyết	Elasticsearch đã được đồng bộ dữ liệu sản phẩm
Luồng chính	1. Người dùng gửi yêu cầu GET /api/search?keyword=X&category=Y&minPrice=Z 2. Search-service xây dựng Boolean Query: full-text match + term filter + range filter 3. Truy vấn Elasticsearch index products 4. Trả về danh sách sản phẩm phù hợp kèm phân trang
Luồng thay thế	A1. Không có kết quả: trả về danh sách rỗng A2. Elasticsearch không khả dụng: trả về lỗi 503 Service Unavailable
Hậu kiểm	Kết quả tìm kiếm được trả về với thời gian phản hồi dưới 100ms